



第8章 流水线中央处理器

突破单周期物理极限的方法



主要内容

- 第一讲 CPU概述
- 第二讲 单周期CPU设计
- 第三讲 多周期CPU设计
- 第四讲 流水线CPU设计
- 第五讲 流水线冒险及其处理
- 第六讲 高级流水线技术



主要内容

- 日常生活中的流水线处理例子：洗衣服
- 单周期处理器模型和流水线性能比较
- 什么样的指令集适合于流水线方式执行
- 如何设计流水线数据通路
 - 以RV32I子集来说明
- 如何设计流水线控制逻辑
 - 分析每条指令执行过程中的控制信号
 - 给出控制器设计过程
- 流水线冒险的概念



指令流水线原理与数据通路

突破单周期物理极限的方法



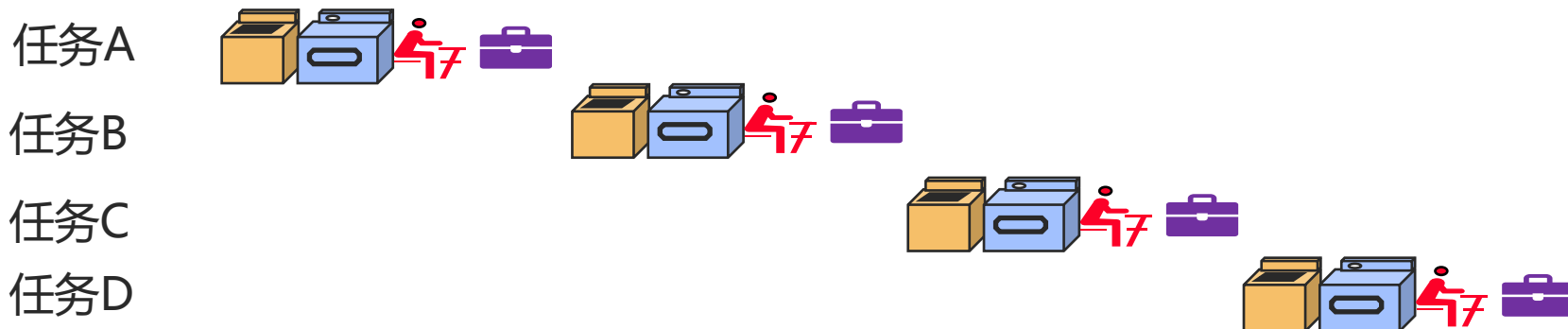
单指令延迟可能变长，但整体吞吐量呈指数级飞跃。



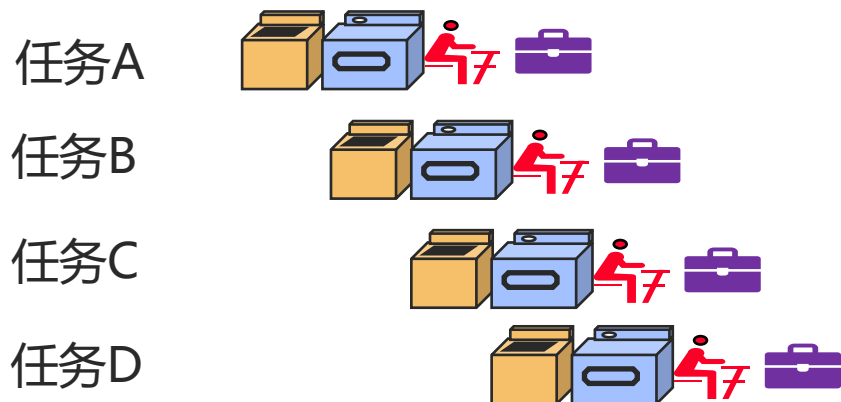
工业革命的启示：流水线

■ 洗衣店工作流程为例

单周期



流水线



核心法则：

- 流水线不改变单个任务的处理时间，但极大提升了系统的吞吐率。

加速比极限：

- 理想加速比 = 流水线级数N。



回顾：Load指令的5个阶段

阶段1	阶段2	阶段3	阶段4	阶段5
Ifetch	Reg/Dec	Exec	Mem	Wr

- Ifetch (IF取指)：取指令并计算PC+4 (用到哪些部件?)

指令存储器、Adder

- Reg/Dec (ID取数和译码)：取数同时译码 (用到哪些部件?)

指令译码器、寄存器堆读口、扩展器

- Exec (EX执行)：计算内存单元地址 (用到哪些部件?)

ALU

- Mem (M读存储器)：从数据存储器中读 (用到哪些部件?)

数据存储器

- Wr(WB写寄存器)：将数据写到寄存器中 (用到哪些部件?)

寄存器堆写口

这里寄存器堆的读口和写口可看成两个不同的部件。

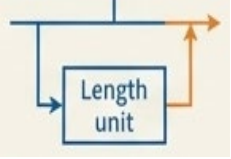
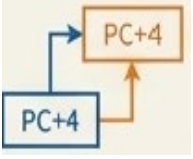
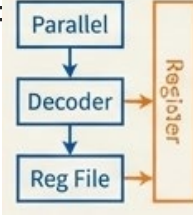
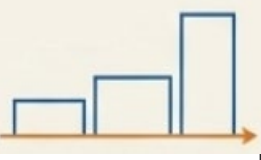
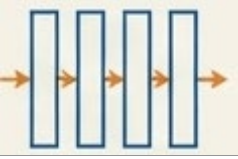
指令的执行过程是否和“洗衣”过程类似？

是否可以采用类似方式来执行指令呢？



为什么RISC-V天生适合流水线?

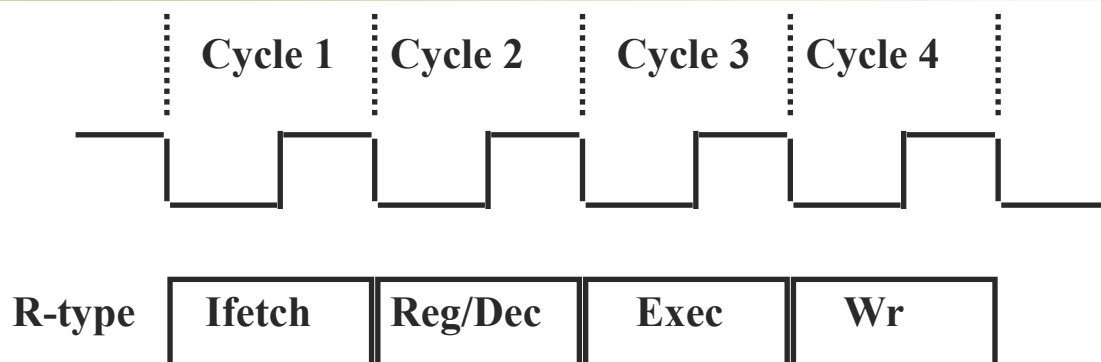
■ 什么特征的指令集有利于流水线执行呢?

指令集特征	CISC架构	RISC-V微架构
32位定长指令	长度可变 (如x86), 取指段无法预测在哪断句, 必须等取完才能解。 	极速IF段: PC永远直接+4,不需要复杂的边界判定逻辑。 
源寄存器位置固定	位置乱变, 必须等译码彻底结束, 才能去寄存器组寻址取数据。 	极速ID段: 译码操作与读取寄存器堆可以同时进行! 
Load/Store架构	运算指令若可直连内存, 各个阶段长短极度失衡, “木桶短板”极其严重。 	规整化: 强制所有指令对齐 5级 物理段, 使流水线调度平滑稳定。 

Tips: 好的指令集不是单纯写出来的代码, 它是为硬件的物理流动量身定制的规范。



R和I型指令的流水段分析



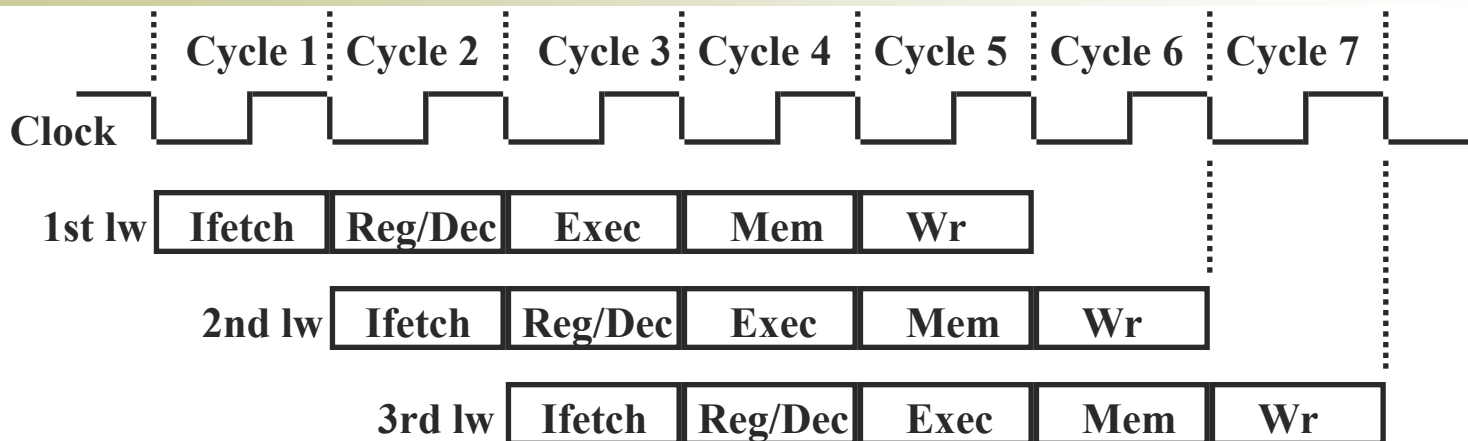
■ R-型指令功能段划分

- Ifetch: 取指令并计算PC+4
- Reg/Dec: 从寄存器取数, 同时指令在译码器进行译码
- Exec: 在ALU中对操作数进行计算
- Wr: ALU计算的结果写到寄存器

■ I-型指令 (除load类指令外) 和U-指令, 都不涉及存储器的读取, 他们的功能段划分和R-型指令相同。



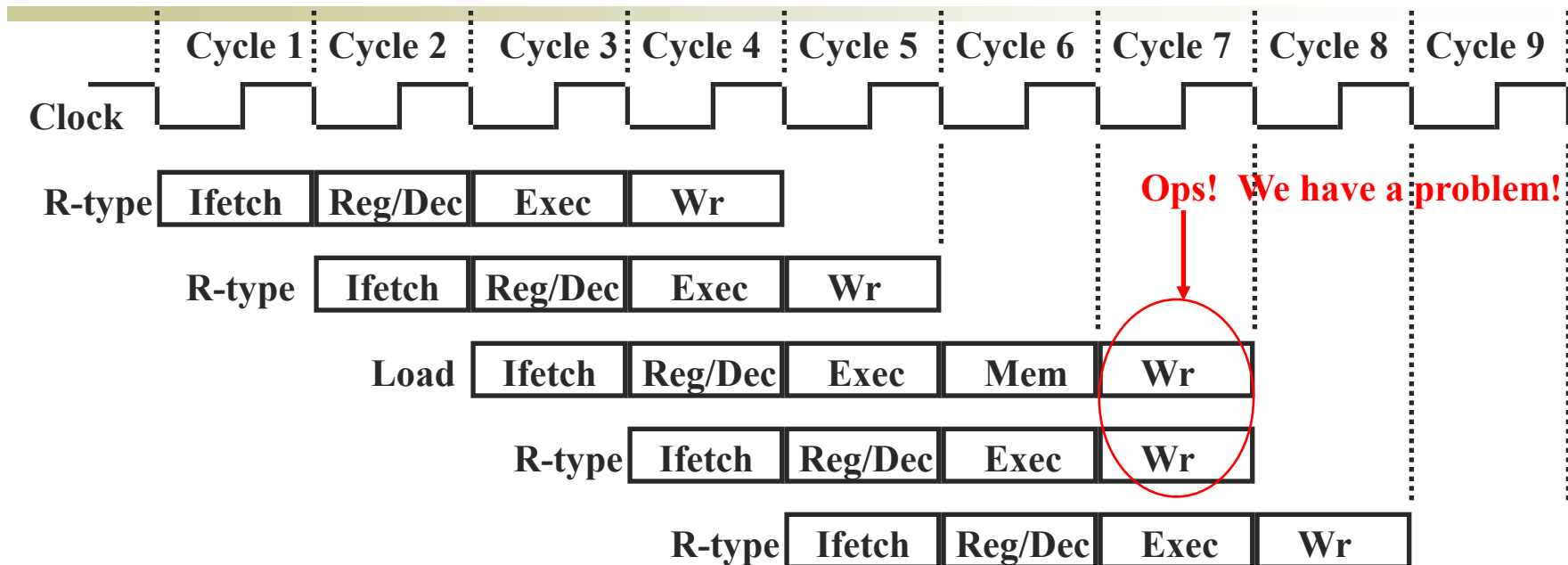
Load类指令的流水段分析



- load 指令的功能为: $R[Rd] \leftarrow M[R[Rs1] + SEXT(imm12)]$
 - 每个周期有五个功能部件同时在工作
 - 后面指令在前面完成取指后马上开始
 - 每个load指令需要五个周期完成
 - 吞吐率(throughput)提高许多, 理想情况下, 有:
 - ◆ 每个周期有一条指令进入流水线
 - ◆ 每个周期都有一条指令完成
 - ◆ 每条指令的有效周期(CPI)为1



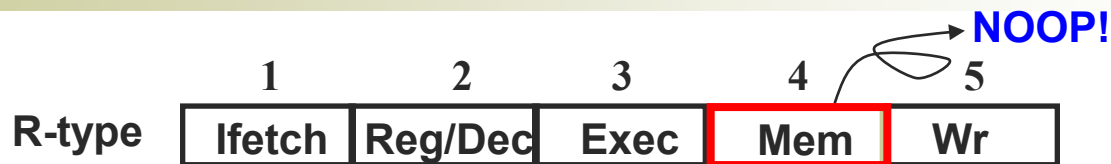
含 R-type 和 Load 指令的流水线



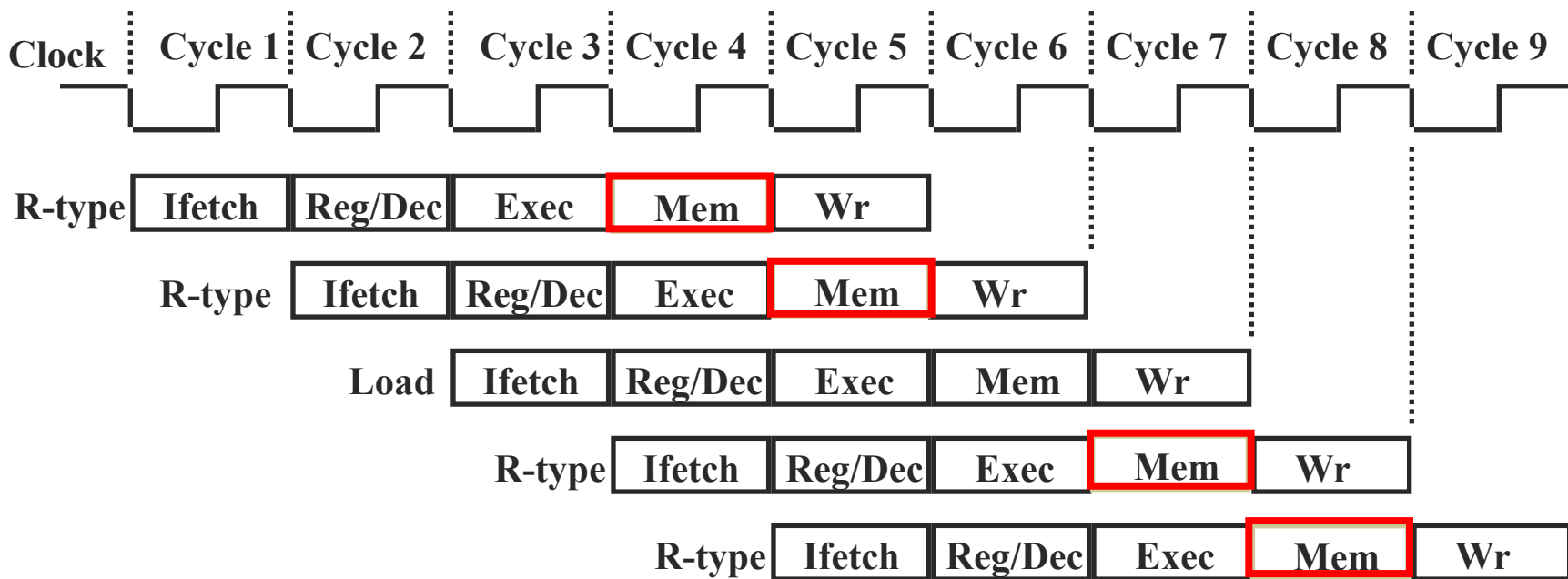
- 上述流水线有个问题: 两条指令试图同时写寄存器, 因为
 - Load 在第5阶段用寄存器写口
 - R-type 在第4 阶段用寄存器写口
- 把一个功能部件同时被多条指令使用的现象称为结构冒险(Structure Hazard)
- 为了流水线能顺利工作, 规定: 或称为资源冲突!
 - 每个功能部件每条指令只能用一次 (如: 写口不能用两次或以上)
 - 每个功能部件必须在相同的阶段被使用 (如: 写口总是在第五阶段被使用)



R-type的Wr操作延后一个周期执行



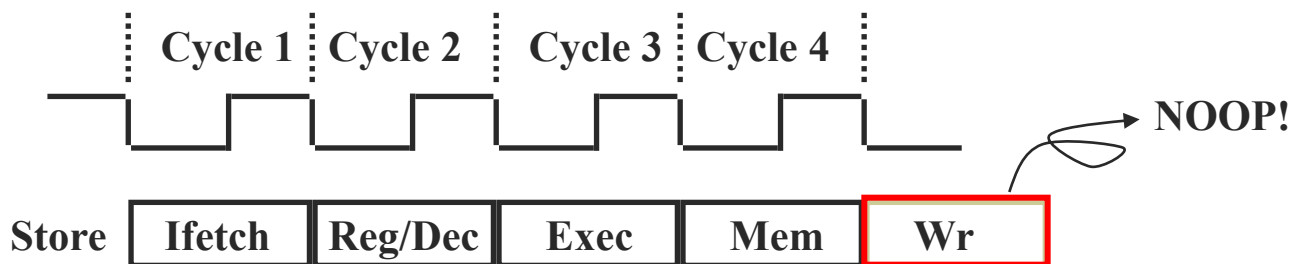
- 加一个NOP阶段以延迟“写”操作：
 - 把“写”操作安排在第5阶段, 这样使R-Type的Mem阶段为空NOP。
 - 如何实现空阶段NOP?



这样使流水线中的每条指令都有相同阶段个数!



Store指令的四个阶段



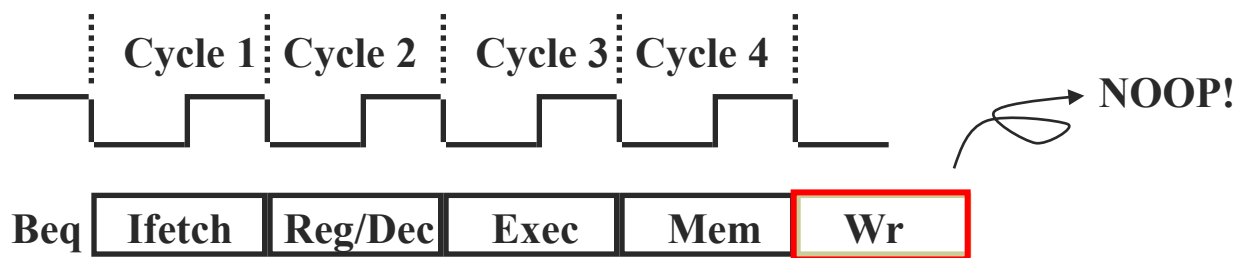
■ S - 型指令功能段划分

- **Ifetch**: 取指令并计算PC+4
- **Reg/Dec**: 从寄存器取数，同时指令在译码器进行译码
- **Exec**: 立即数符号扩展后与寄存器值相加，**计算主存地址**
- **Mem**: 将寄存器读出的数据**写到主存**

Wr: 加一个空的写阶段，使流水线更规整！



Beq的四个阶段



- **Ifetch**: 取指令并计算PC+4
- **Reg/Dec**: 从寄存器取数，同时指令在译码器进行译码
- **Exec**: 执行阶段 **B型指令**: 1.判断条件是否成立； 2.跳转目标地址。
 - ALU中比较两个寄存器的大小（做减法）
 - Adder中计算转移地址
- **Mem**: 如果比较相等, 则:
 - 转移目标地址写到PC
- **Wr**: 加一个空写阶段，使流水线更规整！

执行阶段需要两个组件！
ALU、加法器

可按照上述方式，把所有指令都按照最复杂的“load”指令所需的五个阶段来划分，不需要的阶段加一个“NOP”操作



4.2 指令的流水段分析

- J- 型指令功能段划分
- jal 指令是**无条件转移指令**，指令中给出了20 位相对地址，其功能是将**PC+4 送到寄存器Rd** 中，并且无条件将**转移目标地址**设置到PC 中。
- 转移目标地址： $PC \leftarrow PC + (\text{SEXT}(\text{imm20}) \times 2)$
 - Exec：在**ALU** 中计算**PC+4**，同时用一个**加法器**计算转移**目标地址**；
执行阶段需要两个组件！
ALU、加法器
 - Mem：用于将转移目标地址写到**PC** 中。
 - Wr：用于将ALU 中的计算结果写回寄存器**Rd**。



4.2 指令的流水段分析

- 流水线设计的原则是：
 - 指令流水段个数以最复杂指令所用的功能段数为准
 - 流水段的长度以最复杂的操作所用的时间为准。
- 最复杂的是lw指令，分为5个功能段，其他指令都可以通过加入“空”功能段来向lw 指令看齐。
- 在插入“空”段时，应遵循以下两个原则：
 - ①每个功能部件每条指令只能用一次（如寄存器写口不能用两次或以上）；
 - ②每个功能部件必须在相同的阶段被使用（如寄存器写口总是在第5 阶段被使用）。
- R-、I-（除load外）、U-型指令需在Wr 之前加一个空的Mem段。
- S -和B-型指令在第4 个功能段后加一个空的Wr 段。



4.3 数据通路的“外科手术”

- 流水段寄存器(Pipeline Registers)
 - 物理本质：插入在单周期组合逻辑部件之间的时序状态元件。
 - 核心功能：在时钟上升沿捕获前一阶段的全部输出，作为下一阶段的稳定输入。在空间上隔离时序，彻底防止相邻指令的数据流发生“追尾”。
- 流水段寄存器不仅要向后**传递数据**，还必须向后**传递控制信号**！
- 单周期中的控制信号，在流水线中必须被严格的时空隔离。
- 流水段寄存器都**写使能控制信号始终有效**。
 - 每个时钟都会改变PC的值。
 - 每个**流水段寄存器**在每个时钟都会**写入一次**。
 - **IF段**的功能每条指令都相同，是公共流水段，也不需控制信号。

多周期指令IF阶段的控制信号：IRWr=1,PCWr=1,Add2MUX=1,PCout=0

单周期在执行IF功能时有控制信号吗？



4.3 流水线数据通路的设计

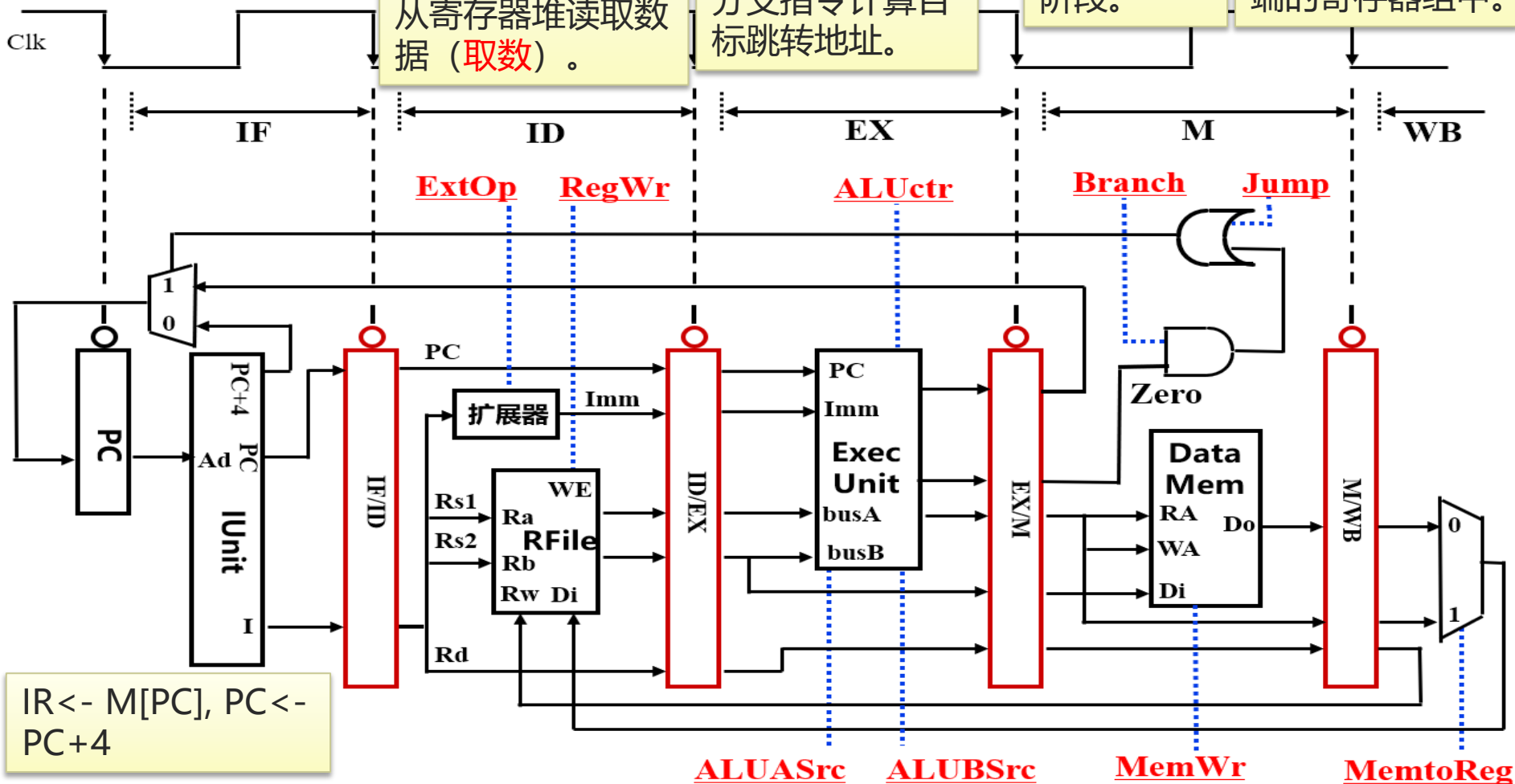
5 段流水线数

控制器生成全部控制信号（译码）；同时根据Rs1和Rs2从寄存器堆读取数据（取数）。

依据运算指令类型完成算术或逻辑运算；同时为分支指令计算目标跳转地址。

唯一与外部数据存储器交互的物理阶段。

将ALU的计算结果或Memory读取出的数据，跨越空间写回到前端的寄存器组中。



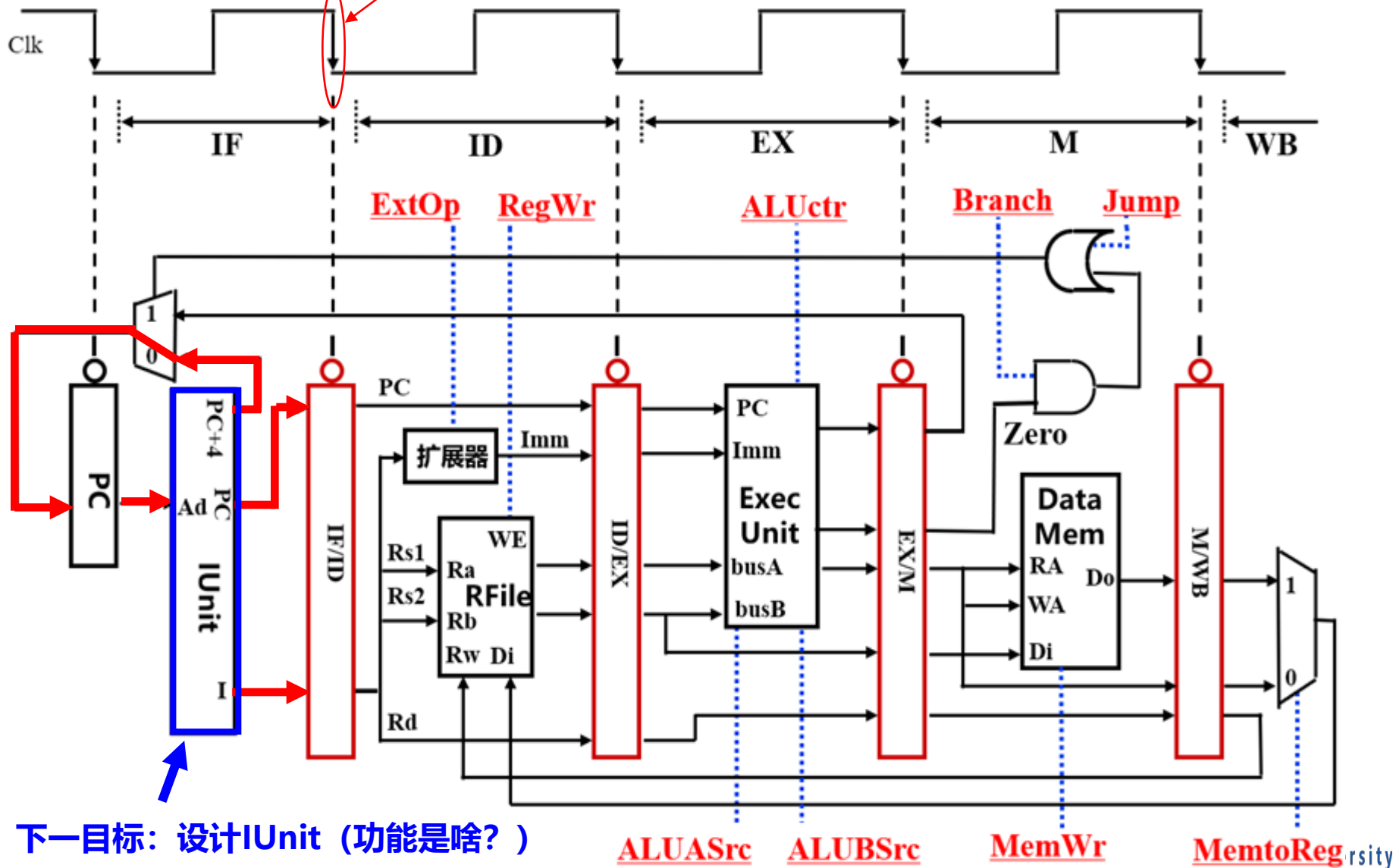


取指令 (IF) 阶段

第10单元指令: `lw x8, 0x100(x9)`

功能: $R[x8] \leftarrow M[R[x9] + \text{SEXT}(0x100)]$

You are here!





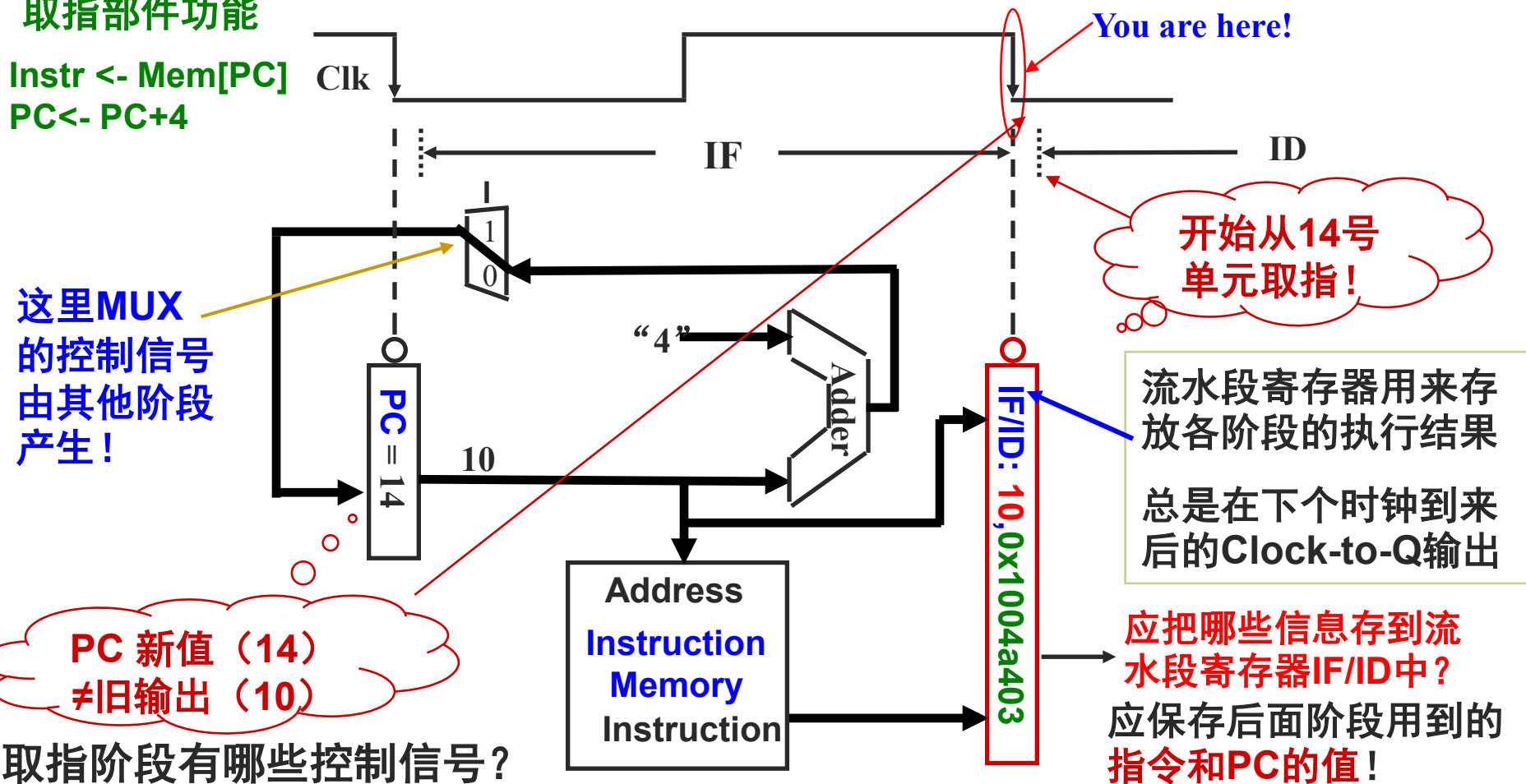
取指令部件 IUnit的设计

■ 第10单元指令 : lw x8, 0x100(x9)

随后的指令在14号单元中!

取指部件功能

$\text{Instr} \leftarrow \text{Mem}[\text{PC}]$
 $\text{PC} \leftarrow \text{PC} + 4$





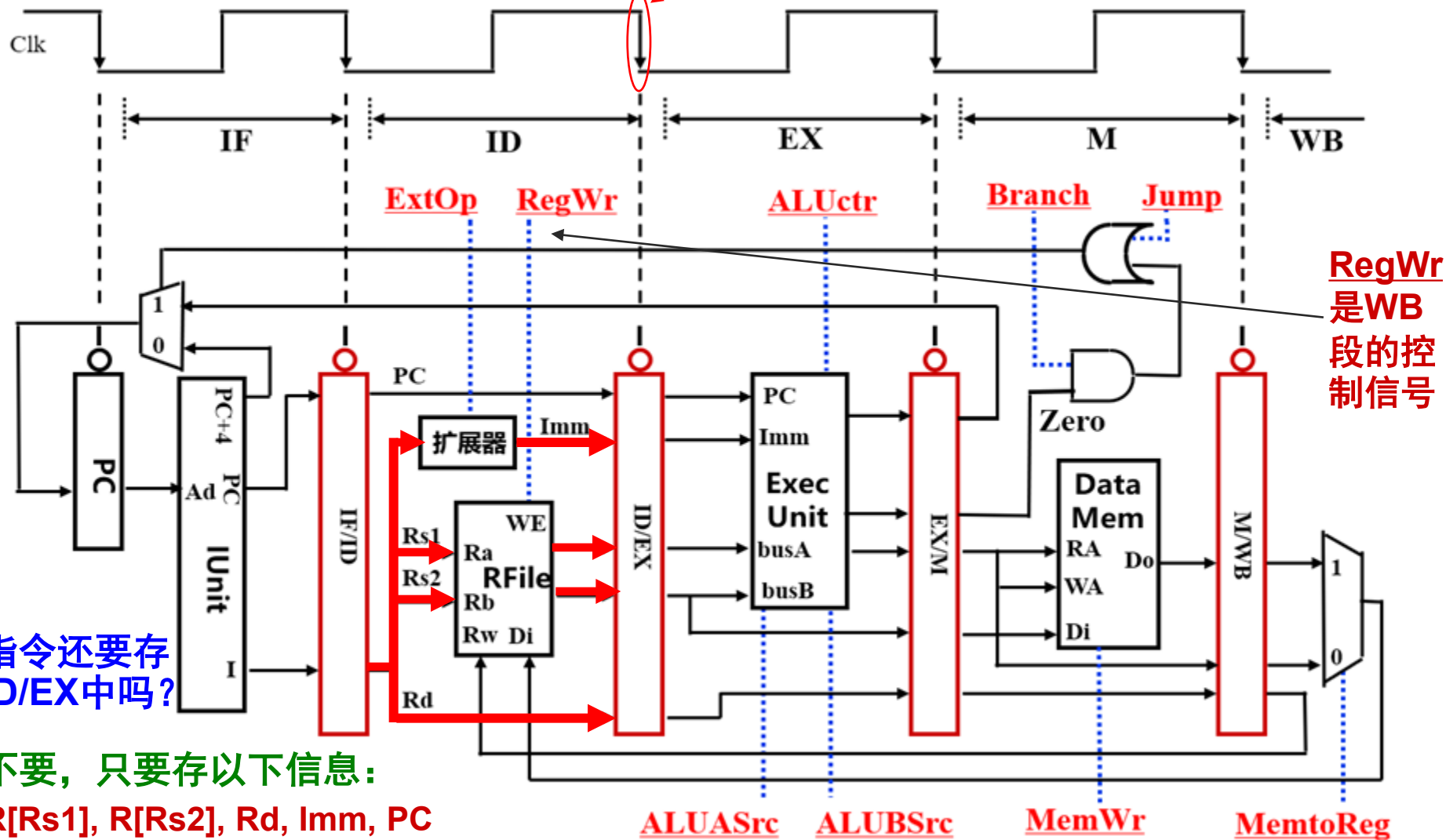
译码/取数 (Reg/Dec) 阶段

第10单元指令: lw x8, 0x100(x9)

功能: $R[x8] \leftarrow M[R[x9] + \text{SEXT}(0x100)]$

I-型立即数时, $\text{ExtOp}=000$ 。

You are here!



RegWr
是WB
段的控制
信号

指令还要存
ID/EX中吗?

不要, 只要存以下信息:

$R[Rs1]$, $R[Rs2]$, Rd , Imm , PC

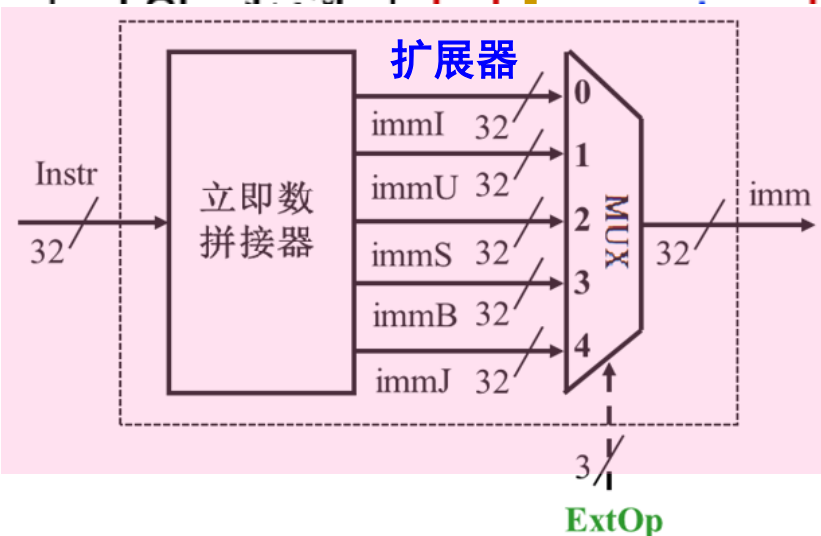
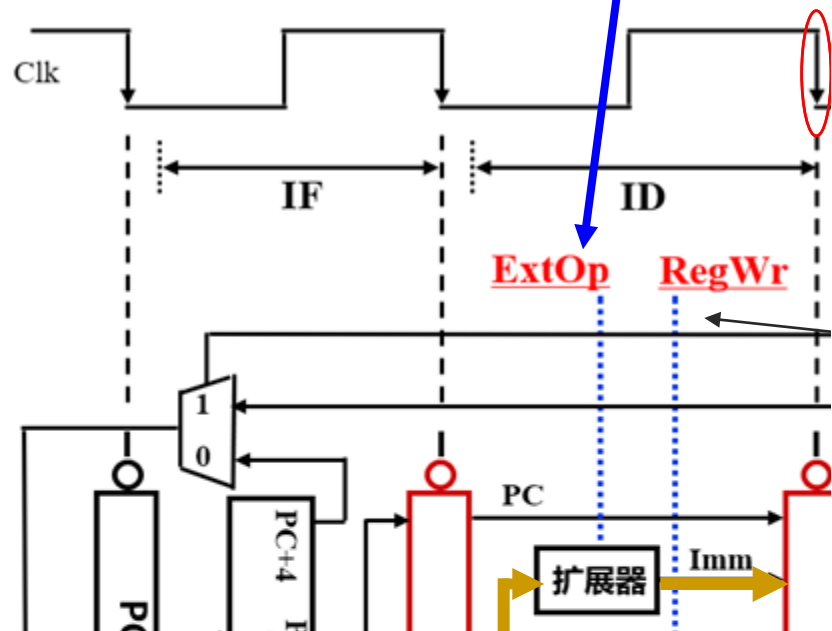
该阶段有哪些控制信号?

ExtOP 用于控制扩展器对立即数进行扩展。 $\text{ExtOP}=?$

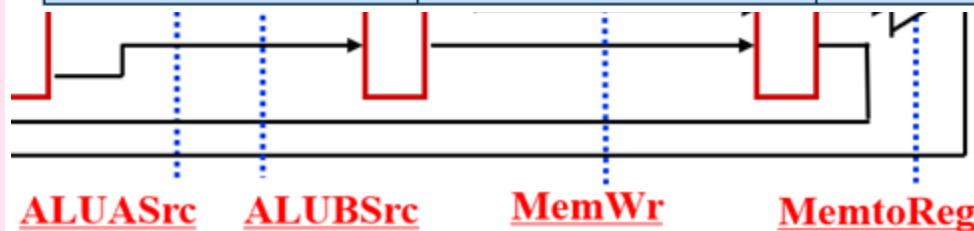


译码/取数

该阶段仅一个控制信号：ExtOp



指令	立即数编码类型	ExtOp<2:0>
add rd, rs1, rs2	无立即数	× × ×
slt rd, rs1, rs2		
sltu rd, rs1, rs2		
ori rd, rs1, imm12	I-型立即数 (immI)	0 0 0
lui rd, imm20	U-型立即数 (immU)	0 0 1
lw rd, rs1, imm12	I-型立即数 (immI)	0 0 0
sw rs1, rs2, imm12	S-型立即数 (immS)	0 1 0
beq rs1, rs2, imm12	B-型立即数 (immB)	0 1 1
jal rd, imm20	J-型立即数 (immJ)	1 0 0

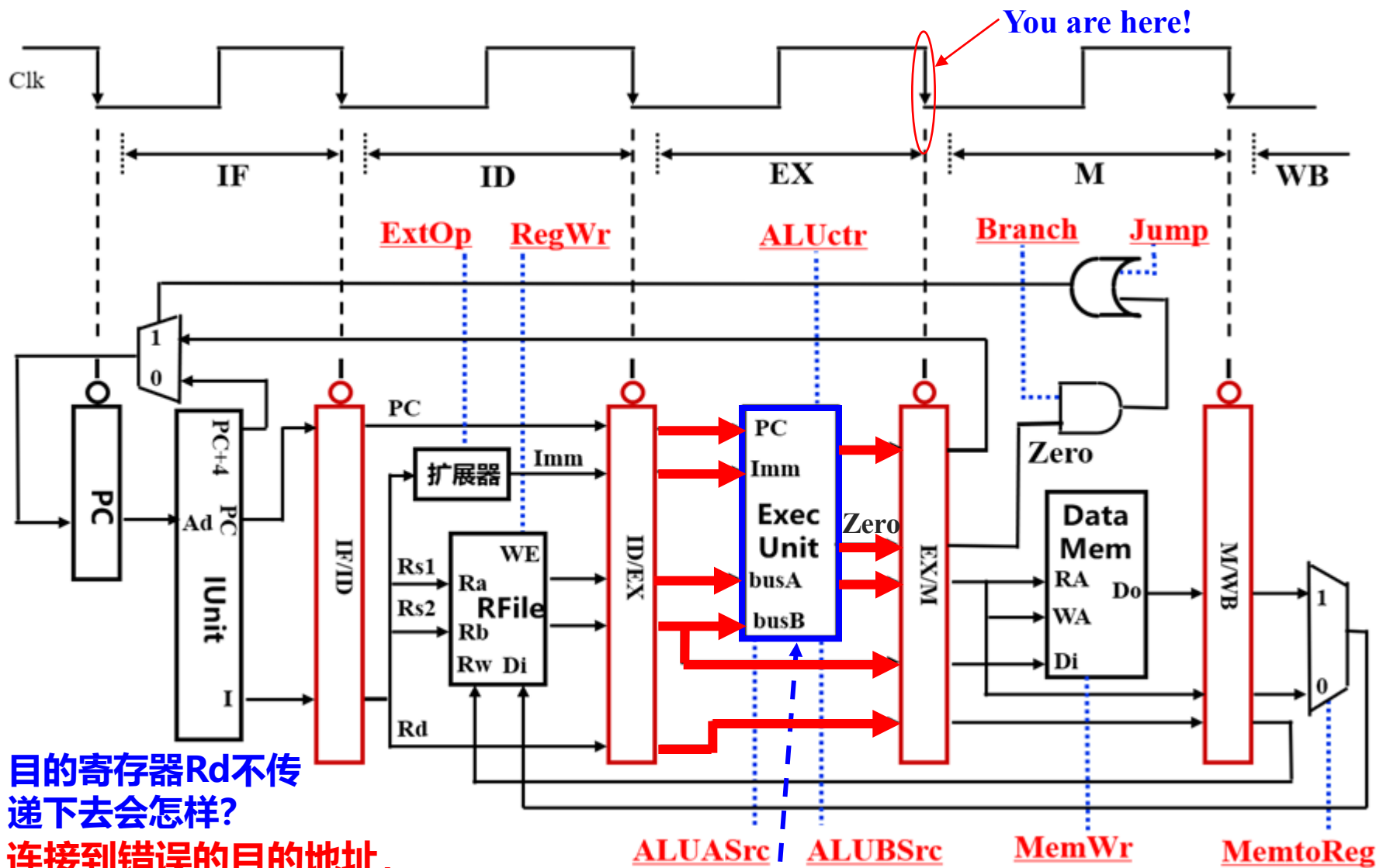




Load指令的地址计算阶段

第10单元指令: lw x8, 0x100(x9)

功能: $R[x8] \leftarrow M[R[x9] + \text{SEXT}(0x100)]$



目的寄存器Rd不传递下去会怎样?

连接到错误的目的地址,
指令执行错误!

下一目标: 设计执行部件(Exec Unit)



You are here!

增加计算转移目标地址的加法器

Target

32

Zero

ALUout

32

J-型指令呢？

ALUASrc=1

ALUBSrc=1

ALUctr=add

U-型指令呢？

ALUASrc=x

ALUBSrc=2

ALUctr=srcB

ALUASrc

ALUBSrc

ALUctr

jalr指令呢？

R型指令呢？

ALUASrc=0

ALUBSrc=0

ALUctr=与指令相关

B-型指令呢？

ALUASrc=0

ALUBSrc=0

ALUctr=sub

I-型指令呢？(除了jalr)

ALUASrc=0

ALUBSrc=2

ALUctr=与指令相关

Store指令呢？

与Load指令完全一样!

ALUASrc=0

ALUBSrc=2

ALUctr=add

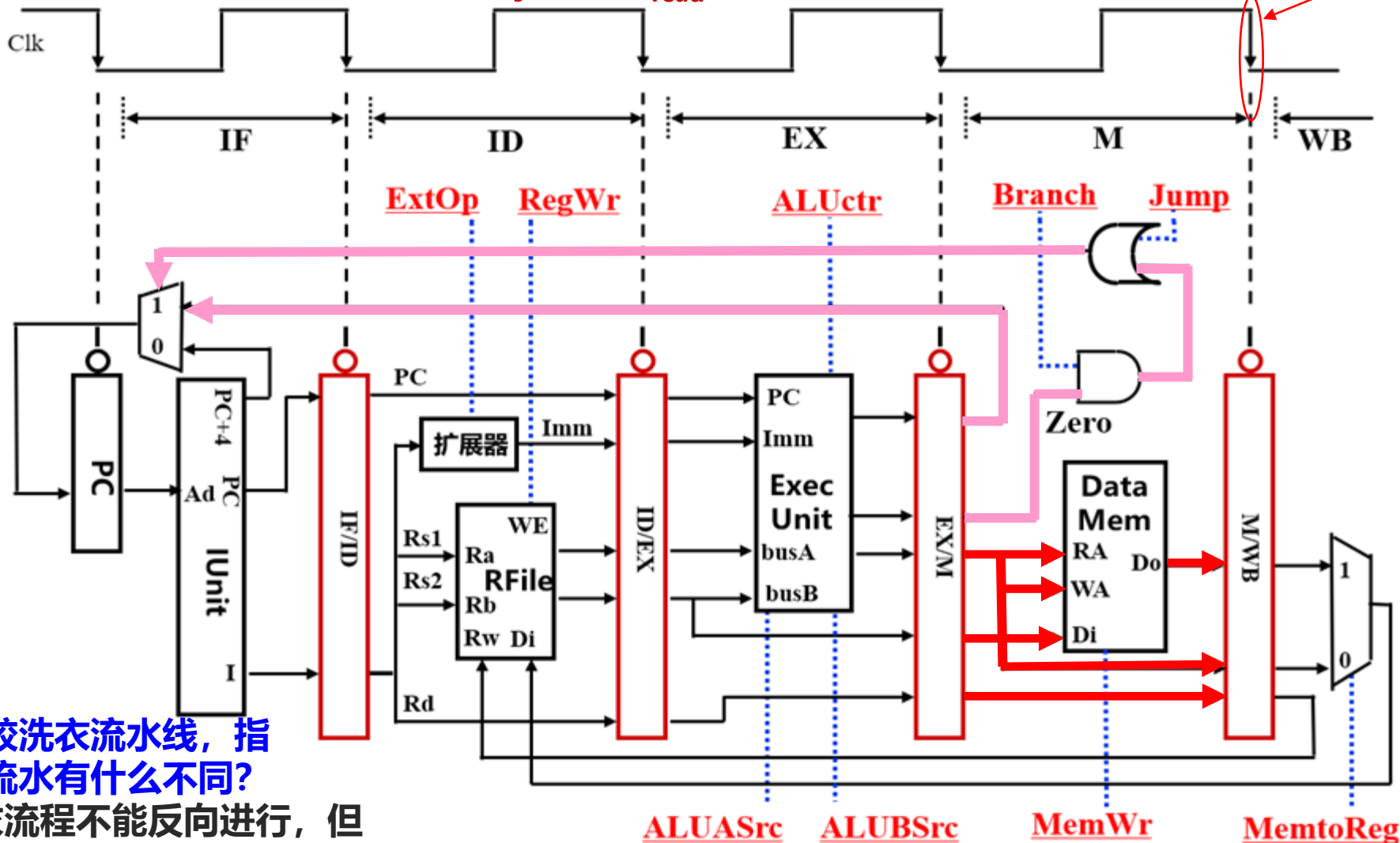


Load指令的存储器读(M)周期

第10单元指令: `lw x8, 0x100(x9)` 功能: $R[x8] \leftarrow M[R[x9] + \text{SEXT}(0x100)]$

以最长操作为准设计时钟周期: $\text{Cycle} > T_{\text{read}}$

You are here!



比较洗衣流水线, 指令流水有什么不同?

洗衣流程不能反向进行, 但该阶段有反向数据流, 可能会引起冒险! 以后介绍。

Load指令控制信号: $\text{Branch}=0$ 、 $\text{Jump}=0$ 、 $\text{MemWr}=0$

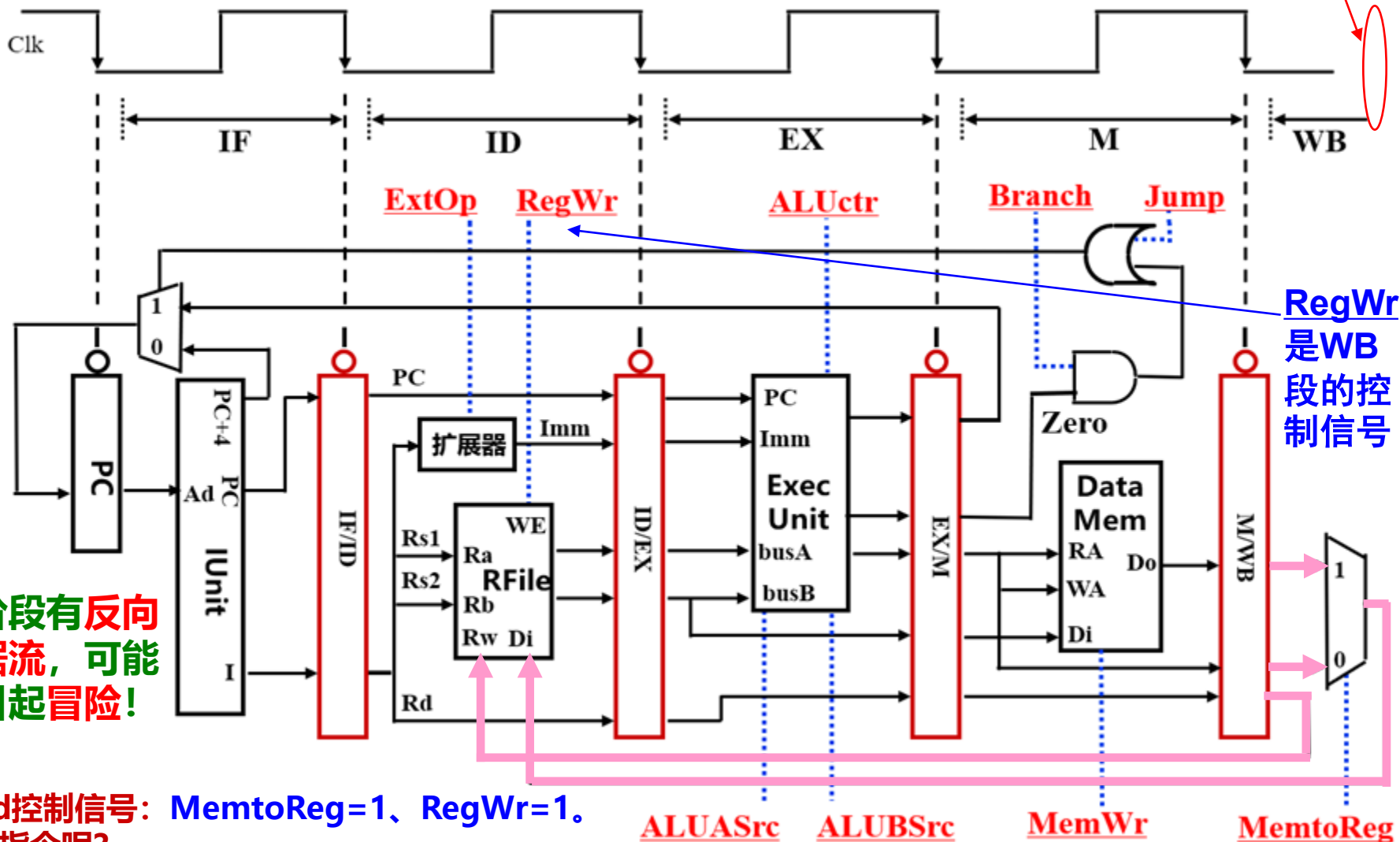
其他指令? B-型: $\text{Branch}=1$, J-型: $\text{Jump}=1$, S-型: $\text{MemWr}=1$



Load指令的回写 (Write Back) 阶段 You are here!

第10单元指令: `lw x8, 0x100(x9)`

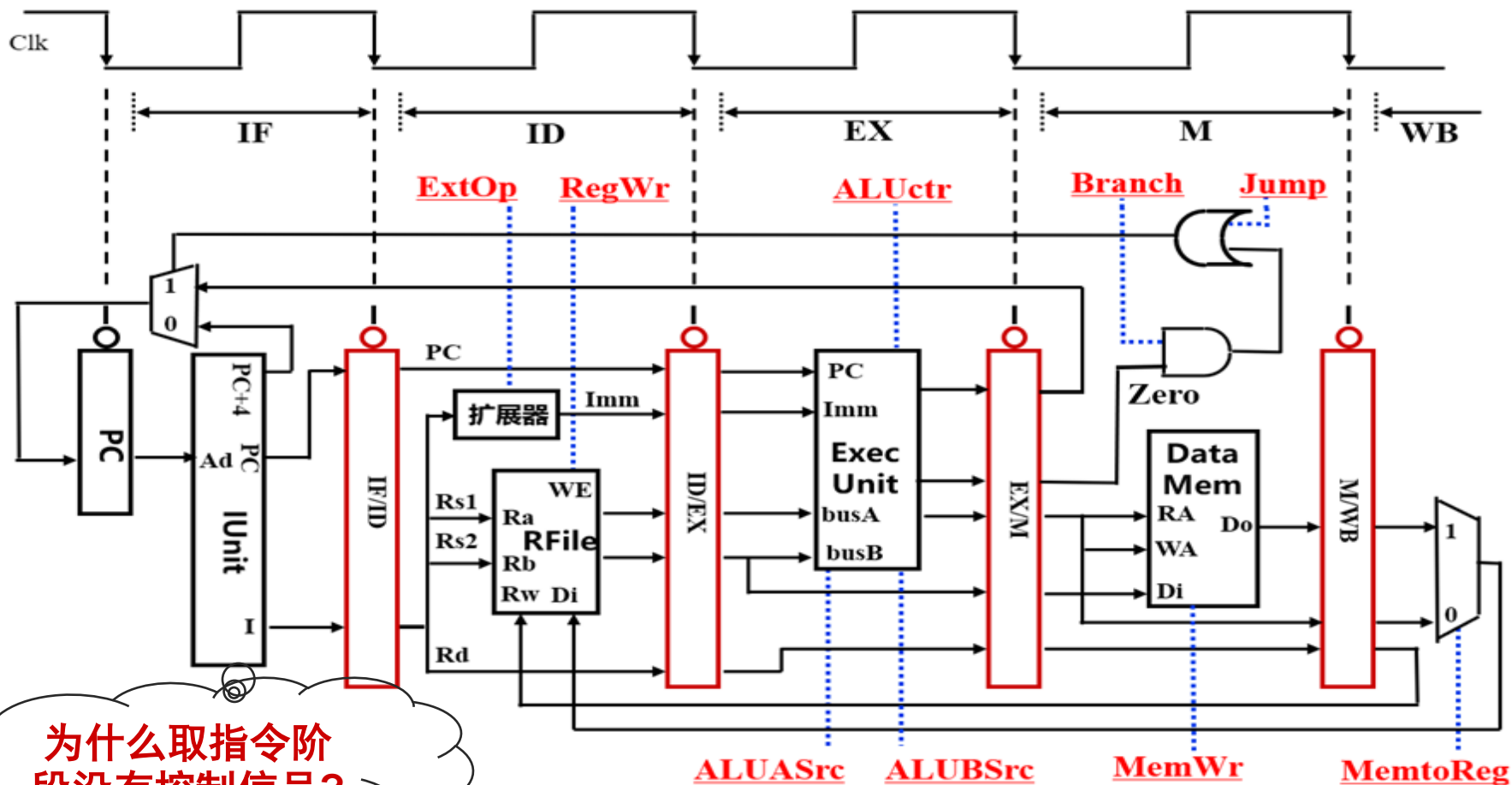
功能: $R[x8] \leftarrow M[R[x9] + \text{SEXT}(0x100)]$





流水线中的Control Signals如何获得?

- 主要考察: 第N阶段的控制信号, 它取决于某条指令的某个阶段。
 - $N = \text{ID、EX、M、WB}$ (只有这四个阶段有控制信号)
 - 例: Load的EX段的控制信号 = Func (Load' s EX)



为什么取指令阶段没有控制信号?

IF阶段的功能对每条指令来说都一样



流水线中的Control Signals

- 通过对前面流水线数据通路的分析，得知：
 - PC需要写使能控制信号吗？ **每个时钟都会改变PC，故不需要!**
 - 流水段寄存器需要写使能控制信号吗？
 - IF阶段没有控制信号 **每个时钟都会改变流水段寄存器，故不需要!**
 - ID阶段一个 ExtOp (扩展操作)：除R-型外的所有指令都需要控制信号
 - Exec阶段的控制信号有三个
 - ◆ ALUASrc (ALU的A口来源)：1- 来源于PC； 0- 来源于BusA
 - ◆ ALUBSrc (ALU的B口来源)：0- BusB； 1- “4”； 2- Imm
 - ◆ ALUctr (控制ALU执行不同的操作)：4位编码
 - Mem阶段的控制信号有三个
 - ◆ MemWr (DM的写信号)：Store指令时为1，其他指令为0
 - ◆ Branch (是否为B-型指令)：B-型指令时为1，其他指令为0
 - ◆ Jump(是否为J-型指令)：J-型指令时为1，其他指令为0
 - Wr阶段的控制信号有两个
 - ◆ MemtoReg (寄存器的写入源)：1- ALU输出； 0- DM输出
 - ◆ RegWr (寄存器堆写信号)：结果写寄存器的指令都为1，其他指令为0
- 需考虑所有指令的控制信号！**



控制逻辑 (Control) 的设计

■ 流水线控制逻辑的设计

- 每条指令的控制信号在该指令执行期间变不变?
(单周期和多周期时各是怎样的情况?) **不变!**
- 与单周期还是多周期的控制逻辑设计类似? **单周期!**
(单周期和多周期控制逻辑各是怎样设计的?)
- 设计过程
 - ◆ 用真值表建立指令和控制信号之间的关系
 - ◆ 写出每个控制信号的逻辑表达式
- 控制逻辑的输出 (**控制信号**) 在**ID阶段**生成, 并存放在**ID/EX流水段寄存器**中, 然后每来一个时钟跟着指令传送到下一级流水段寄存器
- 某时刻在不同阶段同时执行不同指令, 不同的指令得到不同控制信号

复习单周期和多周期控制器设计!

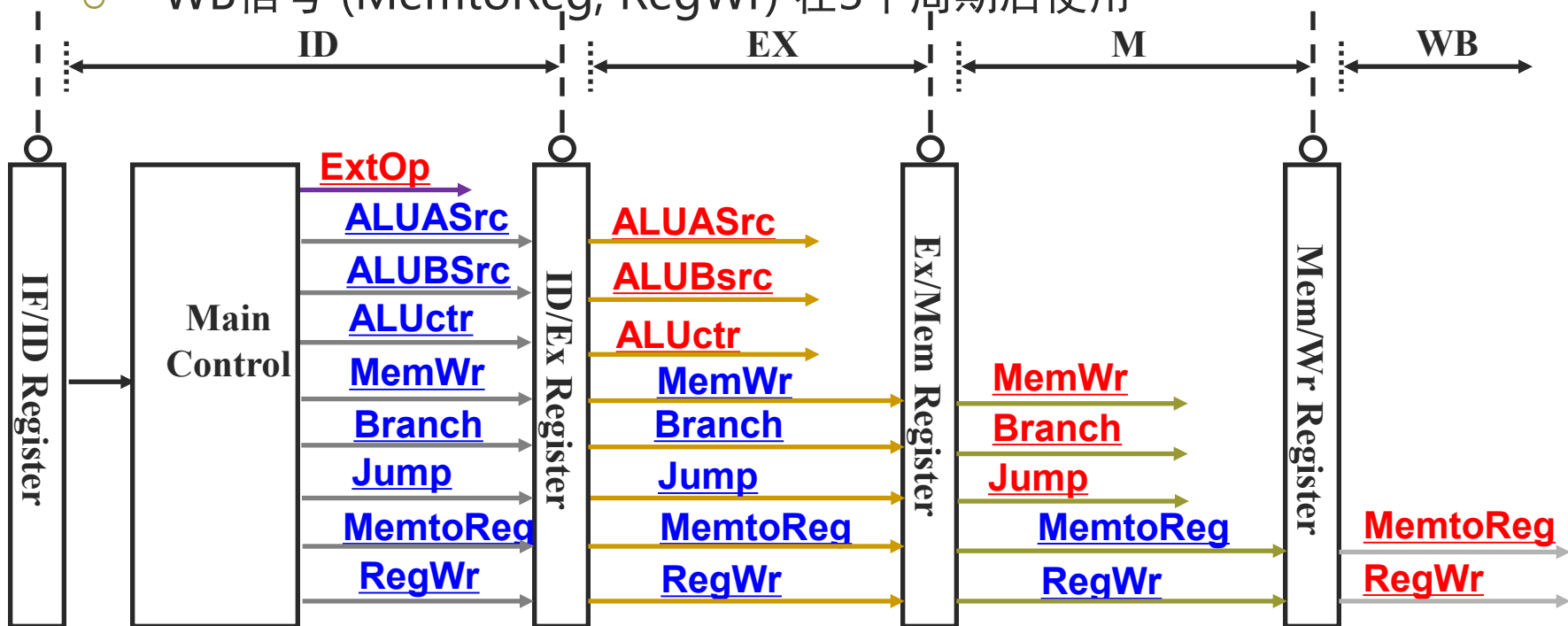


控制信号的时空旅行

- 在取数/译码 (ID) 阶段产生本指令所有控制信号

控制信号需要保存在流水段寄存器中！

- ID信号 (ExtOp) 在当前周期中使用
- EX信号 (ALUASrc, ALUBSrc, ...) 在1个周期后使用
- M信号 (MemWr, Branch, ...) 在2个周期后使用
- WB信号 (MemtoReg, RegWr) 在3个周期后使用



各流水段部件在一个时钟内完成某条指令的某个阶段的工作！

- 在下个时钟到达时，把执行结果以及前面传递来的后面各阶段要用到的所有数据（如：指令、立即数、目的寄存器等）和控制信号保存到流水线寄存器中！



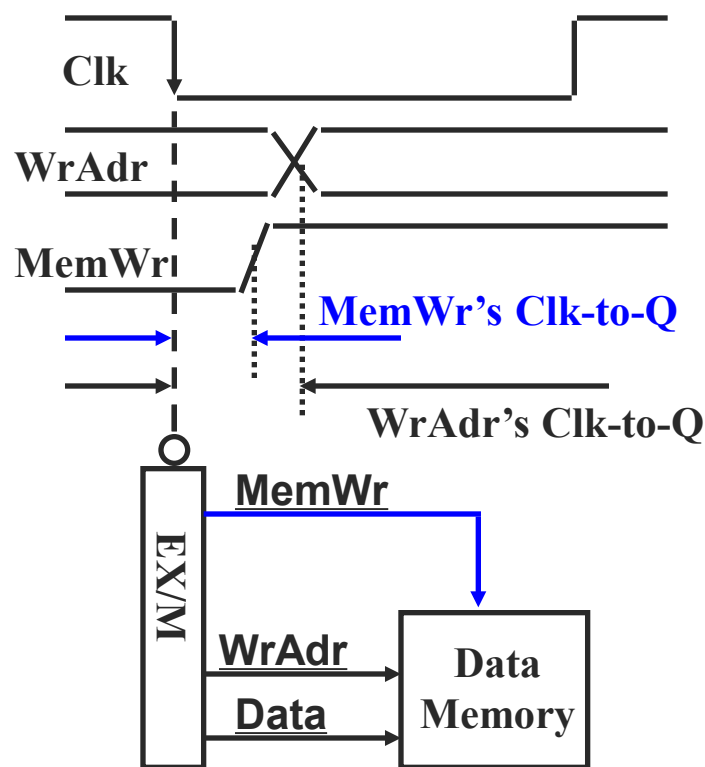
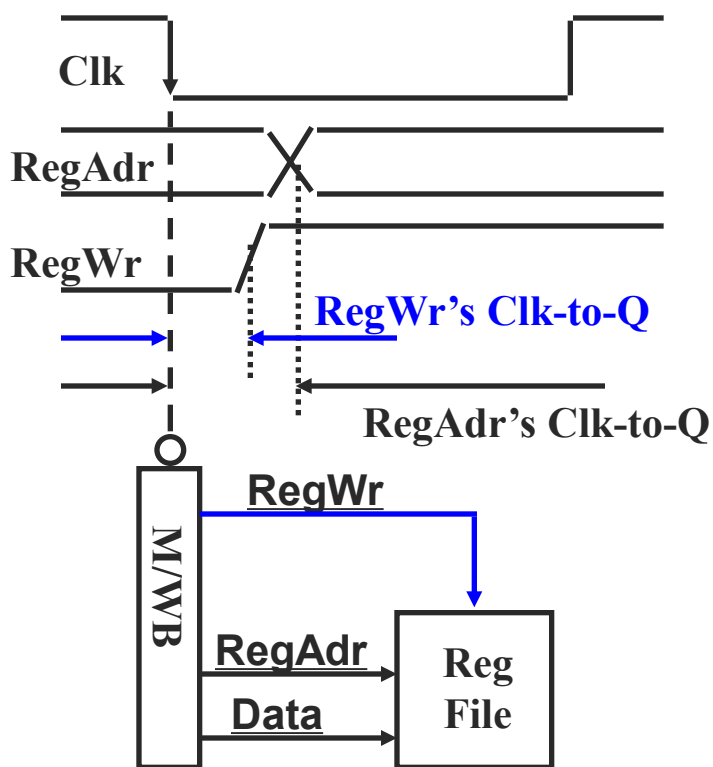
基本流水线CPU的原理图





寄存器/存储器写操作的竞争问题！

- 在流水线中存在地址和写使能之间的“竞争”问题
 - WB段开始时若RegAdr's(Rd)Clk-to-Q > RegWr's Clk-to-Q, 则错写寄存器！
 - M阶段开始时若WrAdr's Clk-to-Q > MemWr's Clk-to-Q, 则错写存储器！





多周期中的“竞争”问题

■ 多周期中解决 Addr 和 WrEn 之间竞争问题的方法:

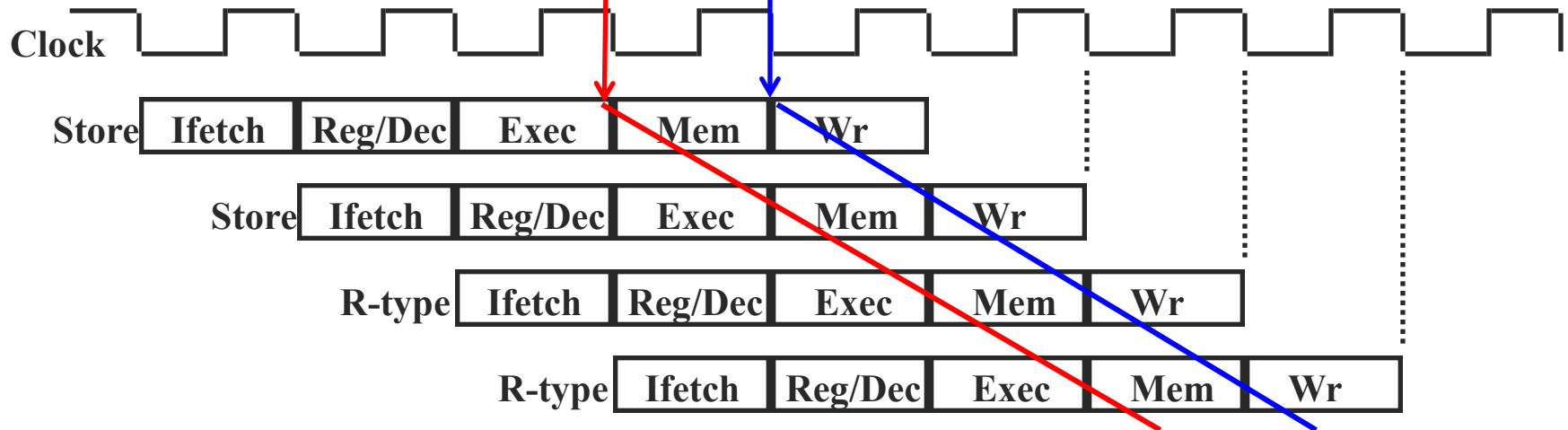
- 在第 N 周期结束时, 让Addr信号有效
- 在第 N + 1 周期让WrEn有效

} 保证Addr信号
在WriteEnable
信号之前到达

■ 上述方法在流水线设计中不能用, 因为:

- 每个周期必须能够写Register
- 每个周期必须能够写Memory

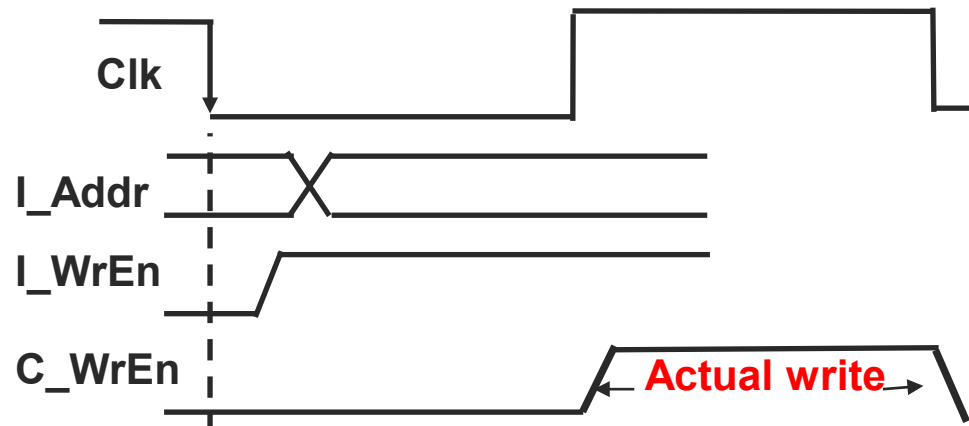
如何解决?





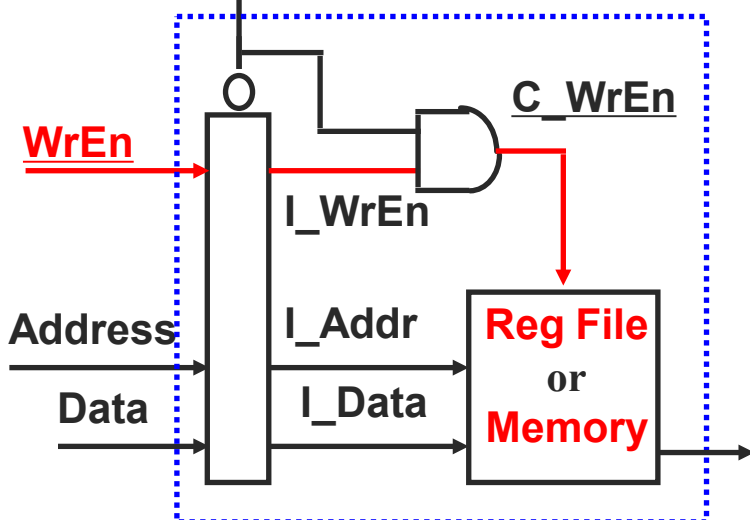
写竞争的解决方案

- 解决方案: 将Write Enable和时钟信号相“与”。

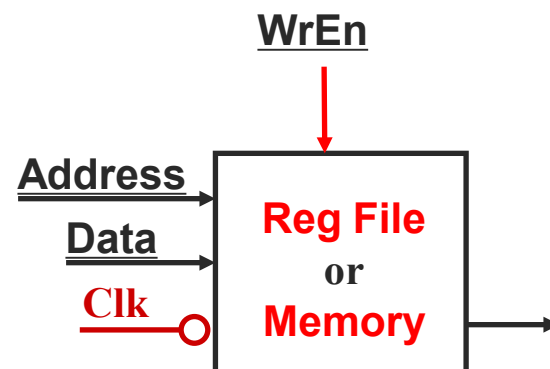


必须合理设计clk时钟周期，确保不会发生“定时错误”！

1. Addr, Data和 WrEn 必须在Clk边沿到来后至少稳定一个 set-up时间
2. Clk高电平时间 大于 写入时间



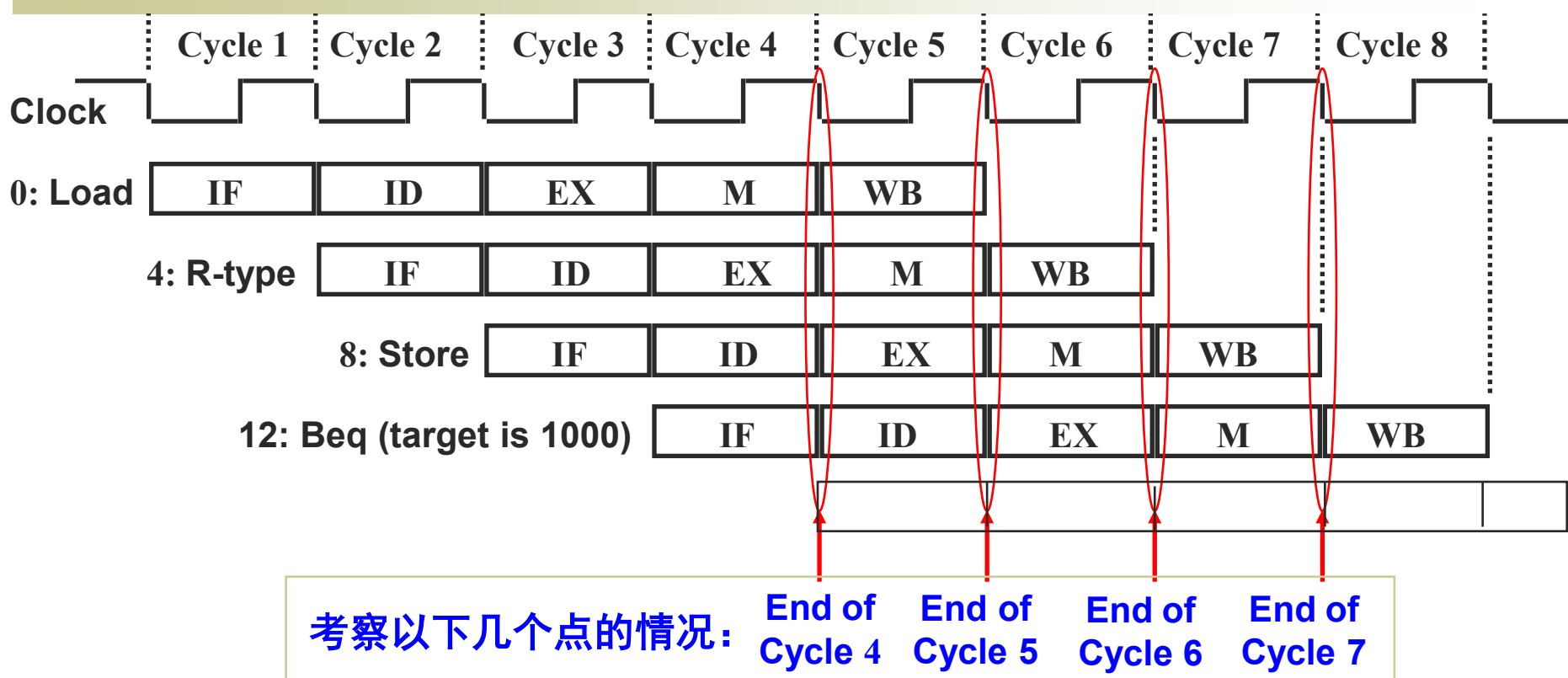
等价于



相当于单周期通路中的理想寄存器和理想存储器



考察流水线DataPath的数据流动情况



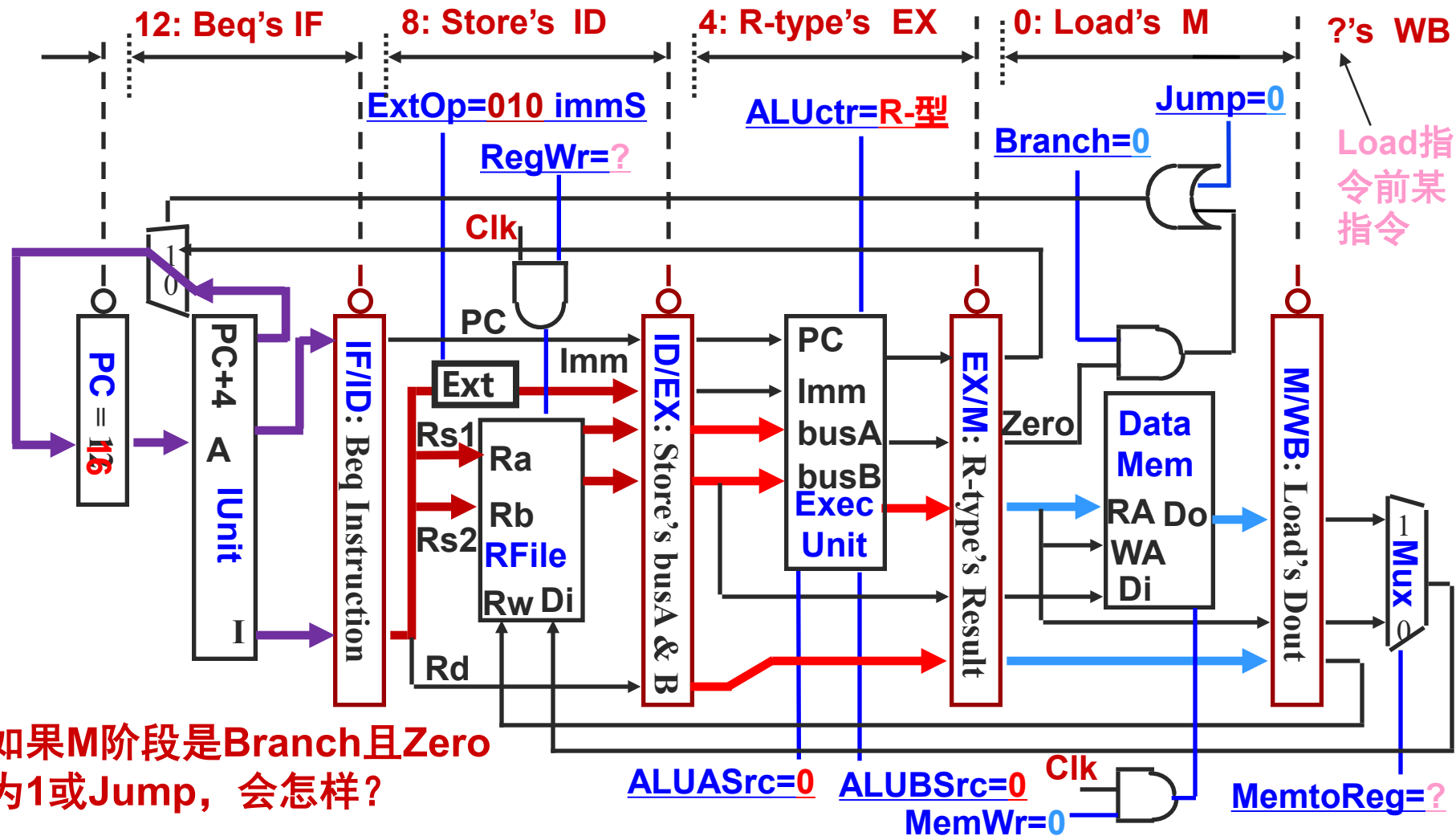
- End of Cycle 4: Load's Mem, R-type's Exec, Store's ID, Beq's Ifetch
- End of Cycle 5: Load's Wr, R-type's Mem, Store's Exec, Beq's ID
- End of Cycle 6: R-type's Wr, Store's Mem, Beq's Exec
- End of Cycle 7: Store's Wr, Beq's Mem

说明：后面仅考察数据流动情况，控制信号随数据同步流动不再说明。



第四周期结束时的状态:

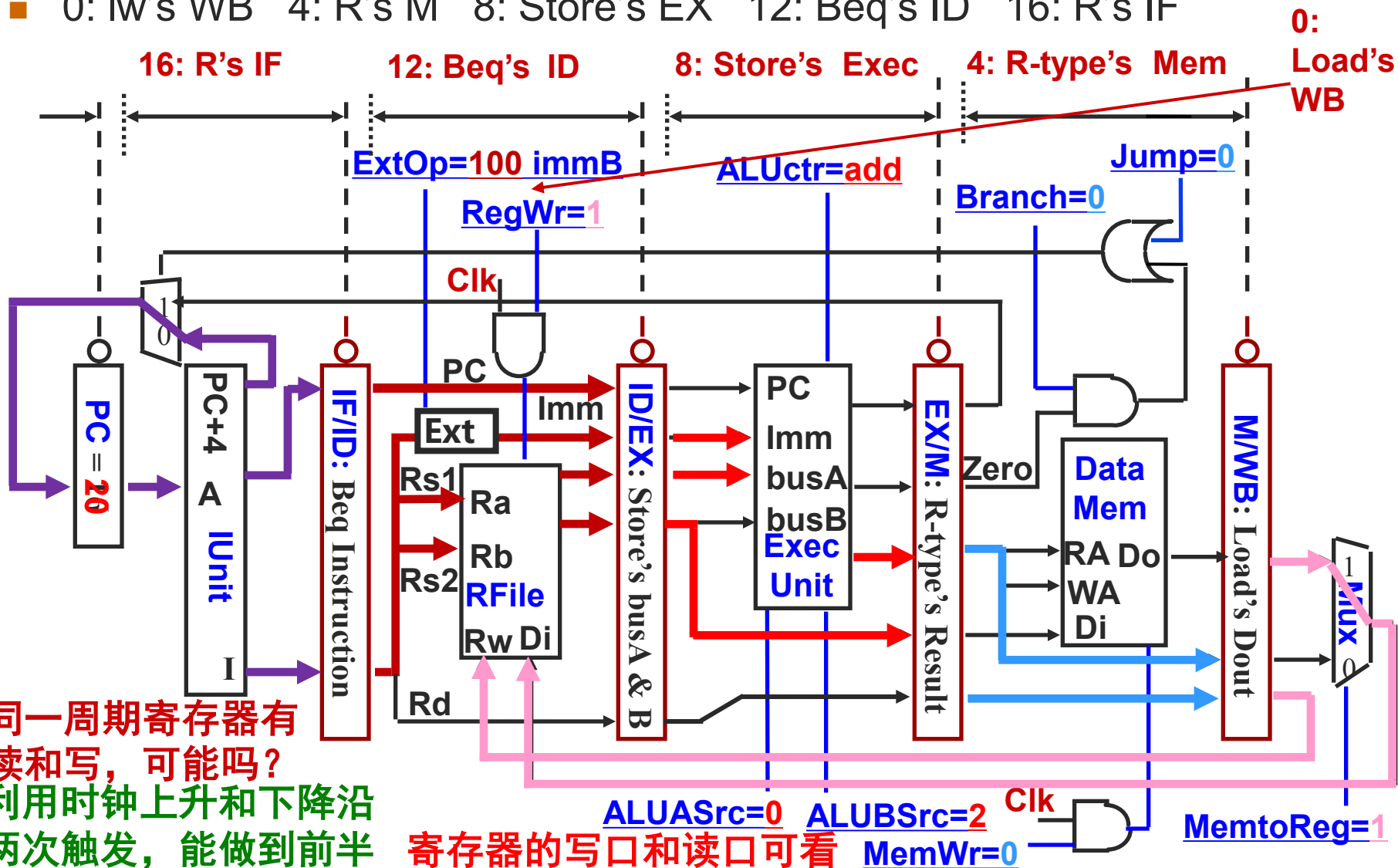
- 0: Load's M 4: R-type's EX 8: Store's ID 12: Beq's IF





第五周期结束时的状态:

- 0: lw's WB 4: R's M 8: Store's EX 12: Beq's ID 16: R's IF



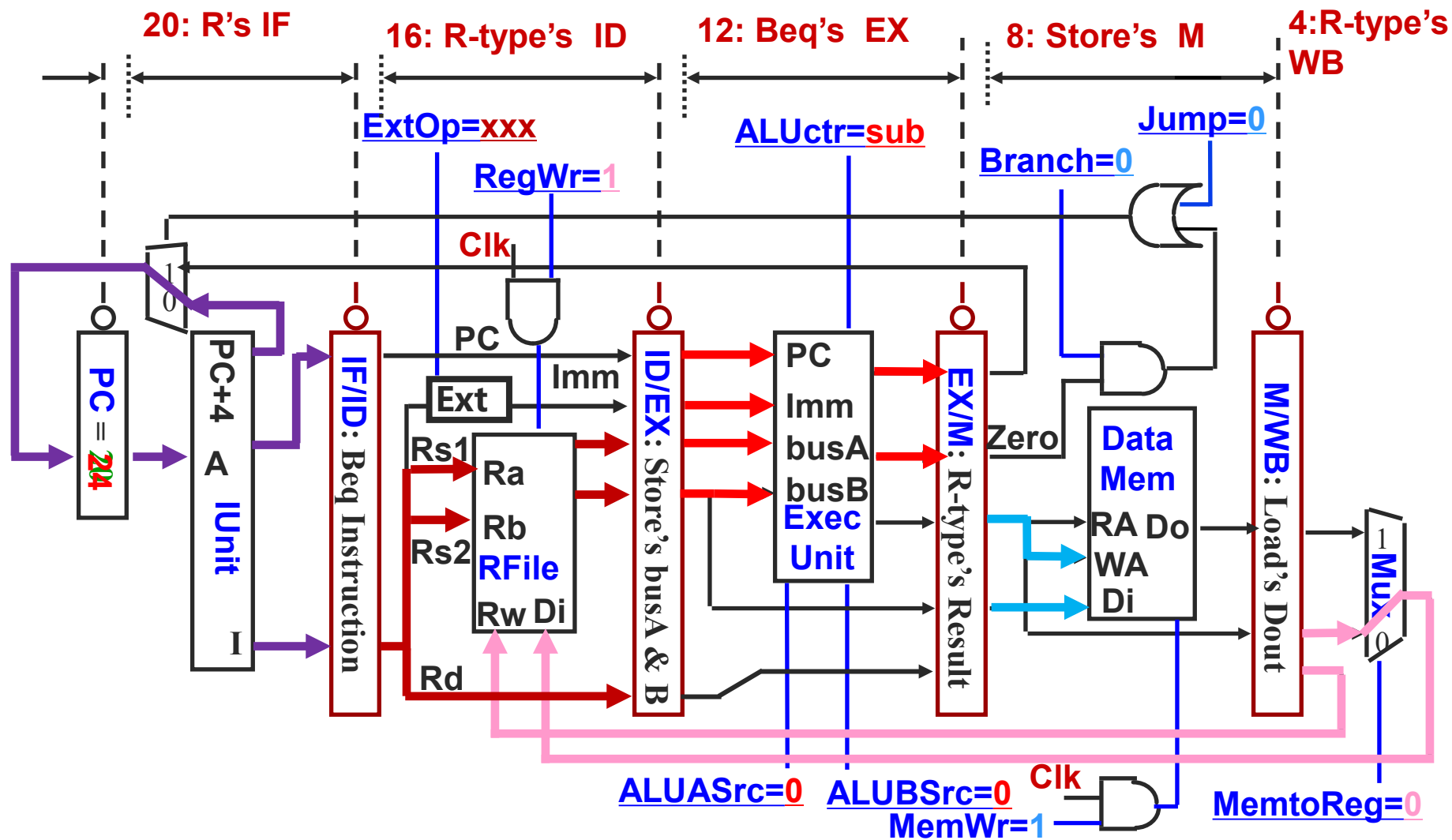
同一周期寄存器有读和写，可能吗？
利用时钟上升和下降沿两次触发，能做到前半周期写，后半周期读

寄存器的写口和读口可看成是独立的两个部件！



第六周期结束时的状态:

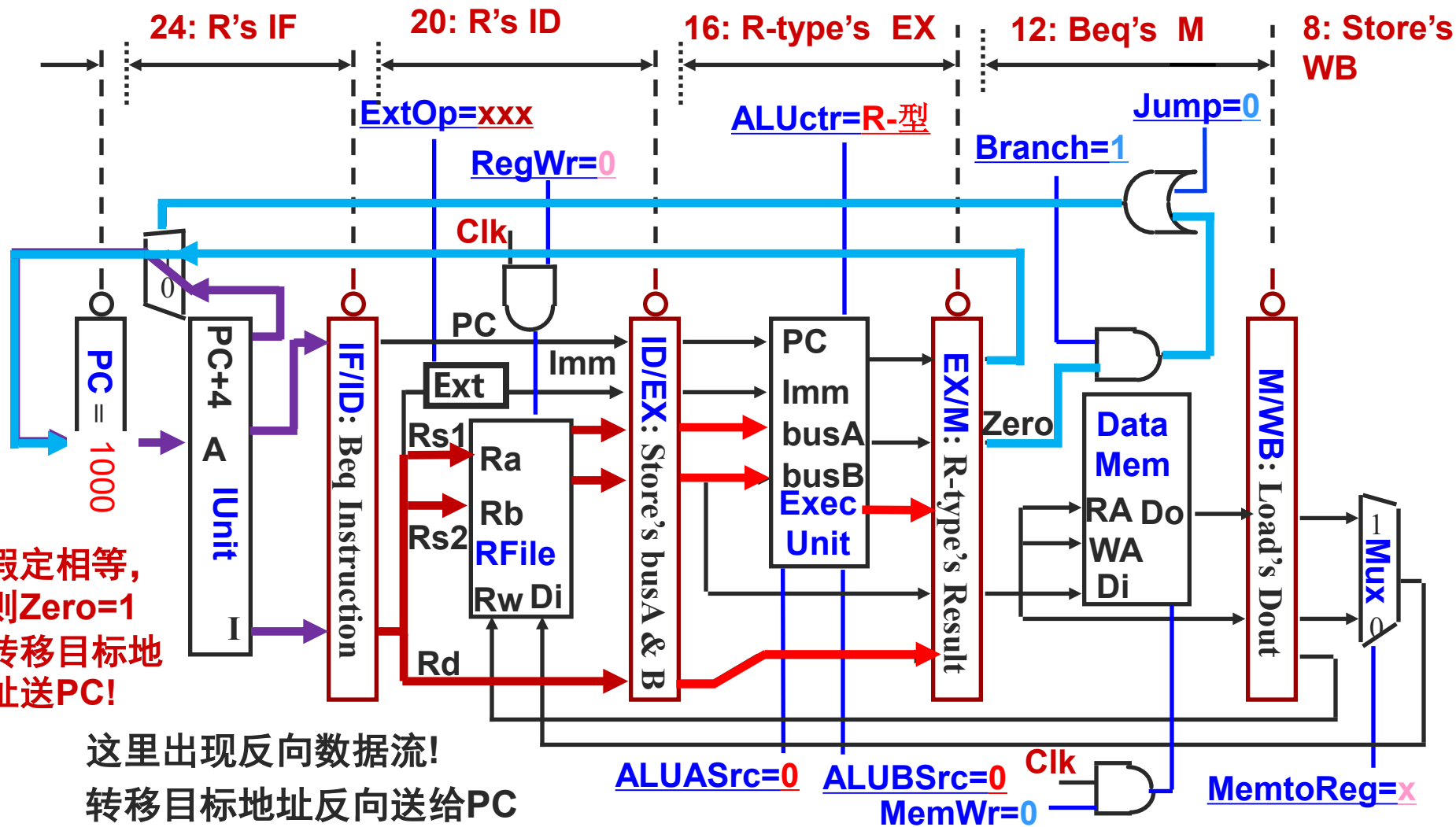
- 4: R's WB 8: Store's M 12: Beq's EX 16: R's ID 20: R's IF





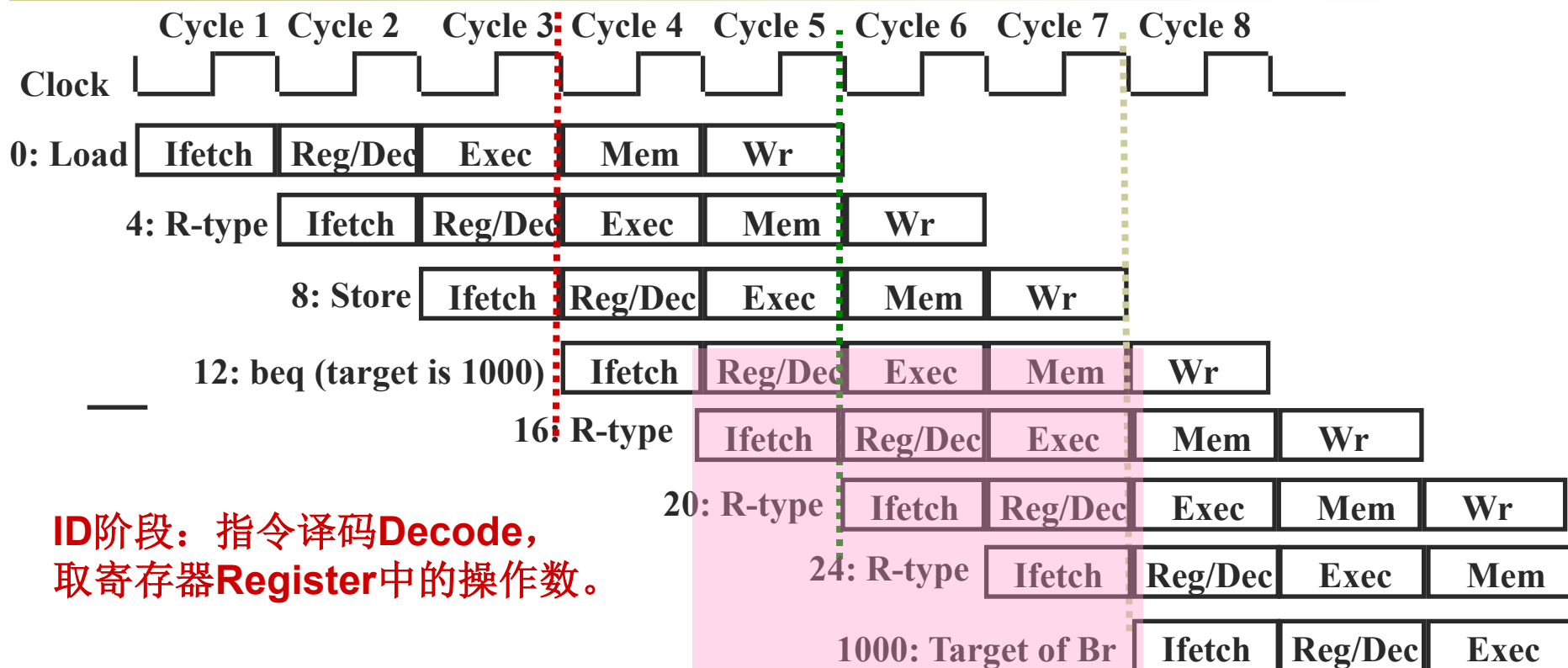
第七周期结束时的状态:

- 8: Store's WB 12: Beq's M 16: R's EX 20: R's ID 24: R's IF





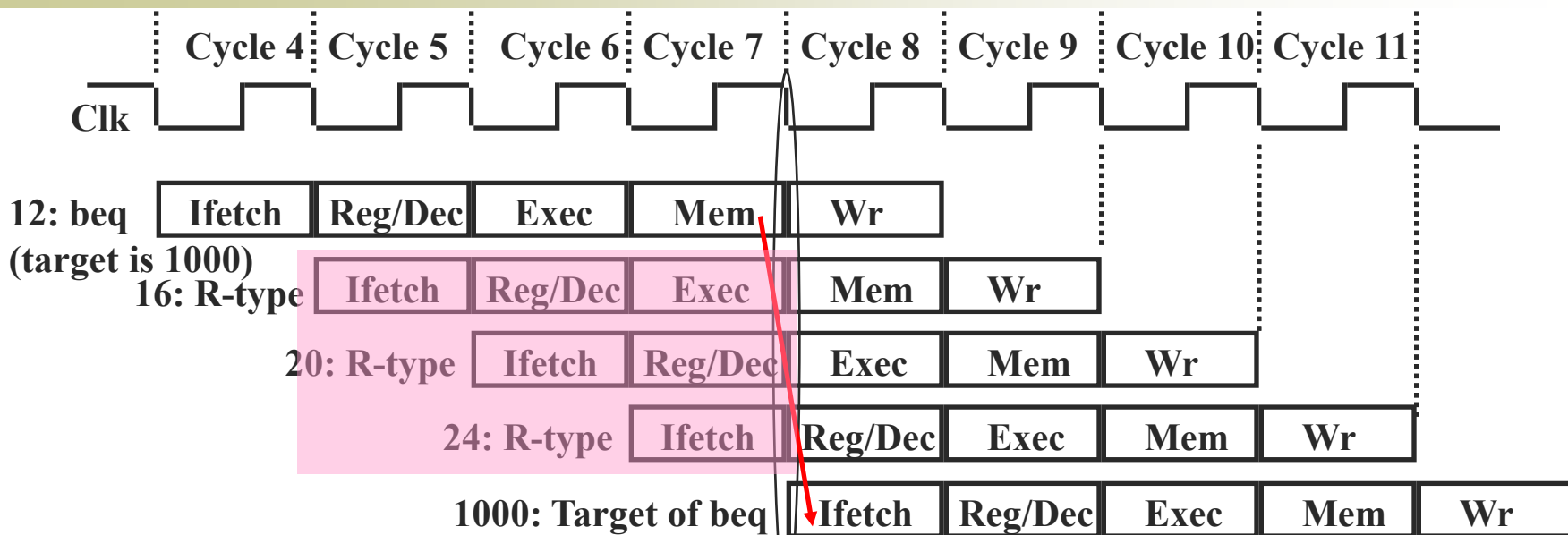
总结前面的流水线执行过程



- 根据流水线执行过程，回答以下问题：
 - beq指令何时确定是否转移？转移目标地址在第几周期计算出来？
 - 如果beq指令执行结果是**需要转移**（称为**taken**），则**流水线会怎样**？
 - Load指令何时能把数据写到寄存器？第几周期开始写数据？
 - 如果后面R-Type的操作数是load指令**目标寄存器**的内容，则**流水线怎样**？



转移分支指令(Branch)引起的“延迟”现象



■ 虽然`beq`指令在第**四**周期取出，但：

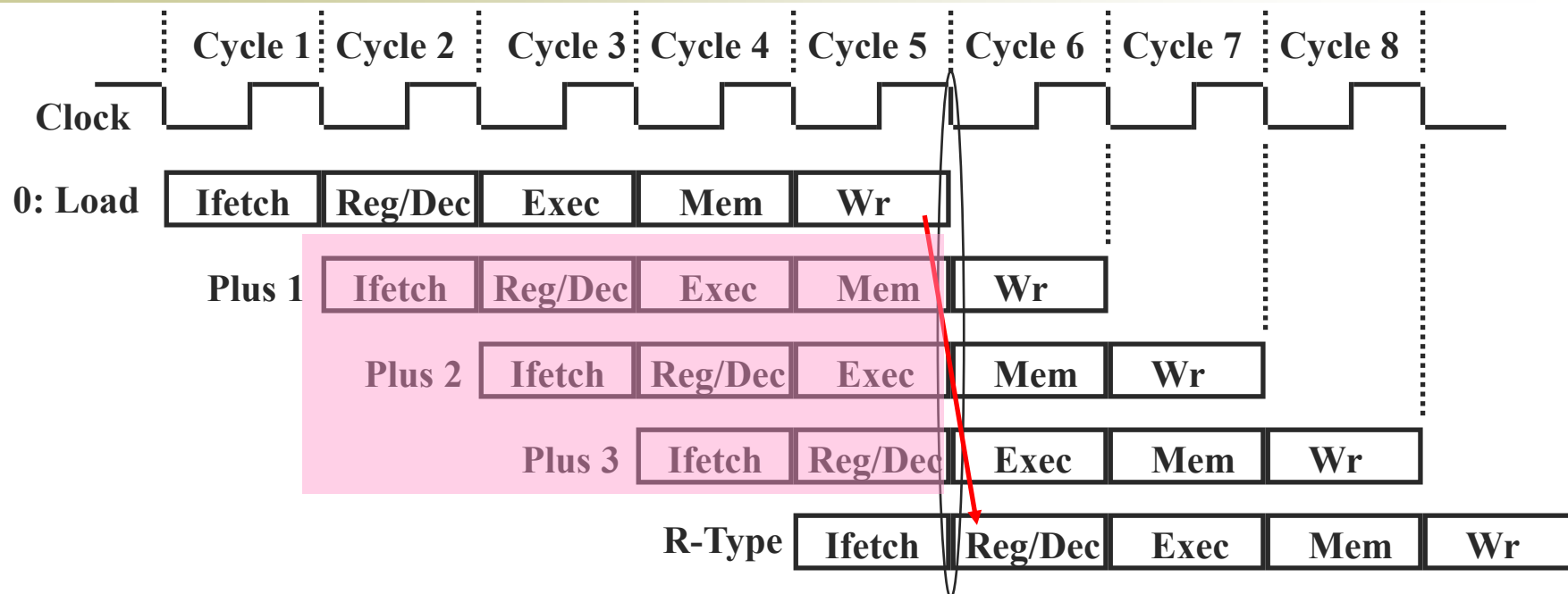
- 第**六**周期得到Zero和转移地址
- 目标地址在第**七**周期才被送到PC的输入端
- 第**八**周期才能取出目标地址处的指令执行

结果：在取目标指令之前，已有三条指令被取出，取错了三条指令！

■ 这种现象称为**控制冒险** (Control Hazard) ，也称为**分支冒险**或**转移冒险** (Branch Hazard) 。



装入指令(Load)引起的“延迟”现象



- 尽管Load指令在**第一**周期就被取出，但：

- 数据在**第5**周期结束才被写入寄存器
- 到**第6**周期写入的数据才能被用

结果：如果随后指令要用到Load的数据的话，就需延迟三条指令！

- 这种现象被称为 **数据冒险 (Data Hazard)** 或**数据相关(Data Dependency)**。



小结

- 指令的执行可以像洗衣服一样，用流水线方式进行
 - 均衡时指令吞吐率提高N倍，但不能缩短一条指令的执行时间
 - 流水段数以最复杂指令所需步骤数为准（有些指令的某些阶段为空操作），每个阶段的宽度以最复杂阶段所需时间为准（尽量调整使各阶段均衡）
- 以Load指令为准，分为五个阶段
 - 取指令段(IF)：取指令、计算PC+4 (IUnit: Instruction Memory、Adder)
 - 译码/读寄存器(ID)段
 - ◆ 指令译码、立即数扩展 (Extender)、读Rs1、Rs2和Rd (寄存器读口)
 - 执行(EX)段：计算转移目标地址、ALU运算 (ALU、Adder)
 - 存储器(M)段：读或写存储单元 (Data Memory)
 - 写回寄存器(WB)段：ALU结果或从DM读出数据写到寄存器 (寄存器写口)
- 流水线控制器的实现
 - ID段生成所有控制信号，并随指令执行过程信息同步向后续阶段流动
 - 与单周期处理器的控制器的实现方法一样，无需采用有限状态机
- 流水线冒险：结构冒险、控制冒险、数据冒险
(下一讲主要介绍解决流水线冒险的数据通路如何设计)



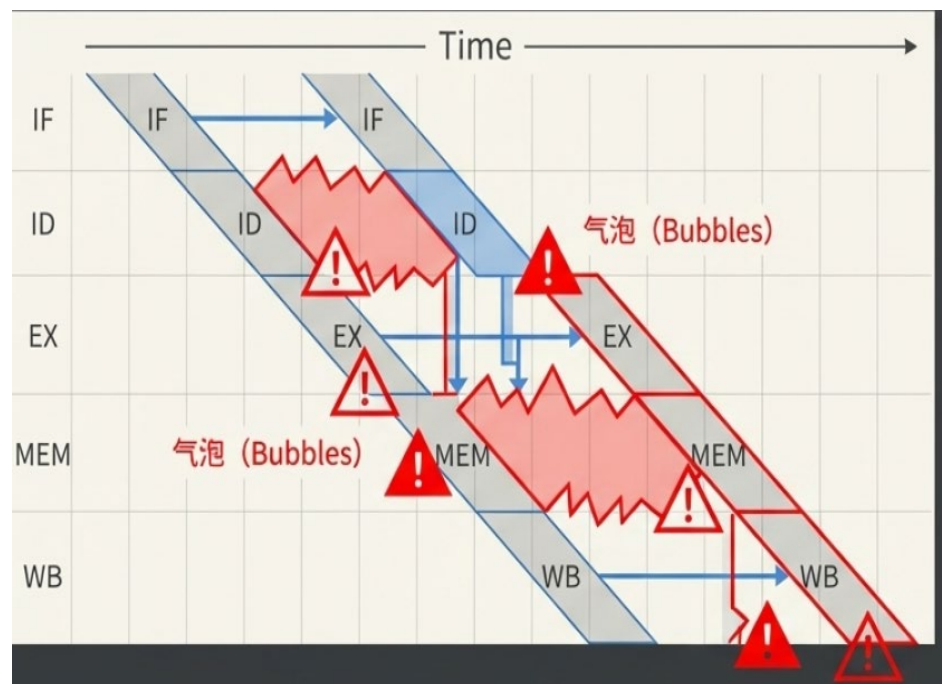
课堂练习

例：对于具有9条指令的单周期CPU，最复杂的lw指令的5个阶段所用时间如下。①取指：200ps；②寄存器读：50ps；③ALU操作：100ps；④存储器读：200ps；⑤寄存器写：50ps。假定采用5段流水线数据通路实现相同的9条指令，请在不考虑控制单元、PC访问、信号传递等延迟的情况下，比较单周期CPU和流水线CPU两种方式下的指令吞吐率的相差倍数。



现实的引力：流水线的阿喀琉斯之踵

- 理想很丰满：CPI完美等于1。
- 现实很骨感：如果下一条指令必须等待上一条指令算完的结果呢？如果遇到分支跳转，整条流水线走错了路呢？
- 三大杀手(Hazards)
 - 结构冒险：硬件资源不够分，发生物理冲撞。
 - 数据冒险：写后读(RAW)的时序悖论。
 - 控制冒险：分支跳跃带来的巨额废弃指令惩罚。





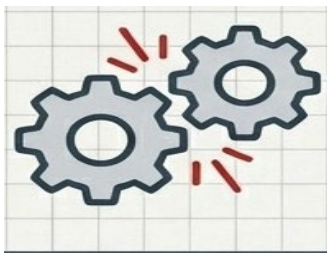
流水线冒险的处理



5 破坏流水线的“三大杀手”

■ 流水线冒险 (hazard)

- 指流水线CPU遇到无法正确执行后续指令或执行了不该执行的指令。



结构冒险
(Structural Hazards)

本质：硬件不够分。

表现：同一周期内，两条处于不同流水段的指令，试图使用同一个物理部件。



数据冒险
(Data Hazards)

本质：结果还没算完。

表现：当前指令亟需的源操作数，前序指令尚未计算完成或尚未写回物理寄存器。



控制冒险
(Control Hazards)

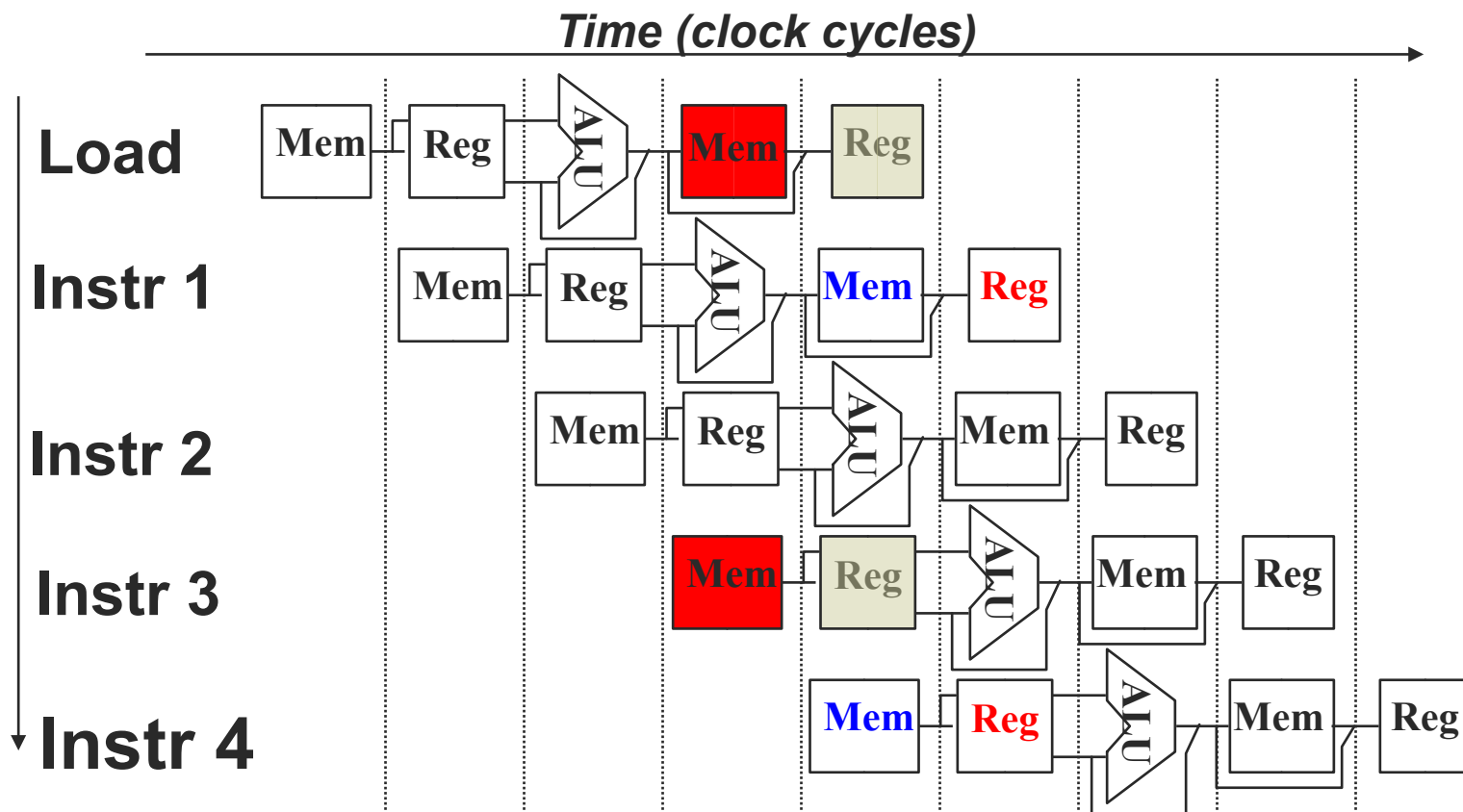
本质：不知道去哪。

表现：分支指令尚未评估出最终的跳转目标地址，后续指令已经盲目涌入流水线。



5.1 结构冒险

- 结构冒险也称**硬件资源冲突**：同一个执行部件被多条指令使用

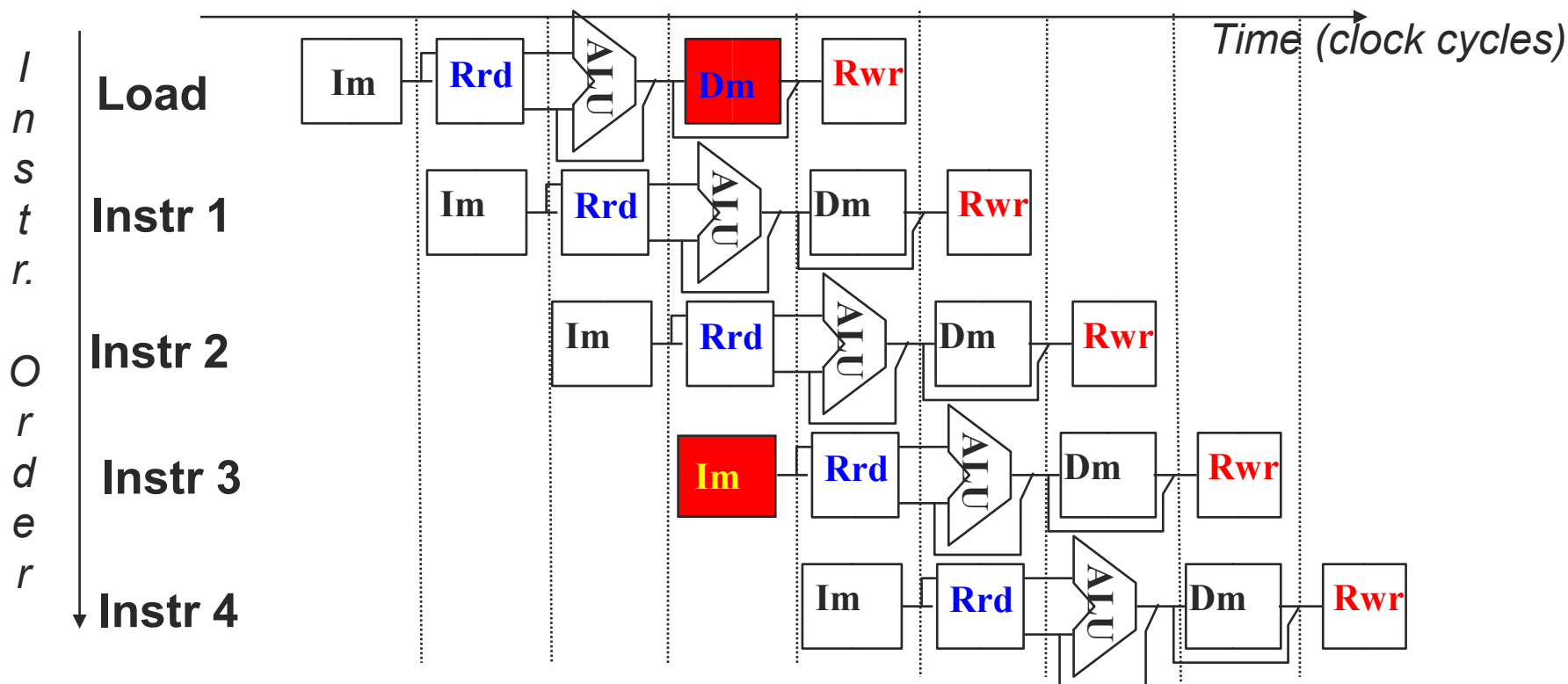


- 存储器：在Load指令取数据时，指令3同时取指令，则发生**冲突**！
- 寄存器堆：如果不对寄存器堆的**写口**和**读口**独立设置的话，则发生冲突！



5.1 结构冒险

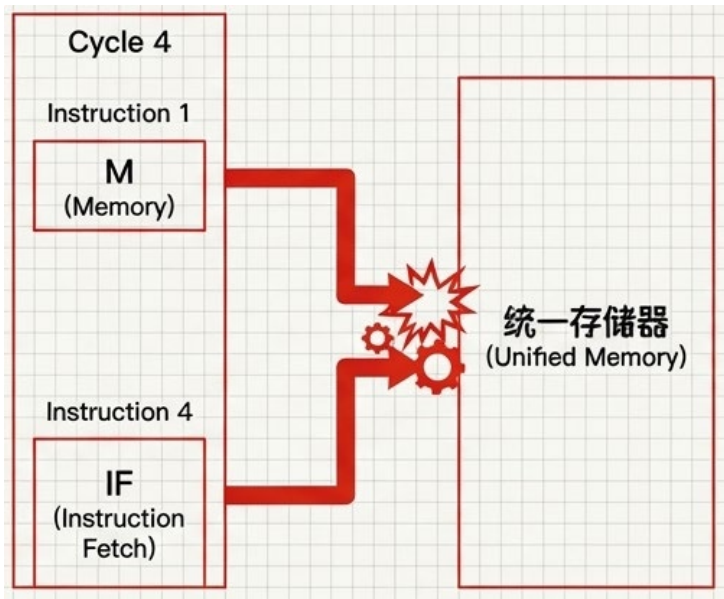
- 解决方法：每个功能部件只在特定的流水段中使用！
 - 将Instruction Memory (Im) 和 Data Memory (Dm) 分开。
 - 在取指令阶段和执行阶段各增加一个加法器 (PC+4, PC/BusA+Imm)。
 - 将寄存器读口Rrd和写口Rwr独立开来，相当于两个部件。



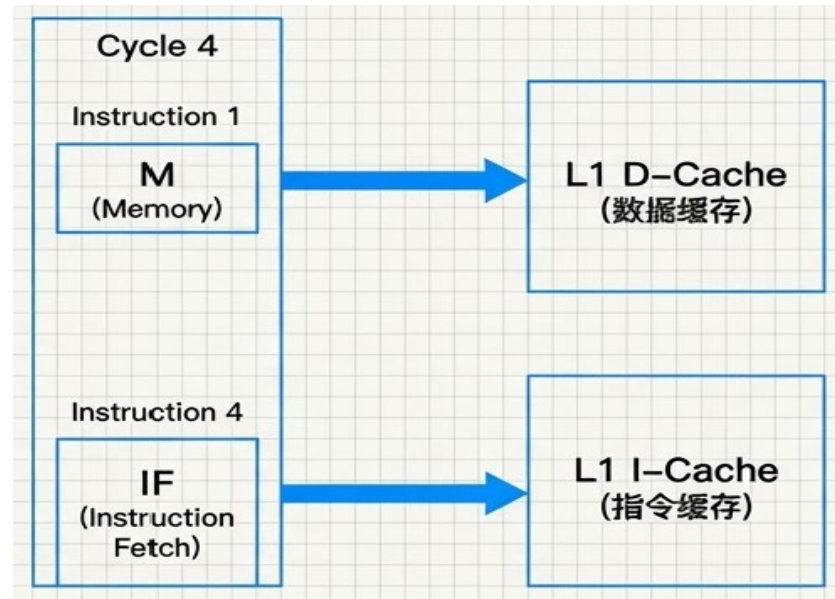
现代计算机中L1 cache通常采用数据cache和代码cache分离的方式，因而也就避免了结构冒险的发生。



结构冒险与哈佛架构的妥协



冯·诺依曼架构的统一存储成为致命瓶颈。



L1缓存层采用哈佛架构：指令与数据物理分离。

寄存器读写冲突如何解？

采用半周期错峰时序设计：前半周期写，后半周期读，兵不血刃化解冲突。



5.2数据冒险与时间倒流

举例说明：以下指令序列中，寄存器**r1**会发生数据冒险

想一下，哪条指令的**r1**是旧的值？

哪条是新的值？

add **r1**, r2, r3

sub r4, **r1**, r3 读**r1**时，add指令正在执行加法(EX段)，旧值！

and r6, **r1**, r7 读**r1**时，add指令正在传递加法结果(M段)，旧值！

or r8, **r1**, r9 读**r1**时，add指令正在写加法结果到**r1**(WB段)，旧值！

xor r10, **r1**, r11 读**r1**时，add指令已经把加法结果写到**r1**，新值

补充：三类数据冒险现象

画出流水线图能很清楚理解！

RAW: 写后读（基本流水线中经常发生，如上例）

WAR: 读后写（基本流水线中不会发生，乱序执行时会发生）

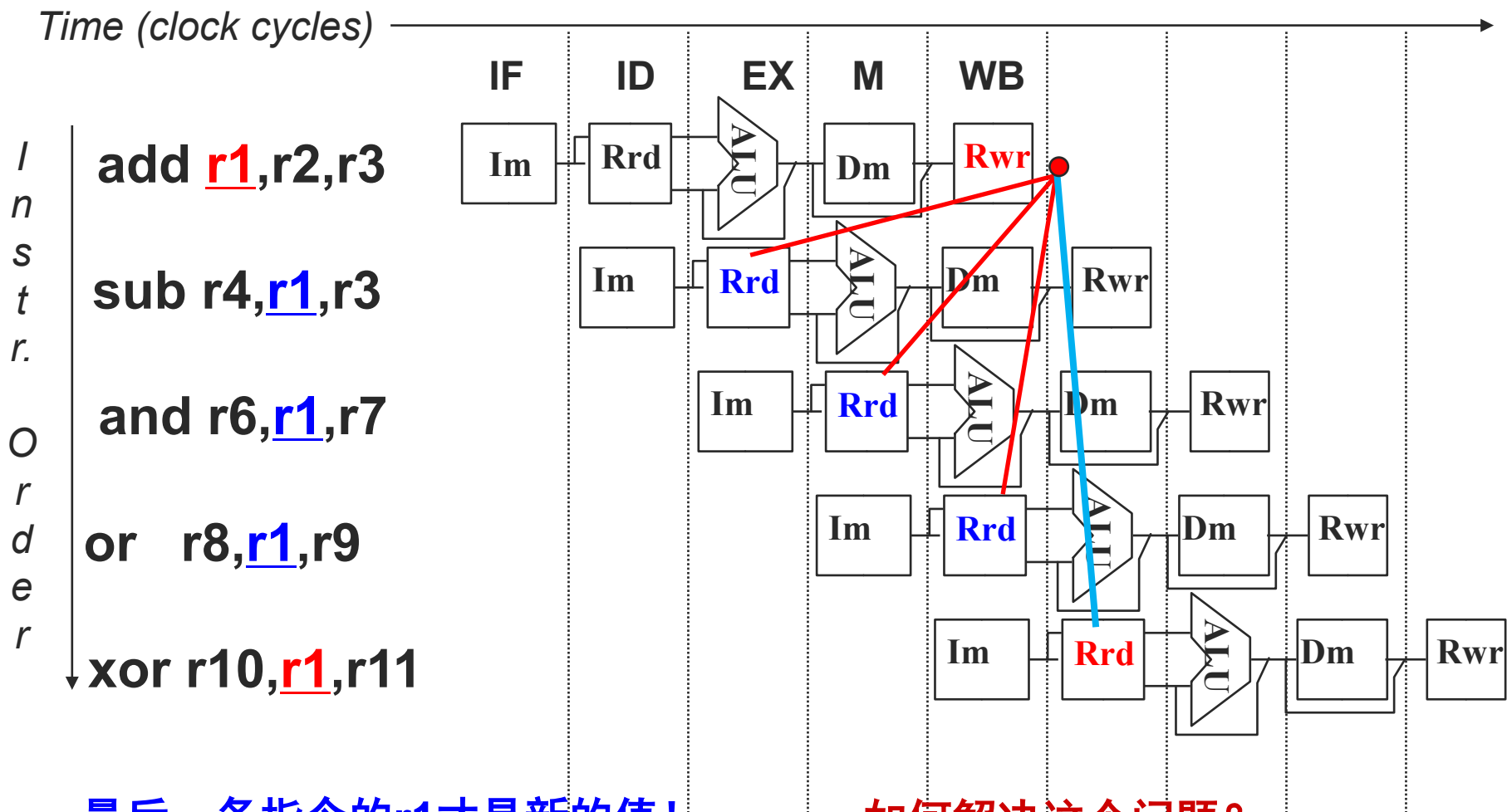
WAW: 写后写（基本流水线中不会发生，乱序执行时会发生）

本讲介绍基本流水线，所以仅考虑RAW冒险



5.2 数据冒险

- 举例：以下指令序列中，寄存器r1会发生数据冒险





5.2 数据冒险

■ 数据冒险的解决方法

- 方法1：软件插入 “NOP” 指令
- 方法2：硬件阻塞 (stall)
- 方法3：合理实现寄存器堆的读/写操作（不能解决所有数据冒险）
 - ◆ 前半时钟周期写，后半时钟周期读，若同一个时钟内前面指令写入的数据正好是后面指令所读数据，则不会发生数据冒险。
- 方法4：转发 (Forwarding或Bypassing 旁路) 技术
 - 若相关数据是ALU结果，则如何？
可通过转发解决
 - 若相关数据是上条指令DM读出内容，则如何？
不能通过转发解决，随后指令需被阻塞一个时钟 或 加NOP指令
称为Load-use数据冒险！
- 方法5：编译优化，调整指令顺序（不能解决所有数据冒险）
实现“转发”和“阻塞”要修改数据通路：
 - (1) 检测何时需要“转发”，并控制实现“转发”
 - (2) 检测何时需要“阻塞”，并控制实现“阻塞”

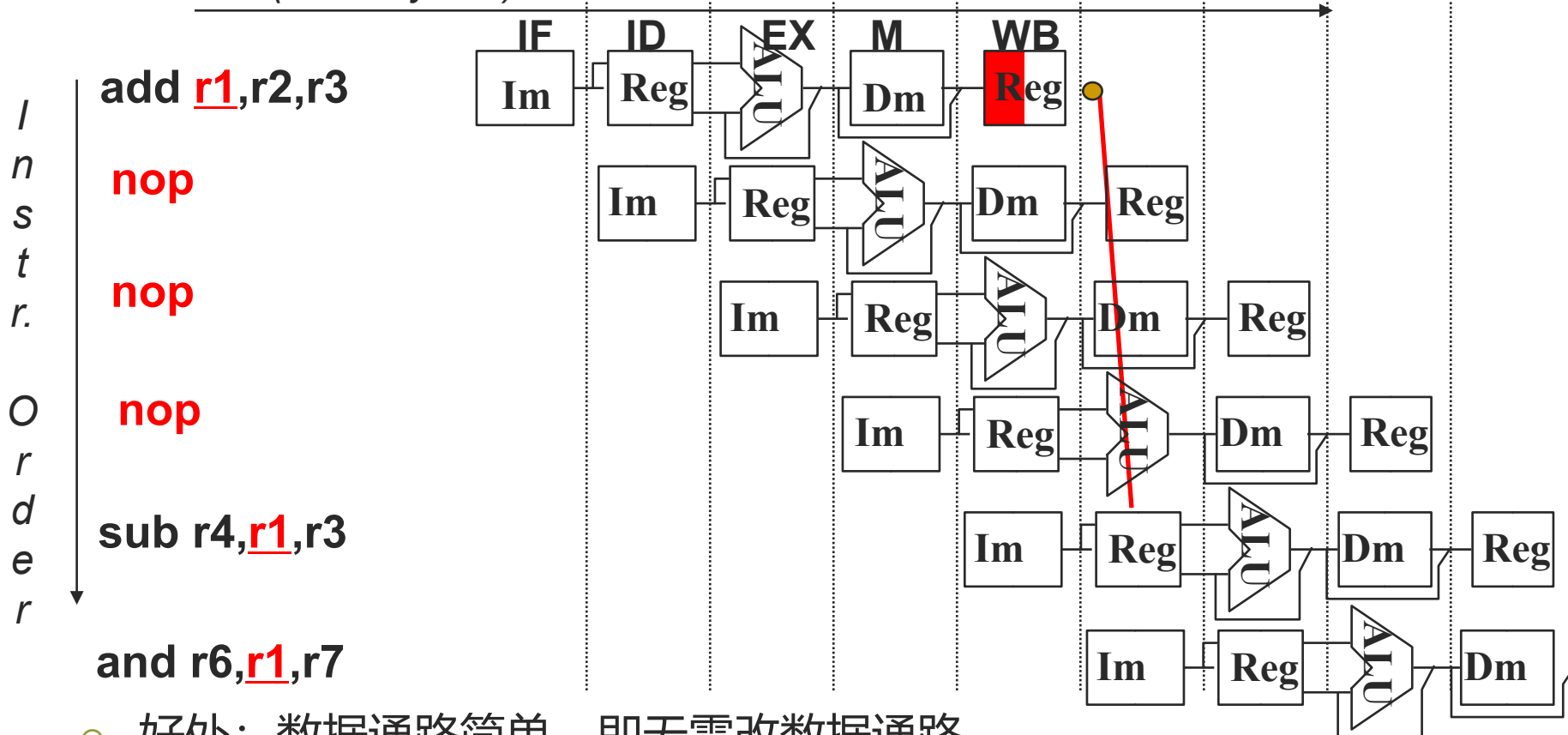


5.2 粗暴的代价：停顿与气泡

■ 方法1：软件插入“NOP”指令

- 软件补丁：由编译器插入三条NOP指令，浪费三条指令的空间和时间。

Time (clock cycles)

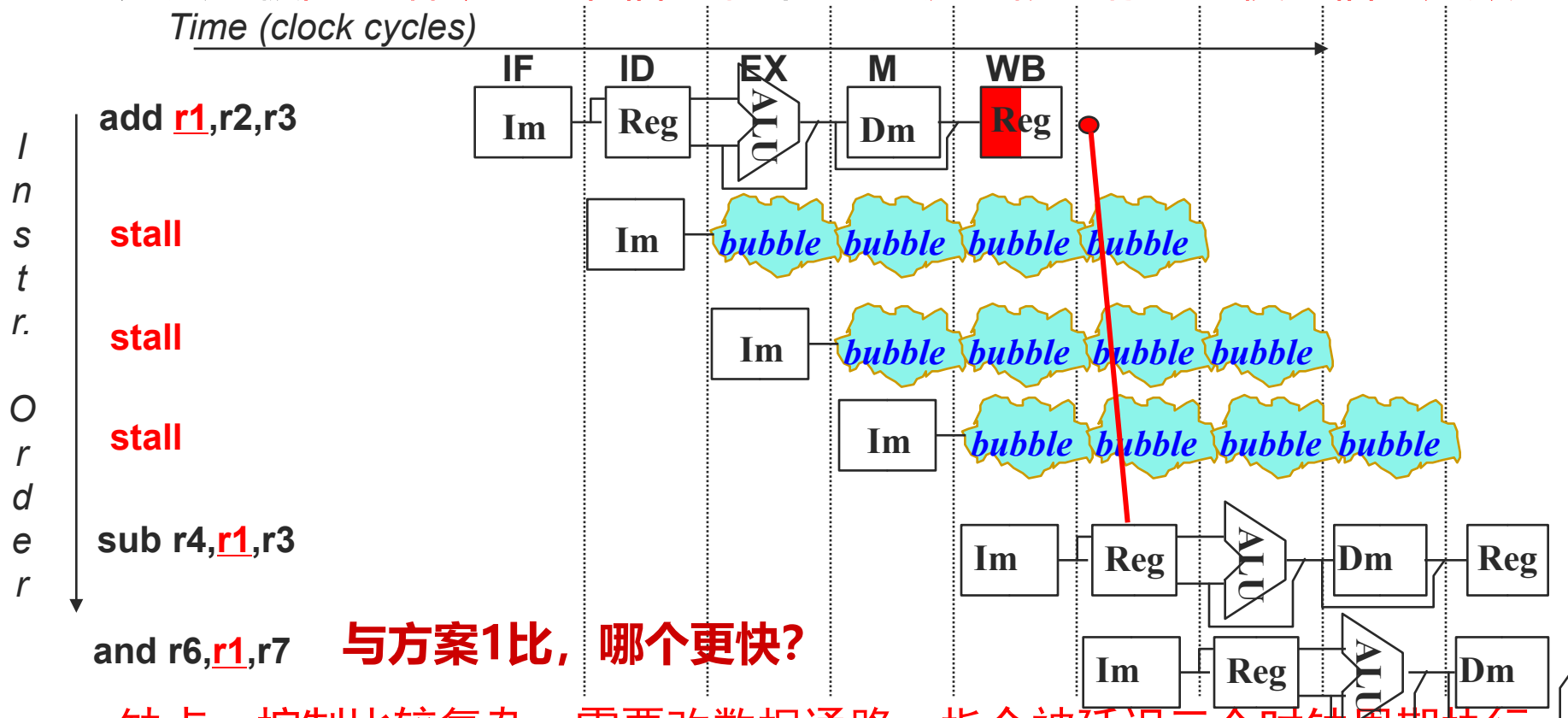


- 好处：数据通路简单，即无需改数据通路。



5.2 粗暴的代价：停顿与气泡

- 方法2：硬件互锁，强行冻结流水线（现代基础方案）！
 - 硬件上通过**阻塞(stall)**方式阻止后续指令执行，**延迟**到有新值以后！这种做法称为**流水线阻塞**，也称为插入“**气泡Bubble**”。
 - 方法：被**阻塞指令**的控制信号清0/**PC**、流水段寄存器写使能信号无效。

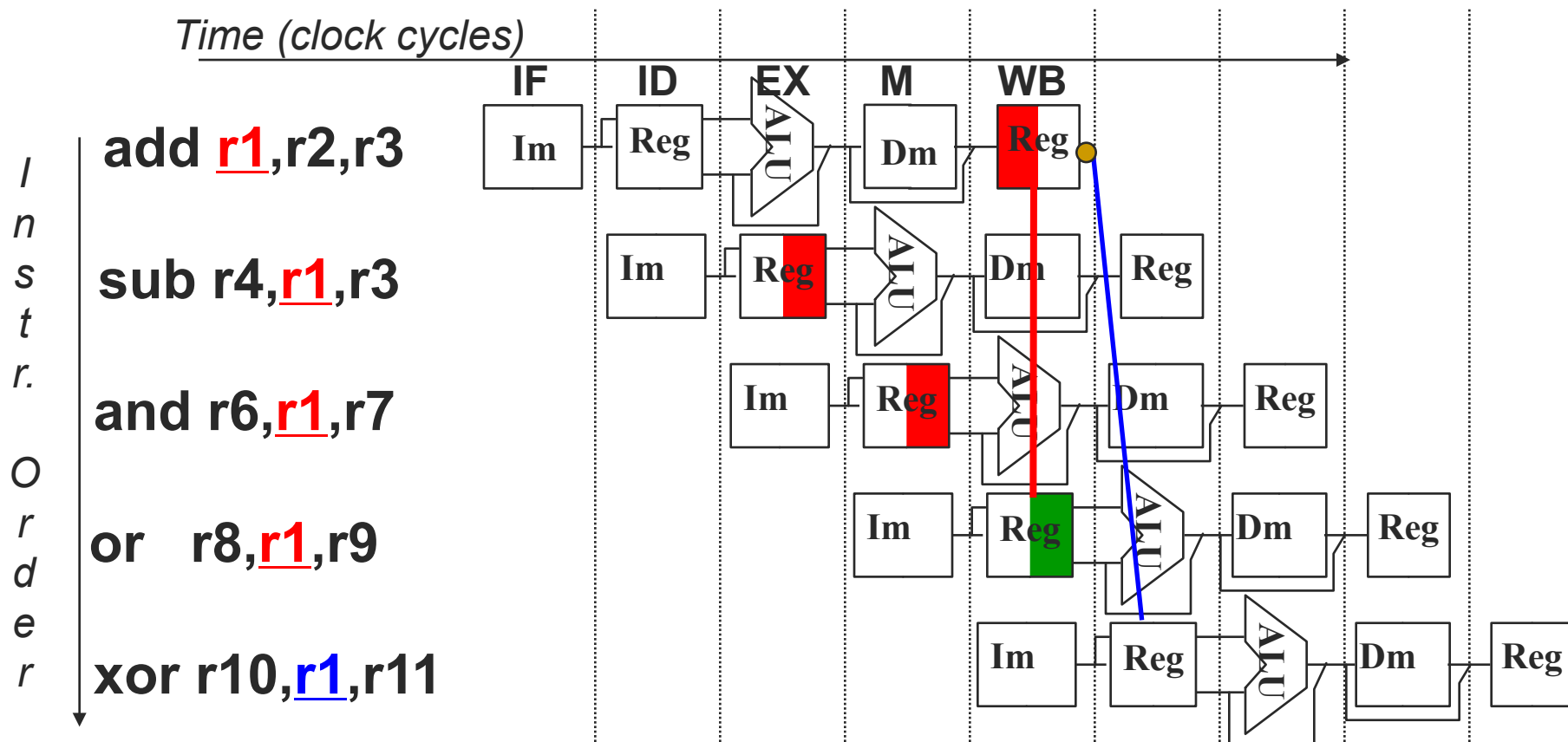


- 缺点：控制比较复杂，需要改数据通路；指令被延迟三个时钟周期执行。



5.2 数据冒险

- 方法3：寄存器堆的读口和写口是相互独立的部件！先写后读
寄存器堆写入边沿信号和其他边沿信号相差半个周期。



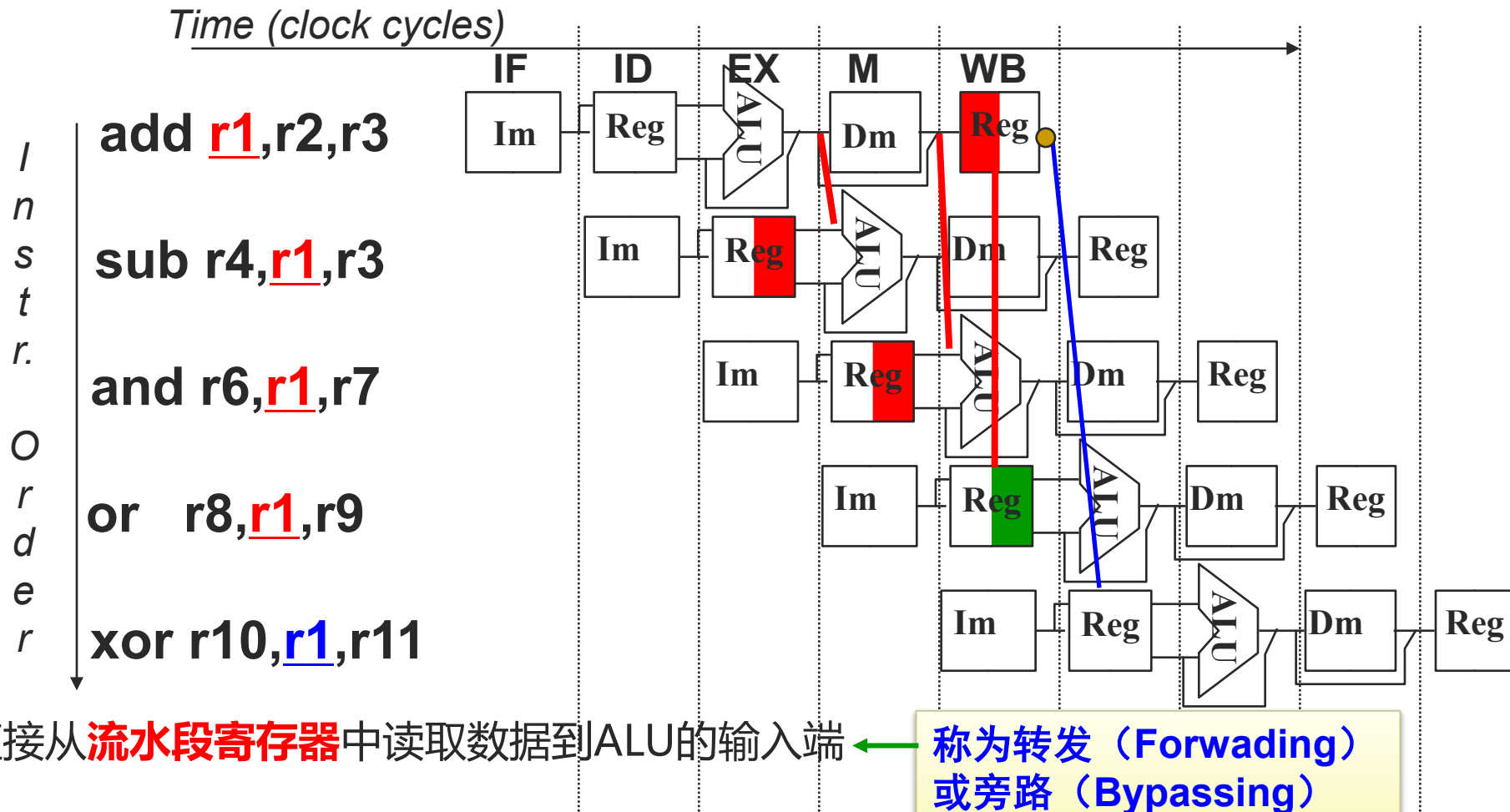
寄存器写口/读口分别在前/后半周期进行操作，使写入数据被直接读出
可以提前一个周期，获取到新值。但是，只能解决部分数据冒险！



5.2 旁路转发的暴力方案

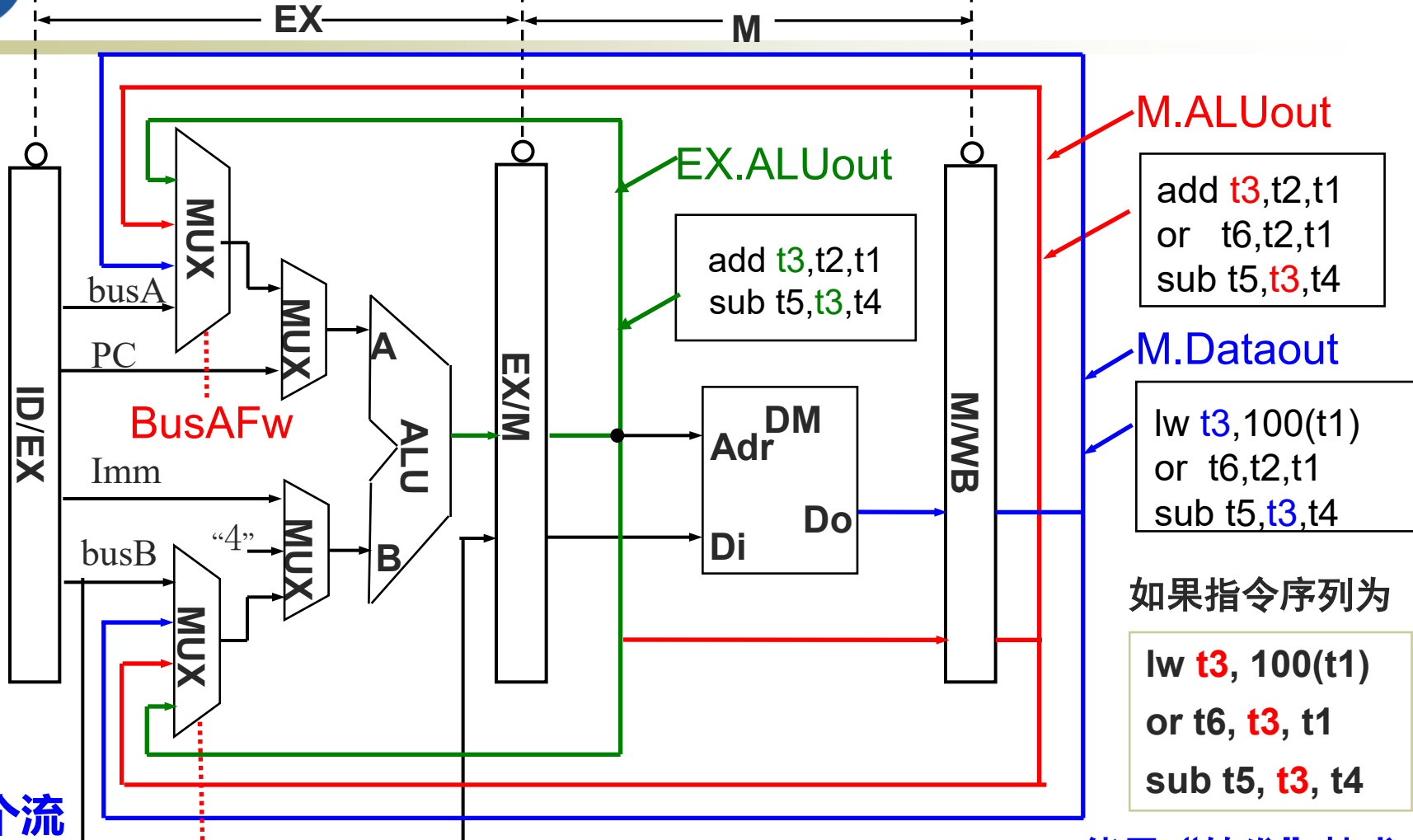
- 方法4: 利用DataPath中的中间数据: **转发+阻塞**

什么时候生成了r1新值? 在EX/M、M/WB流水段寄存器中已有需要的值r1!





支持“转发”技术的硬件设计



在哪个流水段执行转发检测?

如何优化?

新增多路选择器的控制信号是什么?

EX.rs1=M.rd & M.RegWr=1;

EX.rs1=WB.rd & WB.RegWr=1;

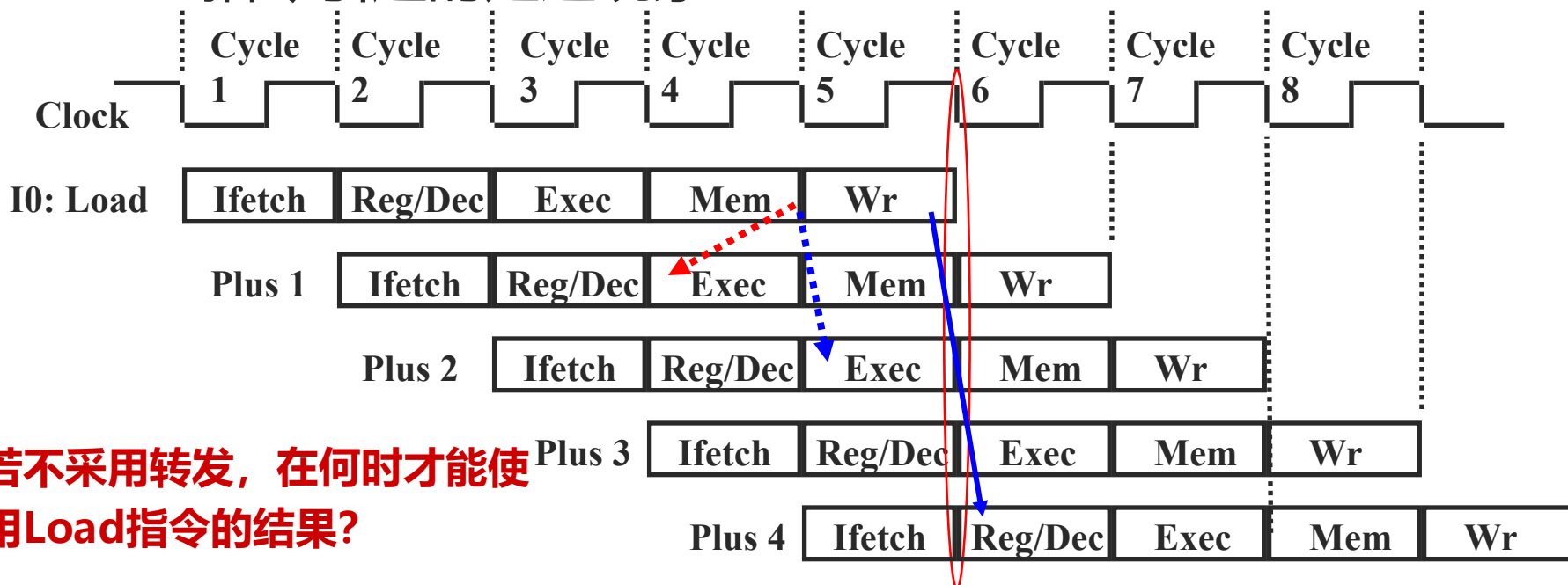
EX.rs1=WB.rd & WB.OP=load & WB.RegWr=1;

能用“转发”技术解决Load-Use指令间的数据冒险吗?



5.2 无法被转发挽救的Load-Use冒险

■ Load指令引起的延迟现象



若不采用转发，在何时才能使用Load指令的结果？

- Load指令最早在哪个流水线寄存器中开始有后续指令需要的值？

实际上，在第四周期结束时，数据在流水段寄存器中已经有值。

采用数据转发技术可以使load指令后面第二条指令得到所需的值

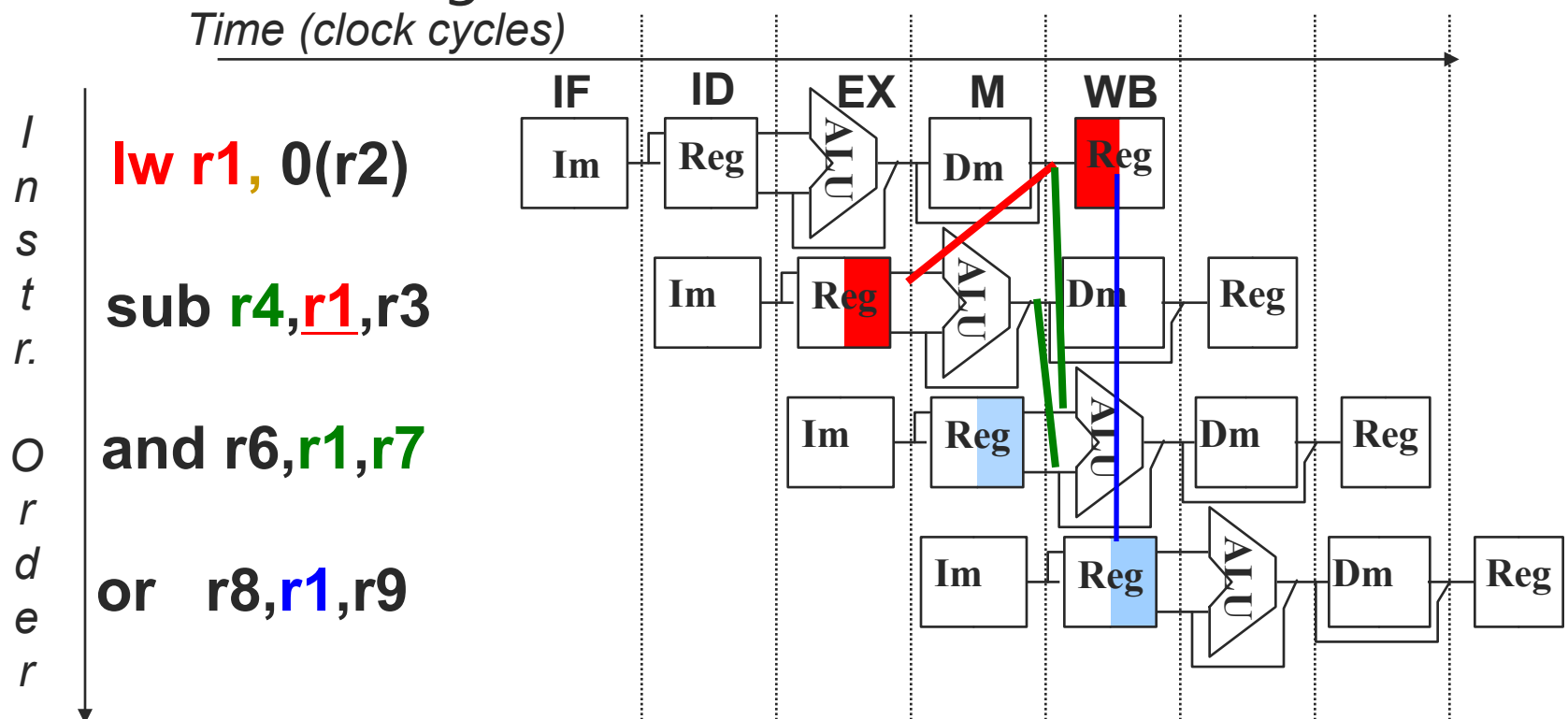
但不能解决load指令和随后第一条指令间的数据冒险，要延迟执行一条指令！

这种load指令和随后指令间的数据冒险，称为“装入-使用数据冒险 (load- use Data Hazard)”



5.2 软硬协同设计：化解Load-Use惩罚

- 转发Forwarding技术使Load-use冒险需阻塞一个时钟周期



采用“转发”后仅第二条指令 SUB r4,r1,r3 不能按时执行!

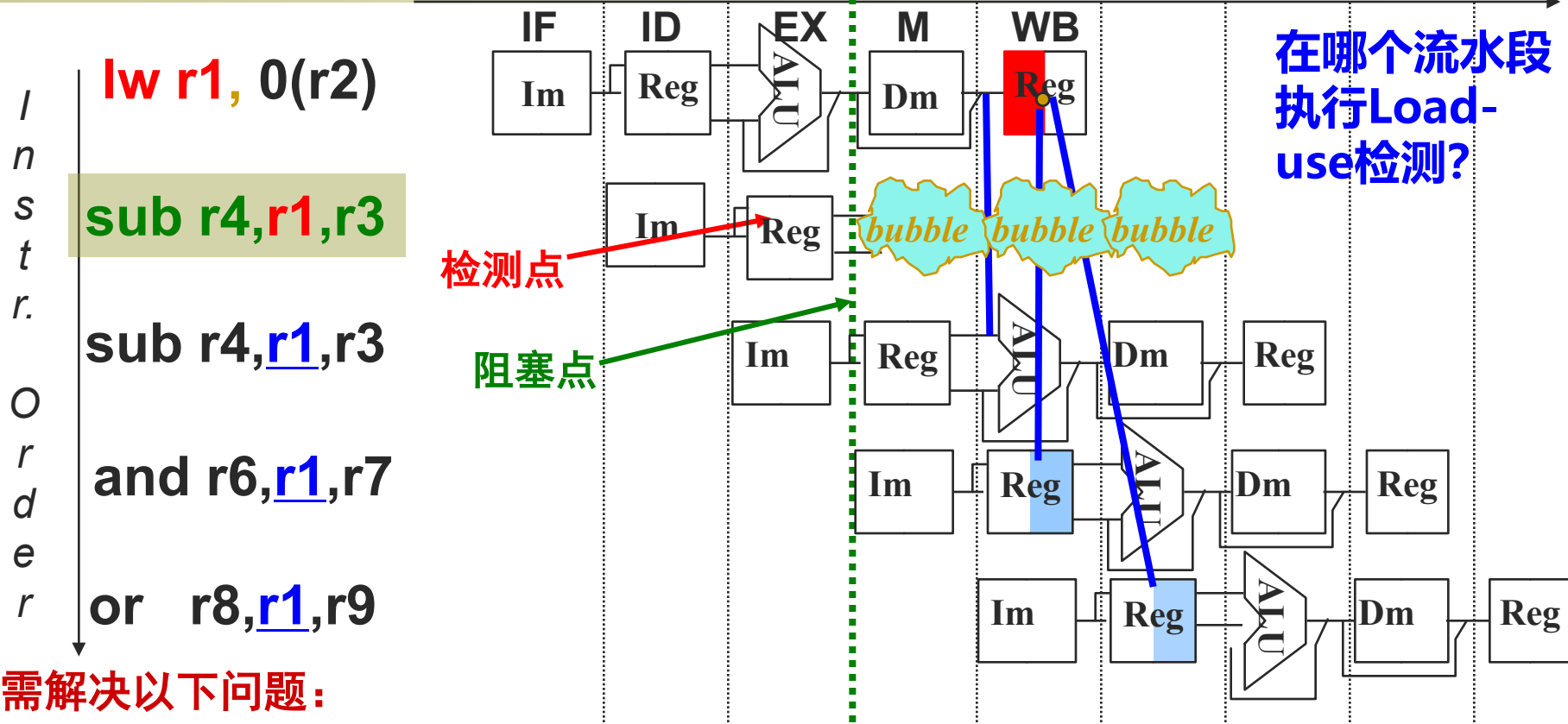
发生“Load-Use数据冒险”时，需要对Load后的指令阻塞一个时钟周期!

数据冒险处理硬件方案：“转发” + “Load-use阻塞”



Load-use Data Hazard (硬件阻塞方式)

Time (clock cycles)



需解决以下问题:

(1) 判断什么条件下需要阻塞

ID/EX.MemtoReg=1

&& (ID/EX.Rd==IF/ID.Rs1

|| ID/EX.Rd==IF/ID.Rs2)

隐含条件: 当前指令类型包含rs1或rs2字段!

上一条指令为Load 并且

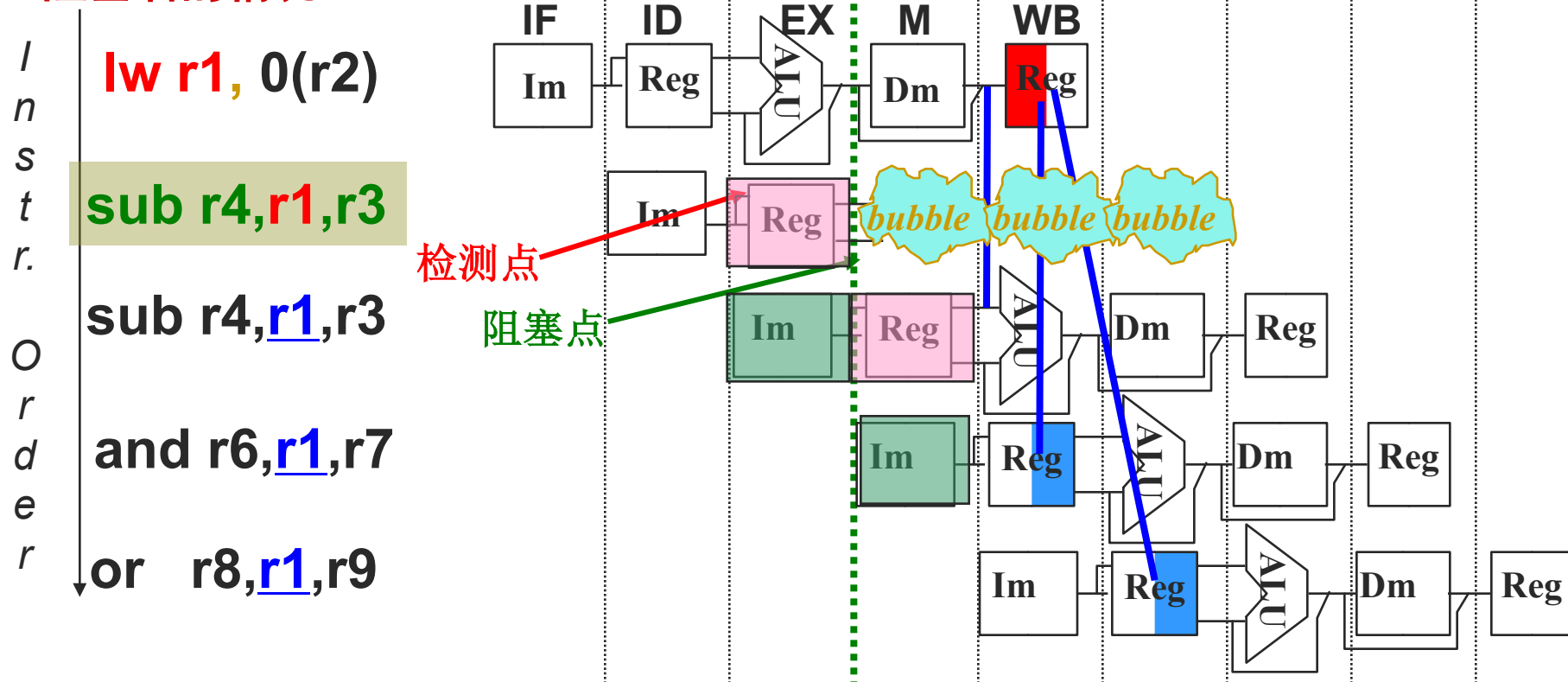
上一条指令的目的寄存器等于
当前刚取出指令的源寄存器

根据
id.opcode判
断指令类型

(2) 如何修改数据通路来实现阻塞

Load-use Data Hazard (硬件阻塞方式)

阻塞后的情况:



在阻塞点, 将当前指令的执行结果清除, 并延迟一个周期执行这条指令

- ① 将ID/EX段寄存器中所有控制信号清0, 以阻塞指令执行;
- ② IF/ID段寄存器中的信息不变 (还是sub指令), sub指令将重新译码执行
- ③ PC中的值不变 (还是and指令地址), and指令重新被取出执行



5.2 软硬协同

- 方法5：编译器进行指令**顺序调整**来解决数据冒险。

以下源程序可生成两种不同的代码，优化的代码可避免**Load-use阻塞**

$a = b + c; d = e - f;$

假定 a, b, c, d, e, f 在主存

编译器的优化很重要！

Slow code:

```
lw    t2, [b]
lw    t3, [c]
add   t1, t2, t3
sw    t1, [a]
lw    t5, [e]
lw    t6, [f]
sub   t4, t5, t6
sw    t4, [d]
```

调整后

Fast code:

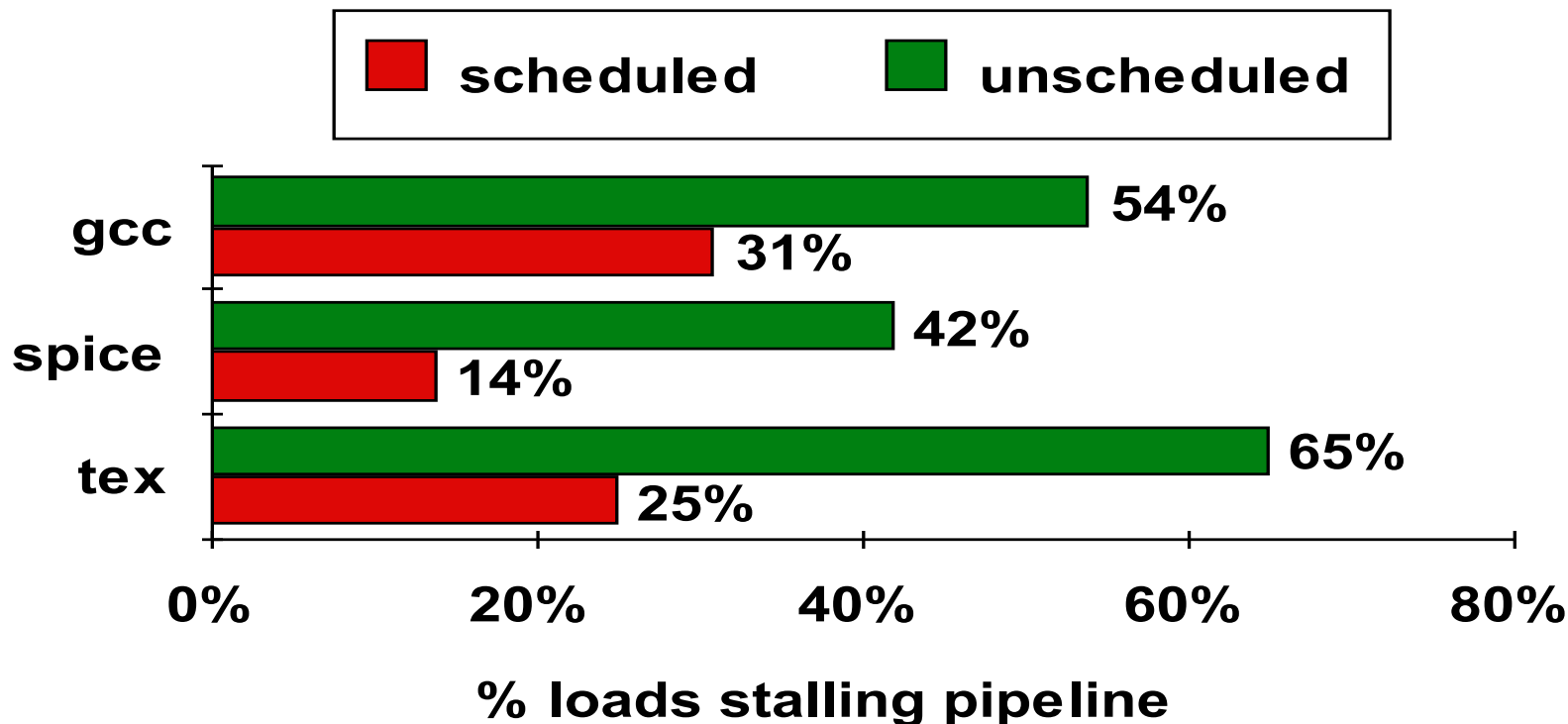
```
lw    t2, [b]
lw    t3, [c]
lw    t5, [e]
add   t1, t2, t3
lw    t6, [f]
sw    t1, [a]
sub   t4, t5, t6
sw    t4, [d]
```

免费性能提升：编译器通过数学证明重排序的安全性，在不增加晶体管成本的情况下重塑物理现实。



5.2 数据冒险

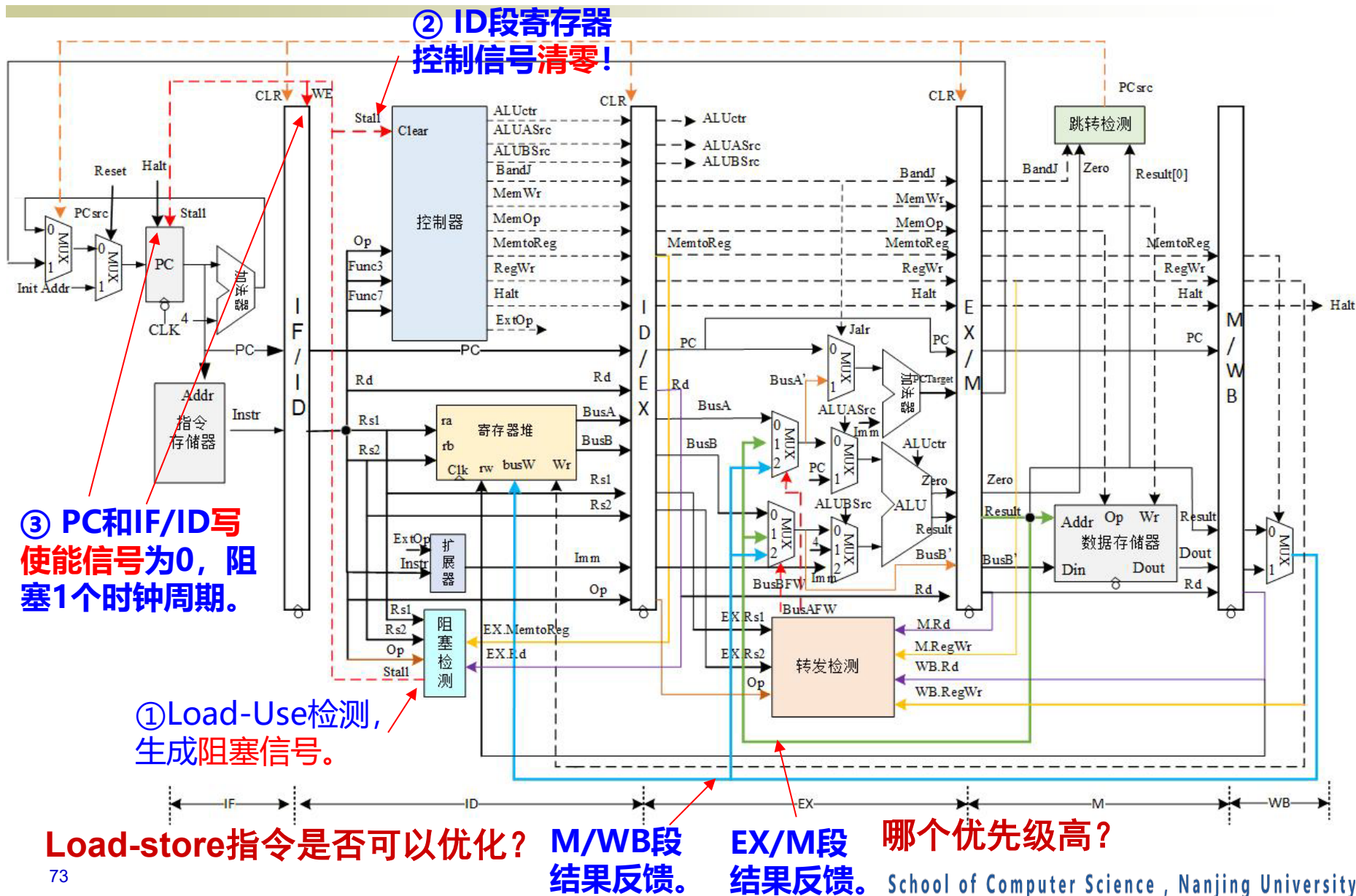
- 编译器优化以避免阻塞的情况调查:



由此可见，优化调度后load阻塞现象大约降低了1/2~1/3



带“阻塞”和“转发”检测的基础流水线CPU



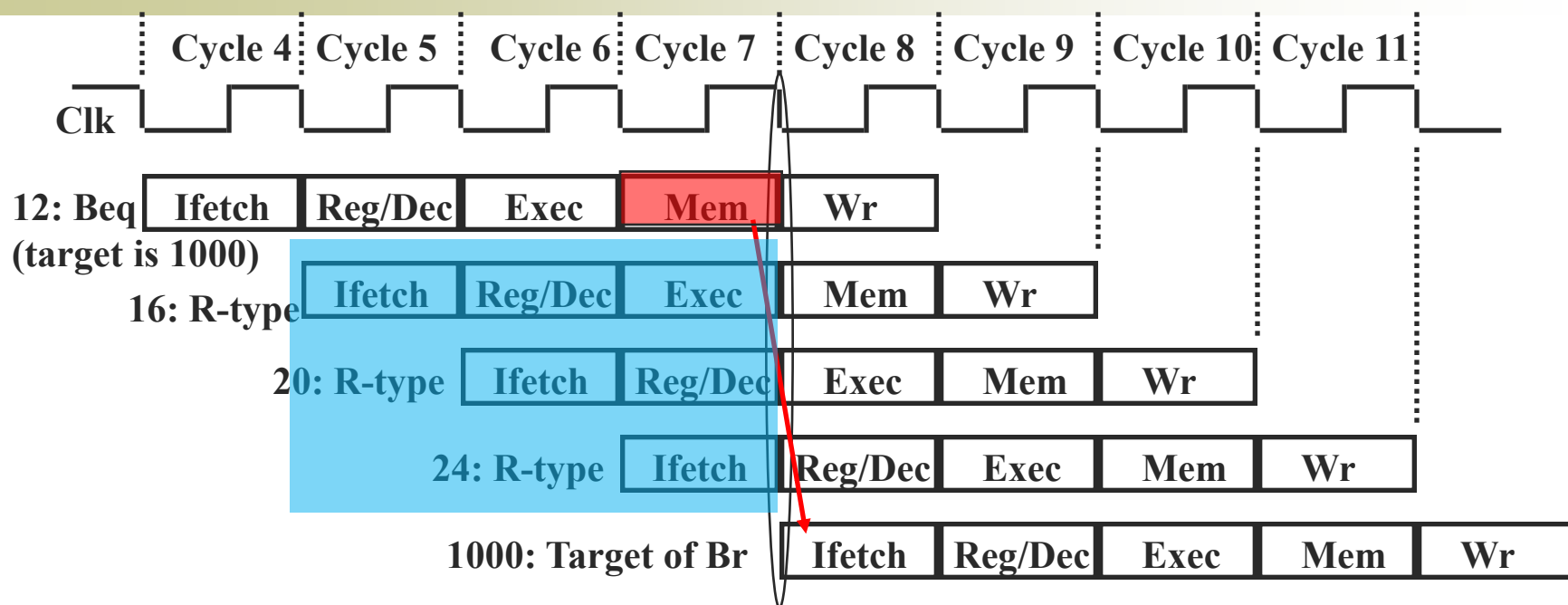


5.3 控制冒险：流水线的最深梦魇

- 由于发生了**指令执行顺序改变**而引起的流水线阻塞称为**控制冒险**。
 - 各类转移指令（包括调用指令、返回指令等），也称为**分支冒险**(Branch Hazards)
 - ◆ 硬件阻塞方式（插入气泡）或软件阻塞方式（插入空操作指令）
 - ◆ 简单（静态）预测
 - ◆ 动态预测
 - **异常和中断**
 - ◆ 通过在数据通路的不同流水段中加入相应的检测逻辑可检测出哪条指令发生了异常。



回顾:控制冒险现象



- 虽然Beq指令在第四周期取出，但：
 - “是否转移”在M阶段确定，目标地址在第七周期才被送到PC输入端
 - 第八周期才取出目标地址处的指令执行
- 结果：在取目标指令之前，已有三条指令被取出，取错了三条指令！
- 发生转移时，要清除Beq后面的三条指令，分别在EX、ID、IF段中
- 延迟损失时间片C：发生转移时，给流水线带来的延迟损失 这里 C=3



控制冒险的解决方法

- 方法1：软件上插入三条“NOP”指令
- 方法2：硬件上阻塞 (stall) 分支指令后三条指令的执行 在哪个阶段阻塞？
 - 使后面三条指令清0或其操作信号清0
- 方法3：分支预测 (Predict)
 - 简单 (静态) 预测：
 - ◆ 总是预测条件不满足(not taken)，即：继续执行分支指令的后续指令
 - ◆ 可加启发式规则：在特定情况下总是预测满足(taken)，其他情况总是预测不满足。如：循环顶部（底部）分支总是预测为不满足（满足）。能达到65%-85%的预测准确率
 - 动态预测：
 - ◆ 根据程序执行的历史情况进行动态预测调整，能达到90%的预测准确率

注：流水线控制必须确保被错误预测指令的执行结果不能生效，而且要能从正确的分支地址处重新启动流水线工作
- 方法4：延迟分支 (Delayed branch) （通过编译程序优化指令顺序！）
 - 把分支指令前面与分支指令无关的指令调到分支指令后执行，也称延迟转移

另一种控制冒险： 异常或中断控制冒险的处理



简单（静态）分支预测方法

■ 静态预测(Static Prediction)

- 策略：无视历史，一条路走到黑。

- 基本做法

- ◆ 总是预测条件不满足，即：继续执行分支指令的后续指令
可加启发式规则：

在特定情况下总是预测满足，其他情况总是预测不满足

- 预测失败时，需把流水线中三条错误预测指令丢弃掉

- ◆ 将指令或指令的控制信号清0（流水段寄存器中的数据清零）来实现（注：涉及当时在IF、ID和EX三个阶段的指令）

■ 评价

- 硬件极简，但在面对复杂的for/while循环结构时，预测效率极其低下。

■ 预测错误的代价

- 预测错误的代价与何时能确定是否转移有关。越早确定代价越少

- 可否把“是否转移”的确定工作提前，而不等MEM阶段才确定？

可以！ 那最早可以提前到哪个阶段呢？



分支预测方法

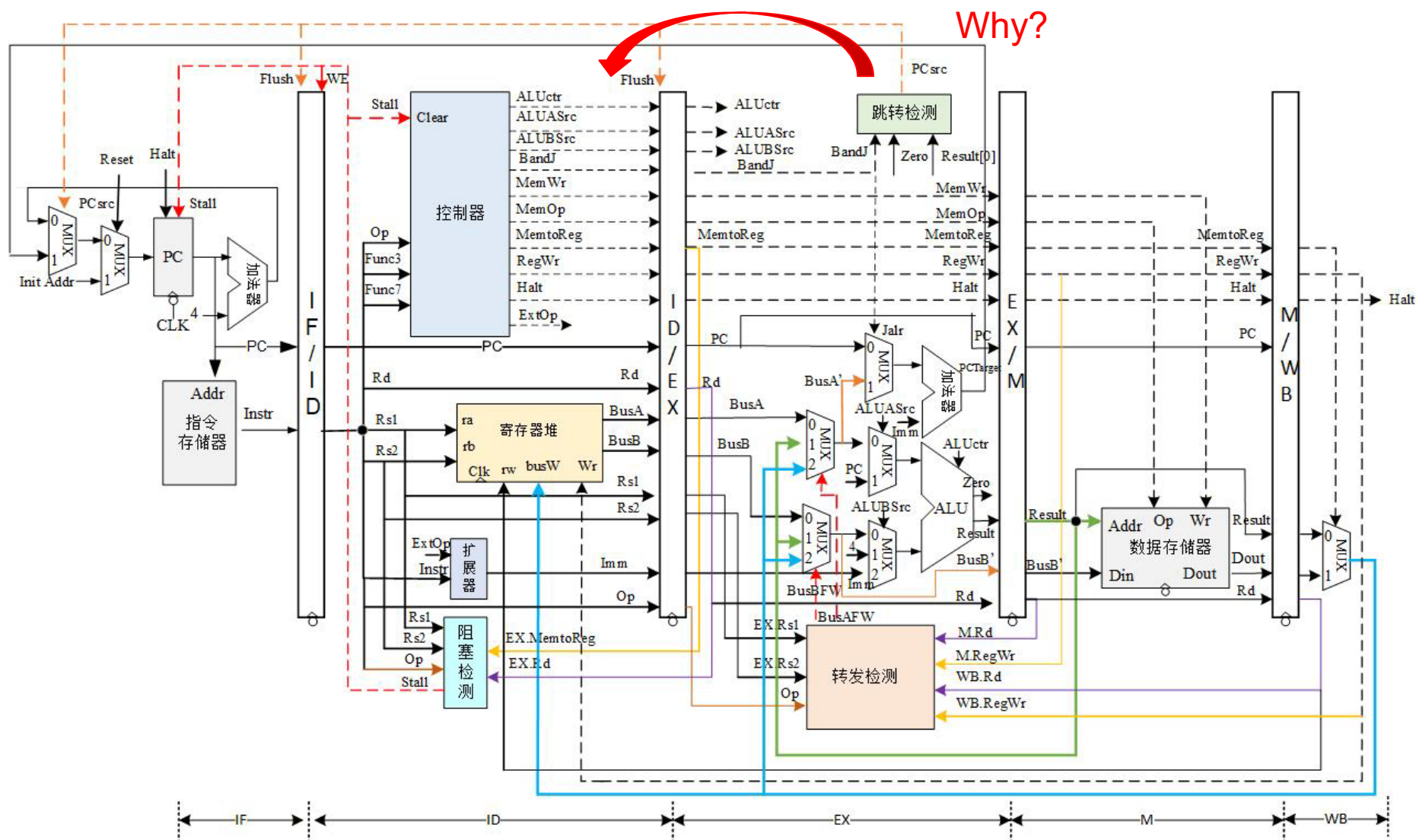
- **缩短**分支延迟，**减少**错误预测代价
 - 可以将“**转移地址计算**”和“**分支条件判断**”操作调整到**EX阶段**来缩短延迟
 - ◆ 将**转移地址**生成从M阶段移到**EX阶段**，可以吗？为什么？
(可：ID/EX流水段寄存器中已有PC的值、BusA和立即数Imm)
 - ◆ 将“**条件判断**”操作移到EX阶段，可以吗？为什么？
(可，已有Result、Zero和分支指令类型)
 - 能调整到**ID阶段**吗？
 - ◆ **Jmp指令可以，其他跳转和分支指令不能。** 需等待EX段转发数据！
- 预测错误的检测和处理（称为“**冲刷、冲洗**” -- Flush）
 - 当Branch=1并且Zero=1时，发生转移 (taken)
 - 增加控制信号：**IF.Flush=Branch && Zero**，取值为1时，说明预测失败
 - 预测失败 (条件满足) 时，完成以下两件事 (延迟损失时间片C=2时)：
 - ① **将转移目标地址->PC**
 - ② 清除IF、ID段中取出的指令，即：**将IF、ID段寄存器0**，转变为nop指令

原来要清除三条指令，调整后只需要清除二条指令，因而只延迟二个时钟周期，每次预测错误减少了一个周期的代价！ 即：这里 C=2

如果预测正确率是50%，则分支指令的执行时钟周期是多少？ 2个时钟周期



静态预测的基础流水线CPU设计





动态分支预测方法

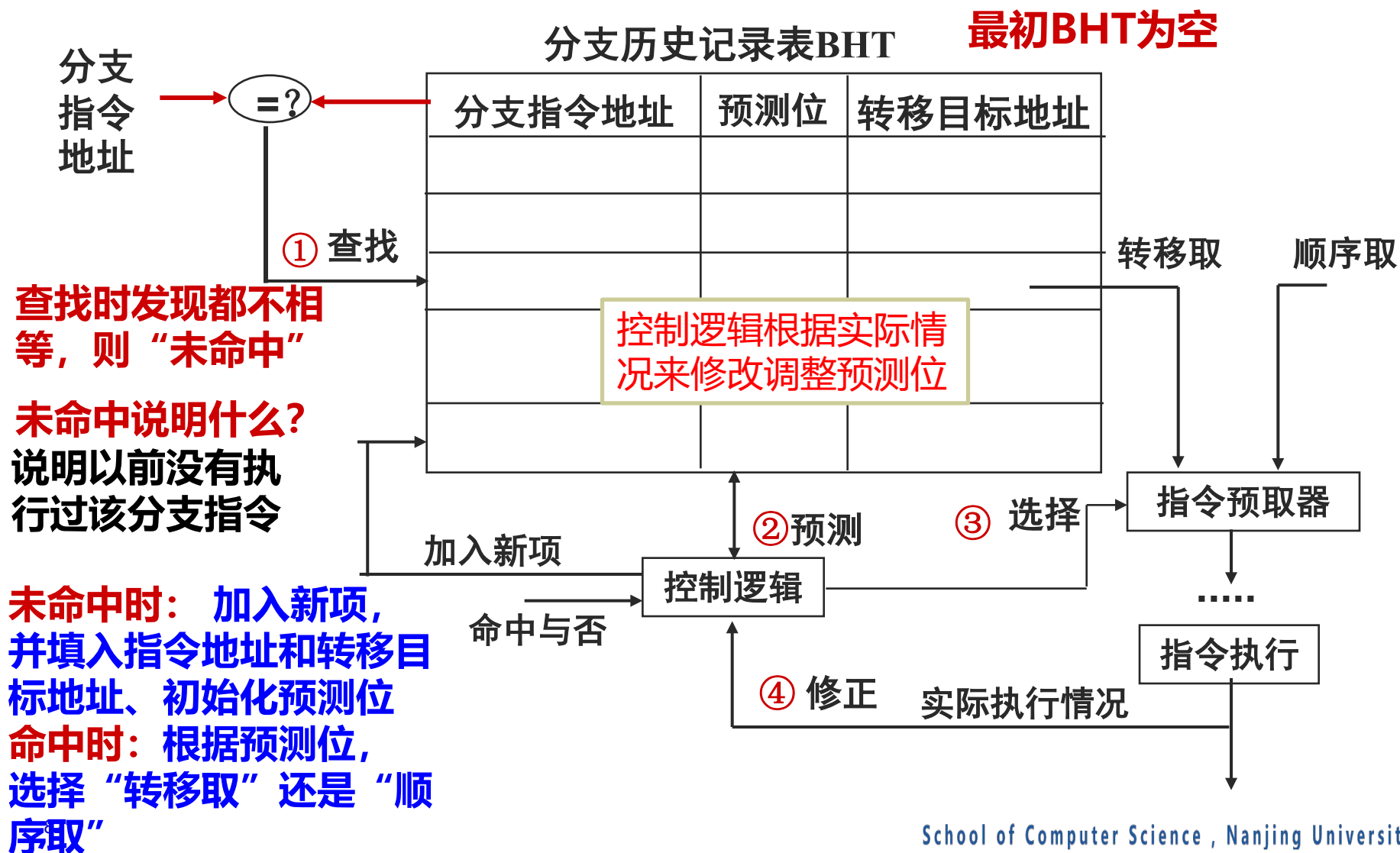
- 静态分支预测方法的预测成功率不高，应考虑动态预测
 - 动态预测的准确率可达90%
- 动态预测基本思想：
 - 利用最近转移发生的情况，来预测下一次是转移还是不转移
 - 根据实际情况来调整预测
 - 转移发生的历史情况记录在BHT中（有多个不同的名称）
 - ◆ 分支历史记录表BHT（Branch History Table）
 - ◆ 分支预测缓冲BPB（Branch Prediction Buffer）
 - ◆ 分支目标缓冲BTB（Branch Target Buffer）
 - 每个表项由分支指令地址低位作索引，故在IF阶段就可以取到预测位
 - ◆ 低位地址相同的分支指令共享一个表项，所以，可能取的是其他分支指令的预测位。会不会有问题？
 - ◆ 由于仅用于预测，所以不影响执行结果

现在几乎所有的处理器都采用动态预测（dynamic predictor）



5.3 控制冒险

■ 分支历史记录表BHT (或BTB、BPB)



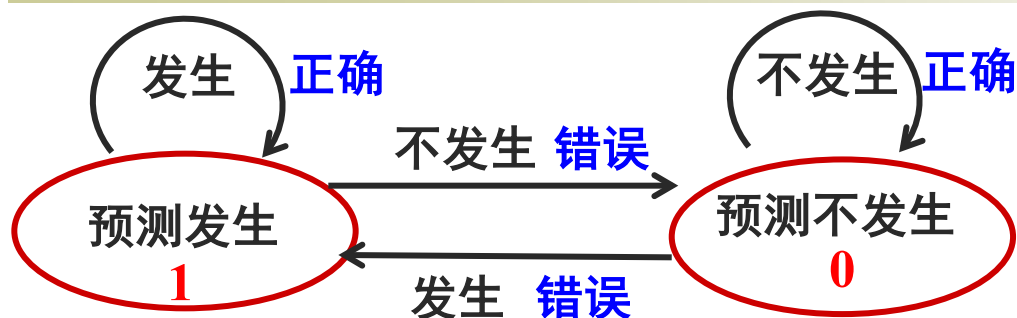


动态预测基本方法

- 采用一位预测位：总是按上次实际发生的情况来预测下次
 - 1表示最近一次发生过转移（taken），0表示未发生（not taken）
 - 预测时，若为1，则预测下次taken，若为0，则预测下次not taken
 - 实际执行时，若预测错，则该位取反，否则，该位不变
 - 可用一个简单的预测状态图表示
 - 缺点：当连续两次的分支情况发生改变时，预测错误
 - ◆ 例如，循环迭代分支时，第一次和最后一次会发生预测错误，因为循环的第一次和最后一次都会改变分支情况，而在循环中间各次总是会发生转移，按上次的实际情况预测时，都不会错。
 - 采用二位预测位
 - 用2位组合四种情况来表示预测和实际转移情况
 - 按照预测状态图进行预测和调整
 - 在连续两次分支发生不同时，只会有一次预测错误
- 采用较多的是二位或二位以上预测位。如：Pentium 4 的BTB2采用4位预测位



一位预测状态图



- 指令预取时，按照预测位读取相应分支的指令
 - 预测发生时，选择“转移取”
 - 预测不发生时，选择“顺序取”
- 指令执行时，按实际执行结果修改预测位
 - 对照状态转换图来进行修改
 - 例如：对于一个循环分支
 - ◆ 若初始状态为0(再次循环时为0)，则第一次和最后一次都错
 - ◆ 若初始状态为1，则只有最后一次会错。(再次循环时又改为0，还是有两次错)

```
int A [n];
Loop:  g = g + A[i];
      i = i + j;
      if (i != h) go to Loop:
Assuming variables g, h, i, j ~
t1, t2, t3, t4 and base address
of array is in t5
```

↓

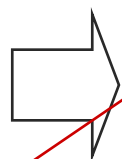
```
Loop: add t7, t3, t3      ; i*2
      add t7, t7, t7      ; i*4
      add t7, t7, t5
      lw t6, 0(t7)        ; t6=A[i]
      add t1, t1, t6       ; g= g+A[i]
      add t3, t3, t4       ; i=i+j
      bne t3, t2, Loop
      ... ..
```

即：只要本次和上次的发生情况不同，就会出现一次预测错误。



举例：双重循环的一位动态预测

```
into sum (int N)
{
  int i, j, sum=0;
  for (i=0; i < N; i++)
    for (j=0; j < N;
        j++)
      sum=sum+1;
  return sum;
}
```



```
... ..
Loop-i: beq t1,a0, exit-i  # 若( i=N)则跳出外循环
        add t2, zero, zero #j=0
Loop-j: beq t2, a0, exit-j  # 若(j=N)则跳出内循环
        addi t2, t2, 1      # j=j+1
        addi t0, t0, 1      #sum=sum+1
        j Loop-j
exit-j:  addi t1, t1, 1      # i=i +1
        j Loop-i
exit-i:  ... ..
```

外循环中的分支指令共执行 $N+1$ 次，
内循环中的分支指令共执行 $N \times (N+1)$ 次。

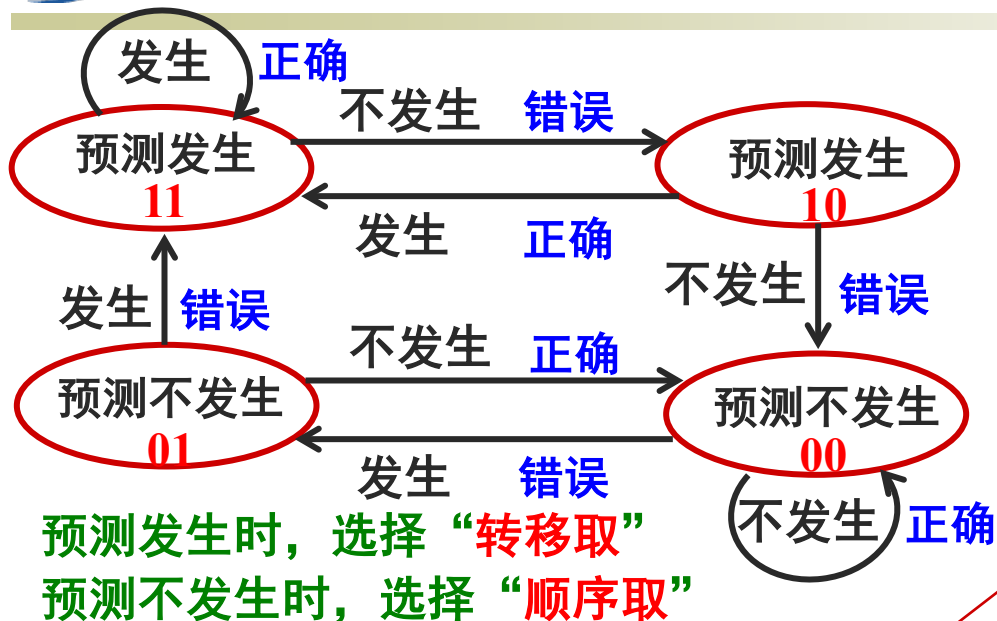
$N=10$, 准确率分别90.9%和82.7%

$N=100$, 准确率分别99%和98%

若预测位初始为**0**，则外循环只有**最后一次预测错误**；跳出内循环时预测位变为**1**，再进入内循环时，**第一次总是预测错误**，并且任何一次循环的**最后一次总是预测错误**，因此，总共有 $1+2 \times (N-1)$ 次预测错误（第一次内循环有1次预测错，后面 $(N-1)$ 次内循环每次有2次预测错）。 **N 越大预测准确率越高！**



两位预测状态图



```
Loop: add t7, t3, t3      ; i*2
      add t7, t7, t7      ; i*4
      add t7, t7, t5
      lw t6, 0(t7)        ; t6=A[i]
      add t1, t1, t6       ; g= g+A[i]
      add t3, t3, t4
      bne t3, t2, Loop
      ... ..
```

■ 基本思想：只有两次预测错误才改变预测方向

- 11状态时预测发生（强转移），实际不发生时，转到状态10（弱转移），下次仍预测为发生，如果再次预测错误（实际不发生），才使下次预测调整为不发生00

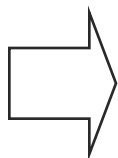
■ 好处：连续两次发生不同的分支情况时，会预测正确

- 例如，对于循环分支的预测（假定预测初始位为11）
- 第一次：初始态为11（再次进入循环时为10），预测发生，实际也发生，正确
- 中间：状态为“11”，预测发生，实际也发生，正确
- 最后一次：状态为“11”，预测发生，但实际不发生，错



举例：双重循环的两位动态预测

```
into sum (int N)
{
  int i, j, sum=0;
  for (i=0; i < N; i++)
    for (j=0; j < N; j++)
      sum=sum+1;
  return sum;
}
```



```
... ..
Loop-i: beq t1,a0, exit-i    # 若( i=N)则跳出外循环
        add t2, zero, zero  # j=0
Loop-j: beq t2, a0, exit-j   # 若(j=N)则跳出内循环
        addi t2, t2, 1      # j=j+1
        addi t0, t0, 1      # sum=sum+1
        j Loop-j
exit-j:  addi t1, t1, 1      # i=i +1
        j Loop-i
exit-i:  ... ..
```

外循环中的分支指令共执行 $N+1$ 次，

$N=10$, 准确率分别90.9%和90.9%

内循环中的分支指令共执行 $N \times (N+1)$ 次。

$N=100$, 准确率分别99%和99%

若预测位初始为00，外循环只有最后一次预测错误；跳出内循环时预测位变为01，再进入内循环时，第一次预测正确，只有最后一次预测错误，因此，总共有 N 次预测错误。

N 越大准确率越高！

[BACK](#)



综合题示例

- 在基于RV32I指令集模块实现的一个五段流水线CPU中执行以下指令序列：

假设：beq和jal指令在取数/译码阶段进行跳转目标地址修改，分支预测准确率为90%。不考虑异常、中断和访存缺失等引起的流水线阻塞。数据存储器地址0x00000002处存放的数据为变量n，设置的初始值为5。

```
      addi x12, x0, 1
      add x13, x0, x0
lw x10, 1(x12)
loop: addi x10, x10, -1
      add x13, x13, x12
      beq x10, x13, finish
      addi x12, x12, 1
      jal x0, loop
```

(1) 哪些指令之间存在数据相关？其中哪些可用转发技术解决？

(2) 哪些指令会发生控制冒险？

(3) 写出指令“beq x10, x13, finish”对应的指令机器码。

(4) 代码从第1条指令进入流水段开始算起，执行该段代码总共需要多少个时钟周期？对应的平均CPI为多少（保留一位小数）？

(5) 在保证代码功能不变的情况下，如何通过修改这段代码的顺序来优化其执行性能？修改后总共需要执行多少个时钟周期？

(6) 若该段代码改用一个单周期CPU来执行，则需要多少个时钟周期？你认为可否据此对比分析单周期CPU和流水线CPU的性能好坏？为什么？

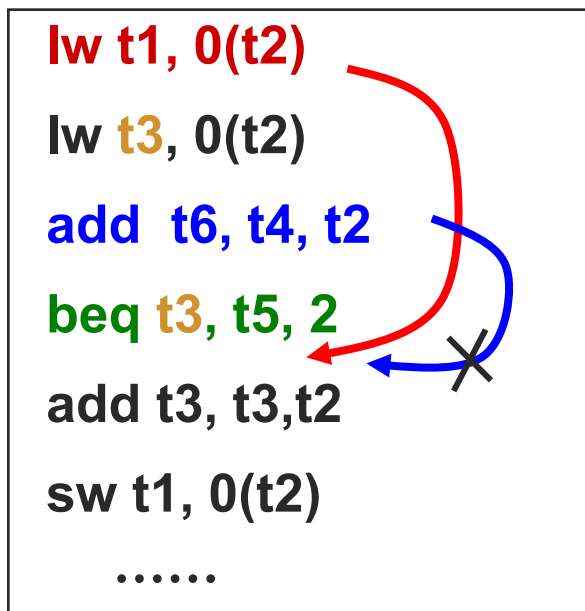


分支延迟时间片的调度

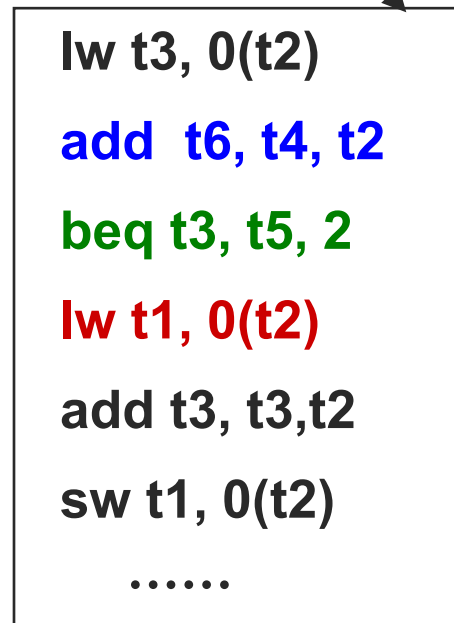
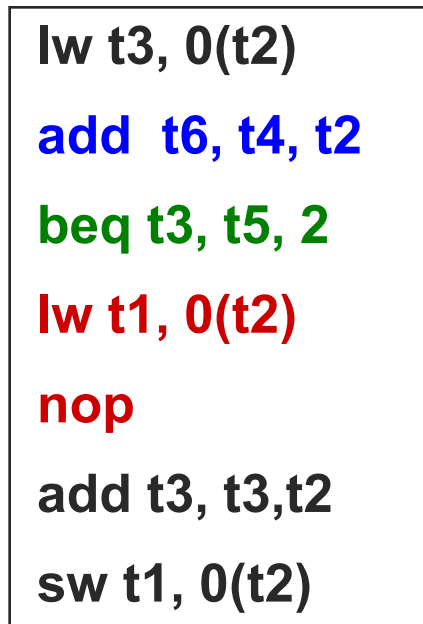
- 属于静态调度技术，由编译程序重排指令顺序来实现。
- 基本思想：把分支指令前面与分支指令无关的指令调到分支指令后面执行，以填充延迟损失时间片（也称分支延迟槽Branch Delay slot），不够时用nop

举例：如何对以下程序段进行分支延迟调度？（假定时间片为2）

若分支条件判断和目标地址计算提前到ID阶段，则分支延迟时间片减少为1



调度后



调度后可能带来其他问题：
产生新的load-use数据冒险

调度后，无需在硬件线路中阻塞branch指令后面指令的执行

[BACK](#)



控制冒险解决方案

解决方案	预测准确率	硬件复杂度	软件依赖度
停顿冲刷 (Stall & Flush)	极低	极低	无
静态预测 (Static Prediction)	中等	低	无
延迟槽 (Delay Slots)	中高	低	极高
动态预测 (Dynamic BHT)	极高	极高	无

RISC-V架构抛弃延迟槽技术！

架构权衡：没有完美的解决方案。

极高的准确率必然伴随硬件的复杂性膨胀以及极高的功耗。



5.3 异常和中断冒险处理

- 异常和中断会改变程序的执行流程。
- 某条指令发现异常时，后面多条指令已被取到流水线中正在执行。
 - 例如ALU指令发现“溢出”时，已经到EX阶段结束了，此时，它后面已有两条指令进入流水线了。
- 流水线数据通路如何处理异常？(举例说明)
 - 假设指令add r1,r2,r3产生了溢出
 - 处理思路：
 - ◆ 清除add指令以及后面的所有已在流水线中的指令
 - ◆ 关中断（将中断允许触发器清0）
 - ◆ 保存PC或PC-4（断点）到EPC
 - ◆ 将异常/中断处理程序首地址送PC



实例

- 分析下列两个相似的程序执行时间相差近10倍的原因。
 - 区别在于阶乘计算拆分

```
#include <stdlib.h>
#include <stdio.h>
#include <time.h>
// test_1.c
// 为方便讲解，禁止GCC把calc()内联
__attribute__((noinline))
long int calc(int n)
{
    long int fact = 1;
    for(int i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
}
```

```
long int calc(int n)
{
    register int fact0 = 1, fact1 = 1, fact2 = 1,
    fact3 = 1;
    for (int i = 1; i < n; i += 4) {
        fact0 *= i;
        fact1 *= i + 1;
        fact2 *= i + 2;
        fact3 *= i + 3;
    }
    return fact0 * fact1 * fact2 * fact3;
}
```

real	0m0.442s
user	0m0.437s
sys	0m0.000s

real	0m0.056s
user	0m0.056s
sys	0m0.000s



实例

- 分析遍历相同大小两个数组时间相差近10倍的原因?
 - 区别在于数组赋值不同

```
#include <stdlib.h>
#include <stdio.h>
#include <time.h>
```

```
#define SIZE (0x100000000)
int array[SIZE];
```

```
int calc() {
    int a = 0, b = 0;
    for(int i = 0; i < SIZE; i++) {
        if(array[i] > 500) {
            a++;
        } else {
            b++;
        }
    }
    return a - b;
}
```

real	0m5.248s
user	0m4.743s
sys	0m0.491s

```
int main() {
    for(int i = 0; i < SIZE; i++) {
        array[i] = rand() % 1000;
        // array[i] = i;
    }
```

```
    clock_t start = clock(); // 标记开始时间
```

```
    calc();
```

```
    clock_t end = clock(); // 标记结束时间
    double cost = (double)(end - start) /
CLOCKS_PER_SEC; // 总耗时
    printf("Time Cost: %.3fs\n", cost);
```

```
    return 0;
}
```

real	0m0.986s
user	0m0.462s
sys	0m0.519s



5.4 三种处理实现方式比较

- 单周期、多周期、流水线三种方式比较CPU执行时间

假设各主要功能单元的操作时间为：

存储单元：200ps

ALU和加法器：100ps

寄存器堆（读 / 写）：50ps

假设MUX、控制单元、PC、扩展器和传输线路都没有延迟，指令组成为：

25%取数、10%存数、52%ALU、11%分支、2%跳转，则下面实现方式中，哪个更快？快多少？

- (1) 单周期：每条指令在一个固定长度的时钟周期内完成
- (2) 多周期：每类指令时钟数为取数-5，存数-4，ALU-4，分支-3，跳转-3
- (3) 流水线：每条指令分取指令、取数/译码、执行、存储器存取、写回五阶段（假定没有结构冒险，数据冒险采用转发处理，分支延迟槽为1，预测准确率为75%；不考虑异常、中断和访存缺失引起的流水线冒险）

流水线CPU中每类指令的执行周期？



5.4 三种处理实现方式比较

解：CPU执行时间 = 指令条数 $\times$ CPI $\times$ 时钟周期长度

三种方式指令条数都一样，所以只要比较CPI和时钟周期长度即可。

Instruction class	Functional units used by the instruction class				
	Instruction fetch	Register access	ALU	Register access	
R-type	Instruction fetch	Register access	ALU	Register access	
Load word	Instruction fetch	Register access	ALU	Memory access	Register access
Store word	Instruction fetch	Register access	ALU	Memory access	
Branch	Instruction fetch	Register access	ALU		
Jump	Instruction fetch				

各指令类型要求的时间长度为：

Instruction class	Instruction memory	Register read	ALU operation	Data memory	Register write	Total
R-type	200	50	100	0	50	400 ps
Load word	200	50	100	200	50	600 ps
Store word	200	50	100	200		550 ps
Branch	200	50	100	0		350 ps
Jump	200					200 ps



5.4 三种处理实现方式比较

对于单周期方式：

时钟周期将由最长指令来决定，应该是load指令，为600ps

所以，N条指令的执行时间为 $600N(\text{ps})$

对于多周期方式：

时钟周期将取功能部件最长所需时间，应该是存取操作，为200ps

根据各类指令的频度，计算平均时钟周期数为：

$\text{CPU时钟周期} = 5 \times 25\% + 4 \times 10\% + 4 \times 52\% + 3 \times 11\% + 3 \times 2\% = 4.12$

所以，N条指令的执行时间为 $4.12 \times 200 \times N = 824N(\text{ps})$



5.4 三种处理实现方式比较

对于流水线方式：

Load指令：当发生Load-use依赖时，执行时间为2个时钟，否则1个时钟，故平均执行时间为1.5个时钟；

Store、ALU指令：1个时钟；

Branch指令：预测成功时，1个时钟，预测错误时，2个时钟，

所以：平均约为： $0.75 \times 1 + 0.25 \times 2 = 1.25$ 个；

Jump指令：2个时钟（总要等到译码阶段结束才能得到转移地址）

平均CPI为： $1.5 \times 25\% + 1 \times 10\% + 1 \times 52\% + 1.25 \times 11\% + 2 \times 2\% = 1.17$

所以，N条指令的执行时间为 $1.17 \times 200 \times N = 234N(\text{ps})$ 结果正确吗？

综上所述，流水线方式的执行速度最快，

与单周期相比，约为 $600\text{ps} / 234\text{ps} = 2.56$ 倍；

与多周期相比，约为 $824\text{ps} / 234\text{ps} = 3.52$ 倍；



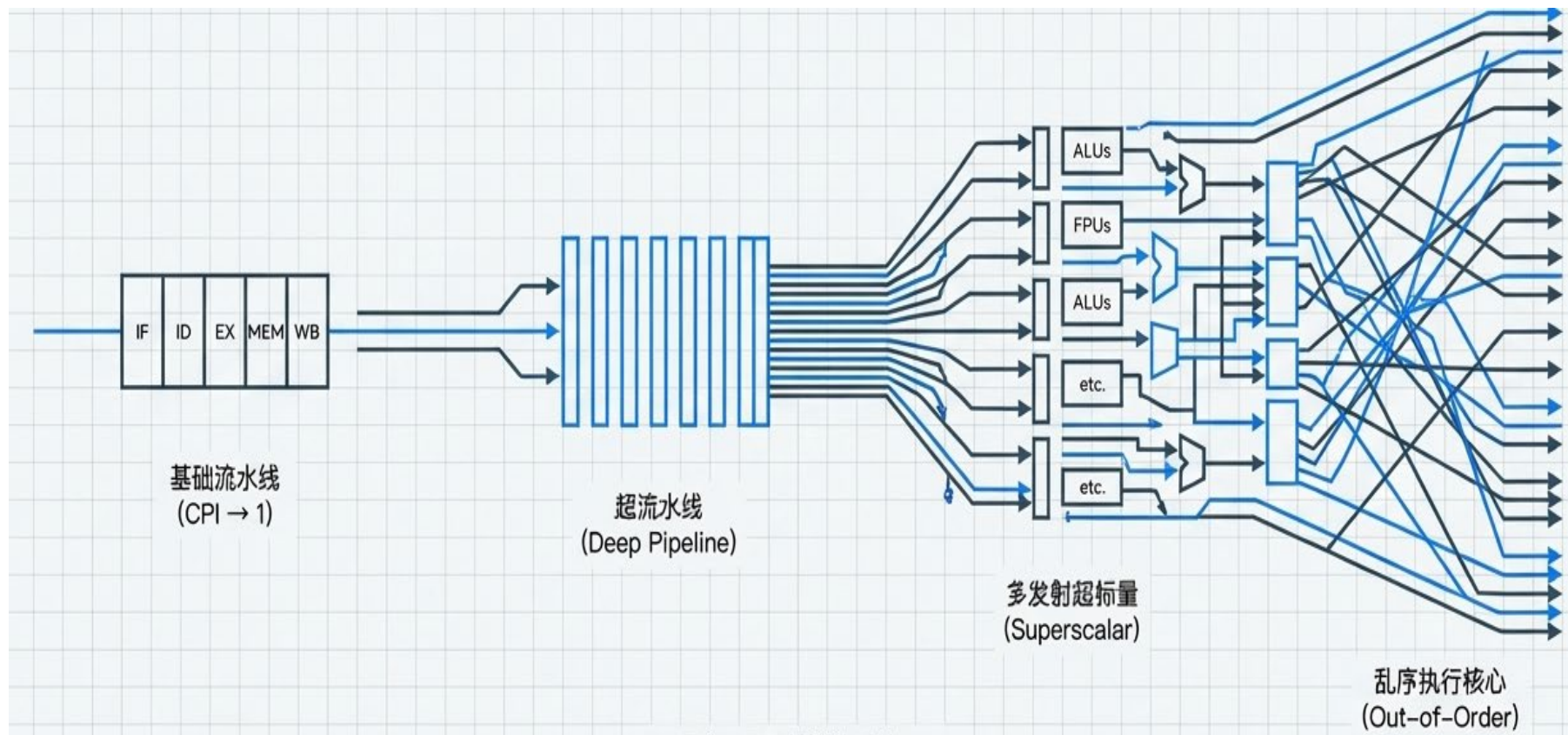
小结

- 流水线冒险的几种类型：
 - 资源冲突、数据相关、控制相关（改变指令流的执行方向）
- 数据冒险的现象和对策
 - 数据冒险的种类
 - ◆ 相关的数据是ALU结果，可以通过转发解决
 - ◆ 相关的数据是DM读出的内容，随后的指令被阻塞一个时钟
 - 数据冒险和转发
 - ◆ 转发检测 / 转发控制
 - 数据冒险和阻塞
 - ◆ 阻塞检测 / 阻塞控制
- 控制冒险的现象和对策
 - 静态分支预测技术
 - 缩短分支延迟技术
 - 动态分支预测技术
- 异常和中断是一种特殊的控制冒险

在Logisim中完成RV32I流水线CPU设计，并通过验收的同学，平时成绩和实验成绩优秀！



终极追寻：突破CPI= 1的物理极限





6 高级流水线技术

- 高性能流水线 - 指令级并行(Instruction-Level Parallelism , ILP)技术
 - 超流水线
 - 多发射流水线
 - ◆ 静态多发射 (VLIW处理器 + 编译器静态调度)
 - ◆ 动态多发射 (超标量处理器 + 动态流水线调度)
- 静态多发射 (VLIW-超长指令字)
 - 编译器静态推测完成“指令打包”和“冒险处理”
 - RV32I 2-发射流水线数据通路
 - 循环展开指令调度
 - IA-64的显式并行指令运算 (Explicitly Parallel Instruction Computer, EPIC)
- 动态多发射
 - 动态多发射流水线的通用模型
 - 动态多发射流水线的执行模式
 - ◆ 按序发射、按序完成
 - ◆ 按序发射、乱序完成
 - ◆ 乱序发射、乱序完成
 - Pentium 4 动态多发射流水线
 - ◆ 超流水、超标量、动态调度、乱序发射、乱序完成



6 高级流水线技术

N段流水线说明一个时钟周期内最多有几条指令同时并行执行?

N条! 故N越大并行度越高!

增加
的开销
体现在
哪里?

流水
段寄存
器!

- 实现指令流内部的并行流水线称为**指令级并行** (ILP)
- 有**两种**指令级并行策略

- **超流水线** (Super- pipelining)

- ◆ 级数更多的流水线 **CPI = ? CPI = 1**
 - ◆ 理想情况下, 流水线的加速比与流水段的数目成正比
(即: 理想情况下, 流水段越多, 时钟周期越短, 指令吞吐率越高)
- 但是, 它会增加开销, 且是有极限的! 可以怎样突破极限呢?

- **多发流水线** (Multiple issue pipelining)

- ◆ 多条指令(如整数运算、浮点运算、装入/存储等) 同时启动并独立运行
- ◆ **前提**: 有多个执行部件。如: 定点、浮点、乘/除、取数/存数部件等
- ◆ **结果**: 能达到小于1的CPI, 定义CPI的倒数为**IPC**
(例如: 理想的四路多发流水线的IPC为4)
- ◆ 需要实现以下两个主要任务
 - **指令打包**: 分析每个周期发射多少条? 哪些指令可以同时发射?
 - **冒险处理**: 由编译器静态调整指令或在运行时由硬件处理
- ◆ 实现上述两个主要任务的基础—**推测技术**
- ◆ 两种实现方法
 - **静态多发**: 由编译器在编译时静态完成指令打包和冒险处理
 - **动态多发**: 由硬件在执行时动态完成指令打包和冒险处理



实现多发射技术的基础—推测

- 推测技术：由编译器或处理器**猜测**指令执行结果，并以此**调整**指令执行顺序，使指令的执行能达到最大可能的**并行**。
 - 指令打包的决策依赖于“推测”的结果
 - ◆ 可根据指令间的相关性来进行推测
 - **与前面指令不相关的指令可以提前执行**
 - ◆ 可对分支指令进行推测
 - **可提前执行分支目标处的指令**
 - ◆ 预测仅是“猜测”，可能推测错误，故需有**推测错误检测**和**回滚机制**
 - ◆ 推测错误时，会增加额外开销
 - 有“软件推测”和“硬件推测”两种
 - ◆ **软件推测**：**编译器**通过推测来**静态重排指令**（一定要正确！）
 - ◆ **硬件推测**：**处理器**在运行时通过推测来动态调度指令



6.1 静态多发射处理器

- 由编译器在编译时进行相关性分析和静态分支预测，以静态完成“指令打包”和“冒险处理”
 - 指令打包（将同时发射的多条指令合并到一个长指令中）
 - ◆ 将同一个时钟周期内发射的多个指令看成是一条多个操作的长指令，称为一个“发射包”
 - ◆ “静态多发射”也被称为“超长指令字”（VLIW-Very Long Instruction Word），采用这种技术的处理器被称为VLIW处理器
 - ◆ 在同一个周期内发射的指令类型是受限制的
例如，只能是一条ALU指令/分支指令、一条Load/Store指令
 - ◆ IA-64采用这种方法，Intel称其为EPIC（Explicitly Parallel Instruction Computer—显式并行指令计算机）



6.1 静态多发射处理器

- 由编译器在编译时进行相关性分析和静态分支预测，以静态完成“指令打包”和“冒险处理”
 - 冒险处理（主要是数据冒险和控制冒险）
 - ◆ 做法1：完全由编译器通过代码调度和插入nop指令来消除所有冒险，无需硬件实现冒险检测和流水线阻塞
 - ◆ 做法2：由编译器通过静态分支预测和代码调度来消除同时发射指令间内部依赖，由硬件检测数据冒险并进行流水线阻塞

即：保证打包指令内部不会出现冒险！



6.1 静态多发射处理器

■ 实例：RV32I 指令集的静态多发射----2发射处理器

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

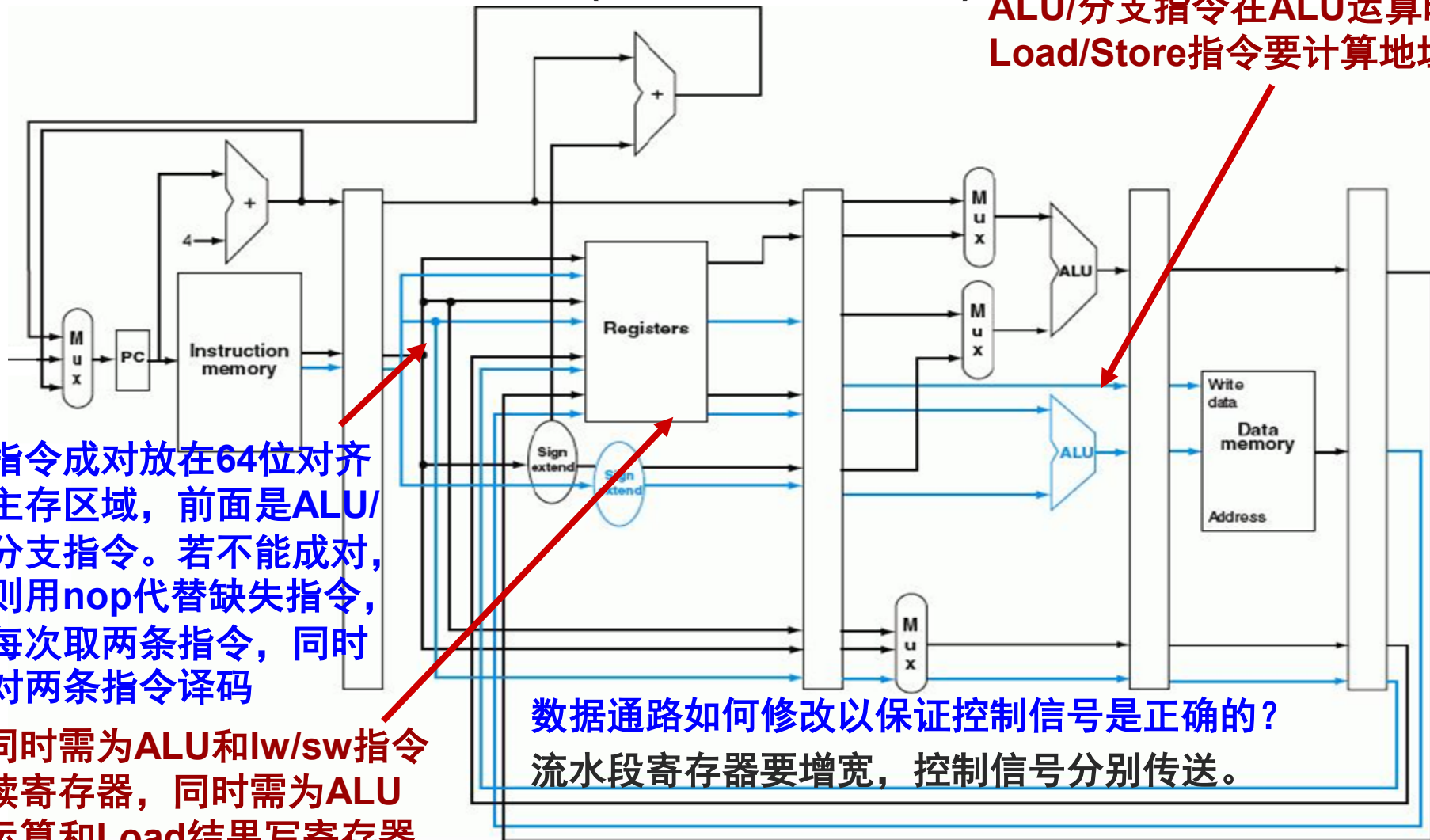
- 要使RV32I能够同时处理**两条**流水线，数据通路需要做哪些改进？
 - 1. 同时要能取并译码两条指令，怎么办？
 - ◆ 将两条指令**打包**成64位长指令，前面为ALU/Branch，后面为lw/sw
 - ◆ 没有配对指令时，就用nop指令代替
 - ◆ 将64位长指令中的两个操作码同时送到控制器（指令译码器）进行译码
 - 2. 两条指令同时要读两个寄存器或写寄存器（和lw配对时），怎么办？
 - ◆ 增加两个读口和一个写口
 - 3. 两条指令同时要使用ALU进行运算，怎么办？
 - ◆ 增加一个ALU（包括两组输入总线 and 一组输出总线）



6.1 静态多发射处理器

- 2发射流水线数据通路（蓝色是增加部分）

需增加额外加法器或ALU
ALU/分支指令在ALU运算时，
Load/Store指令要计算地址





6.1 静态多发射处理器

- 2发射流水线的特点
- 优点：潜在性能将提高大约2倍（实际上达不到！为什么？）
- 缺点：
 - 为消除结构冒险，需增加额外部件
 - 增加了潜在的由于数据冒险和控制冒险导致的性能损失
 - ◆ 例1：对于Load-use数据冒险
 - **单发射流水线：**只有一条指令延迟
 - **2发射流水线：**有一个周期（即2条指令）延迟
 - ◆ 例2：对于ALU-Load/Store数据冒险
 - **单发射流水线：**可用“转发”技术使ALU结果直接转发到Load/Store指令的EX阶段
 - **2发射流水线：**两条指令同时进行，ALU的结果不能直接转发，因而不能提供给与其配对的Load/Store指令使用，只能延迟一个周期

为更有效地利用多发射处理器的并行性，必须有更强大的编译器，能够充分消除指令间的依赖关系，使指令序列达到最大的并行性！



6.1 静态多发射处理器

- 例：2发射指令调度
- 以下是一段循环代码段

```
Loop: lw    t0, 0(s1)
```

这段程序的功能是？

```
    addi    t0, t0, 1
    sw      t0, 0(s1)
```



循环内进行的
是数组访问！

```
    addi    s1, s1, -4
    bne     s1, zero, Loop
```

- 为了能在2发射流水线中有效执行，该怎样重新排列指令
 - 调度方案如下：没有指令配对时，用nop指令代替

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi s1, s1, -4 ?	lw t0, 0(s1)	1
	addi s1, s1, -4		2
	addi t0, t0, 1		3
	bne s1, zero, Loop	sw t0, 4(s1)	4



一个循环内，五条指令在四个时钟内完成，实际CPI为0.8，即：IPC=1.25

在循环中访问数组的更好的调度技术是“循环展开” School of Computer Science, Nanjing University

前三条和后两条各具有相关性
 可把第四条指令调到第一条后面
sw指令是否有问题？怎么办？

s1减4，故sw指令偏移改为4

能否把addi和lw配成一对？

寄存器s1被同时读，并读后写，两条指令的操作不会相互影响。但是



6.1 静态多发射处理器

- 基本思想：展开循环体，生成多个副本，在展开的指令中统筹调度
- 上例采用“循环展开”后的指令序列是什么？
 - 为简化起见，假定循环执行次数是4的倍数
 - “循环展开”4次后循环内指令（lw, addu, sw与数组访问相关）各有4条，再加1条addi和1条bne，共14条指令
 - 指令最佳调度序列如下：

为何第一条指令将s1减16？
与t0关联的指令偏移为何不同？

	ALU or branch instruction	Data transfer instruction	Clock cycle
Loop:	addi s1, s1, -16	lw t0, 0(s1)	1
		lw t1, 12(s1)	2
	addi t0, t0, 1	lw t2, 8(s1)	3
	addi t1, t1, 1	lw t3, 4(s1)	4
	addi t2, t2, 1	sw t0, 16(s1)	5
	addi t3, t3, 1	sw t1, 12(s1)	6
		sw t2, 8(s1)	7
	bne s1, zero, Loop	sw t3, 4(s1)	8

14条指令用了8个时钟，CPI达到 $8/14=0.57$ 。

需要用到“重命名寄存器”技术，多用了三个临时寄存器t1,t2,t3，消除了名字依赖关系（非真实依赖，只是寄存器名相同而已）

代价是什么？多用了三个临时寄存器，并增加了代码大小（存储空间变大）

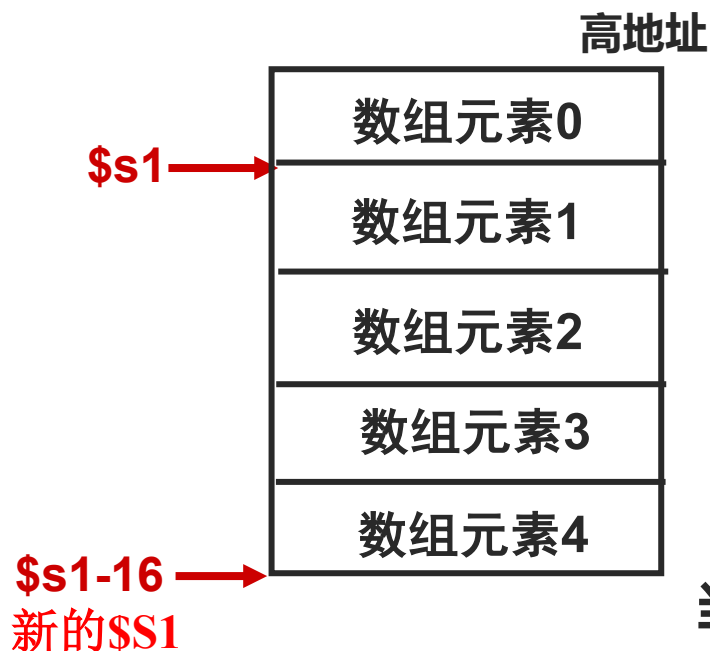
好处：充分利用并行，并消除部分循环分支！



6.1 静态多发射处理器

■ 循环展开后的偏移量

- 第一条指令将s1减16，指令执行后，s1的值变成了循环结束时s1的值
- 所以循环体内各数组元素的访问指令的偏移量依次为：
16 - 数组元素0， 12 - 数组元素1， 8 - 数组元素2， 4 - 数组元素3



为什么第一个周期中的lw指令的偏移量为0?
因为第一个周期中的lw指令进行地址计算时，addi指令的执行结果还没有写到s1中，所以，此时s1中还是原来的值。

为什么第一条addi指令不放在第二周期?
为了避免load-use数据冒险!

当循环次数不是4的倍数时这样做就有问题!

**可见：编译器和机器结构密切相关！系统程序员必须非常了解机器结构！
编译器的好坏直接影响程序执行快慢！**



6.2 动态多发射处理器

- 由硬件在执行时动态完成**指令打包**或**冒险处理**
- 通常被称为超标量处理器 (Superscalar)
 - 同一个时钟动态发射多条指令，一个周期内可执行一条以上指令
- 与VLIW处理器的不同点：
 - **VLIW处理器**：与机器结构密切相关，在结构有差异的机器上要重新编译
 - **超标量处理器**：编译器仅进行指令顺序调整（还是串行序列），不进行指令打包，而是由硬件根据机器结构决定同时发射哪几条指令。因此，编译后的代码能够被不同结构的机器正确执行
- 超标量处理器多结合动态流水线调度 (Dynamic pipeline scheduling) 技术
 - 通过指令相关性检测和动态分支预测等手段，投机性地不按指令顺序执行，当发生流水线阻塞时，可以到后面找指令来执行
 - **举例说明动态流水线调度技术**：



6.2 动态多发射处理器

■ 举例说明动态流水线调度技术

lw	t0, 20(s2)
add	t1, t0, t2
sub	s4, s4, t3
slti	t5, s4, 20

左边指令序列中，哪条指令可以提前执行？

sub指令可提前到add指令前执行

如果不将sub调到前面，则会影响slti指令的执行，而且还会发生load-use冒险

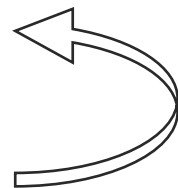
■ 动态流水线调度的通用模型

■ 动态流水线的一个重要思想：在等待解决阻塞时，到后面找指令提前执行！

■ 动态流水线的通用模型：

- 一个指令预取和译码单元：有序发射
- 多个并列执行的功能单元：乱序执行
- 一个提交单元：有序提交

add t0, t0, s2
sw t0, 0(s1)
addi s1, t0, -4
mul t1, t1
○ ○ ○ ○ ○



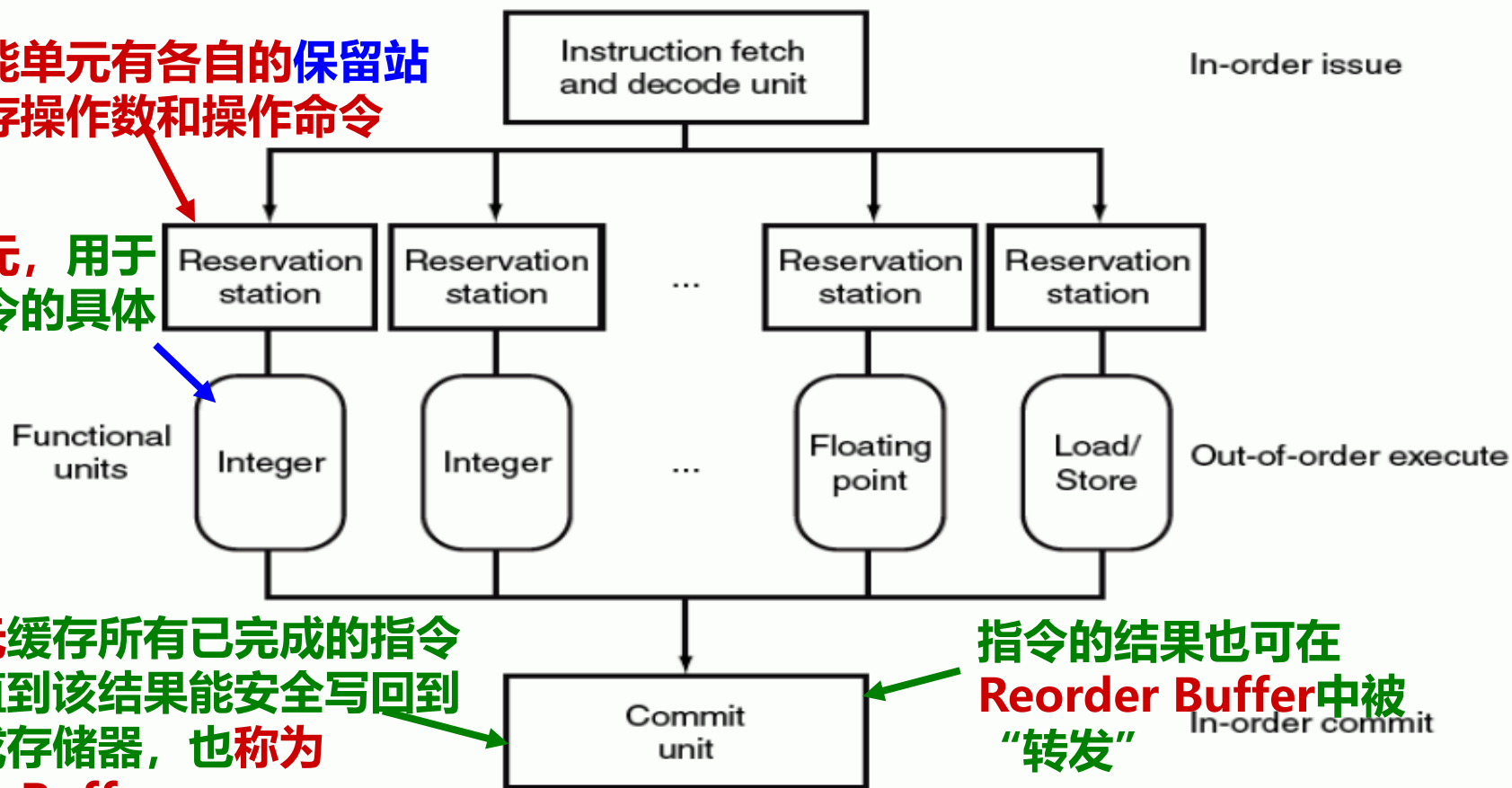


6.2 动态多发射处理器

■ 动态流水线调度技术

每个功能单元有各自的保留站
用于保存操作数和操作命令

功能单元，用于
完成指令的具体
功能



提交单元缓存所有已完成的指令
结果，直到该结果能安全写回到
寄存器或存储器，也称为
Reorder Buffer

指令的结果也可在
Reorder Buffer中被
“转发”



6.2 动态多发射处理器

- 功能单元的性能
- 功能：用来执行特定类型的操作
- 性能：每个功能单元具有基本的操作性能，用两个周期数来刻画
 - 执行周期（Latency）：完成特定操作所花的周期数
 - 发射时间（Issue Time）：连续、独立操作之间的最短周期数

以下是Pentium III 算术功能部件的性能

表 8.7 Pentium III 的功能部件性能

操作类型	执行周期数	发射时间
整数加法	1	1
整数乘法	4	1
整数除法	36	36
浮点数加法	3	1
浮点数乘法	5	2
浮点数除法	38	38
取数 (cache 命中) 存	3	1
数 (cache 命中)	3	1

从上述图中看出，哪些功能部件是流水化的？哪些是非流水化的？

• 整数加、整数乘、浮点加、装入、存储这5种部件是流水化的

• 浮点乘部件是部分流水化

• 整数除和浮点除是完全没有流水化

CPU设计的一个原则：有限的芯片空间应该在各功能部件之间进行平衡！尽量让大多数资源用于最关键的操作（对大量基准程序进行评估）

从上述图中能否看出，哪些是最重要的操作？哪些是不常用的？



6.2 动态多发射处理器

- 根据动态流水线指令发射和完成顺序，可分为三种执行模式：
 - 按序发射按序完成 (Pentium)
 - 按序发射无序完成 (Pentium II和Pentium III)
 - 无序发射无序完成 (Pentium 4)



6.2 动态多发射处理器

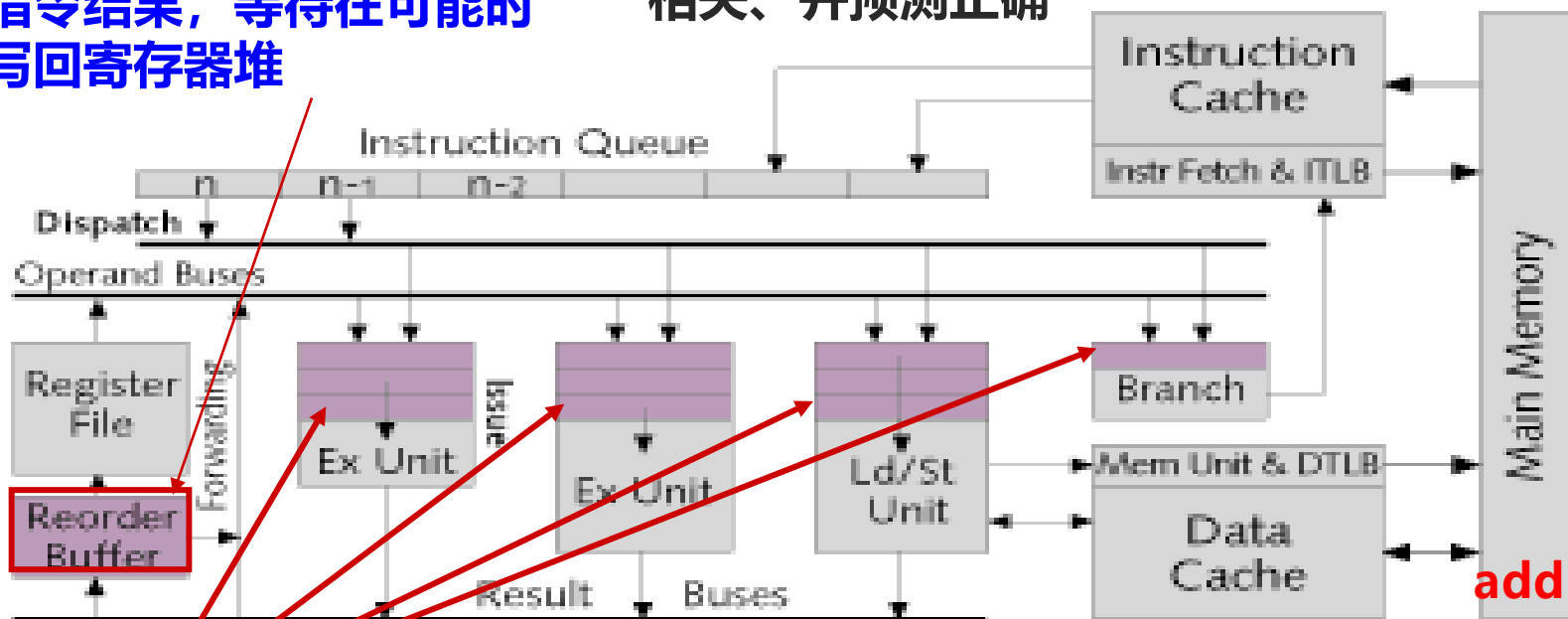
■ 动态流水线的几种执行模式

ReOrder Buffer 重排序缓冲: 用于保存已完成的指令结果, 等待在可能时写回寄存器堆

写回条件: 与前面的所有指令结果不相关、并预测正确

保守的是顺序完成, 好处:

- (1) 简化异常检测及其处理
- (2) 能在被推测指令完成前得知推测结果的正确性



保留站: 存放操作数和操作命令

add t0, t0, s2
sw t0, 0(s1)
addi s1, t0, -4
mul t1, t1

• • • • •

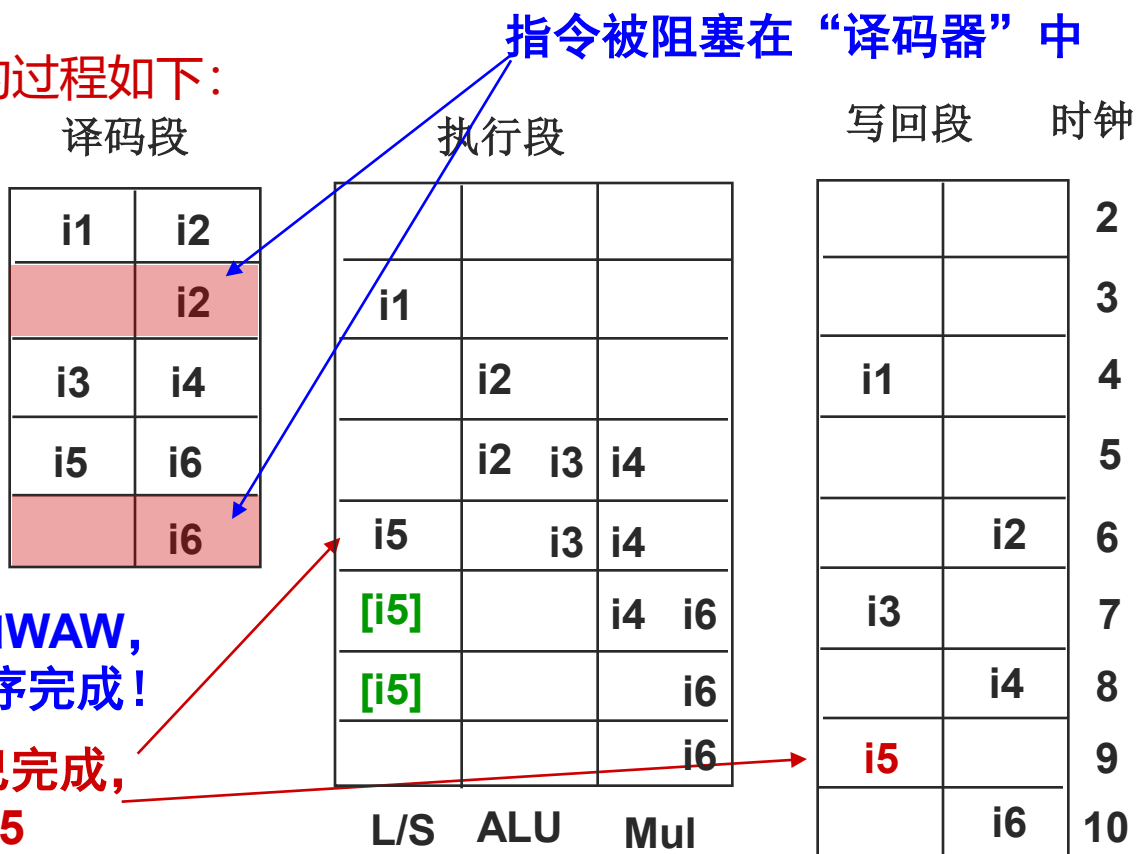


按序发射按序完成

- 举例：2发射超标量，分为取指（F）、译码（D）、执行（E）、写回（W）。F、D、W段在一个时钟周期内完成（可同时有两条指令在这三个阶段）；E段有三个执行部件：Load/Store部件完成数据Cache访问需要1个时钟，整数ALU完成简单ALU操作需2个时钟，整数乘法器完成乘法运算需要3个时钟。执行部件采用流水化方式。

按序发射(到E阶段)按序完成的过程如下：

i1 lw r1, A
i2 add r2, r2, r1
i3 add r3, r3, r4
i4 mul r4, r5, r4
i5 lw r6, B
i6 mul r6, r6, r7



i1、i2间RAW; i5、i6间RAW和WAW, 需阻塞一个时钟周期，并须按序完成！

为了按序完成，虽i5在时钟6已完成，但一直推迟到i4写回后才写回i5

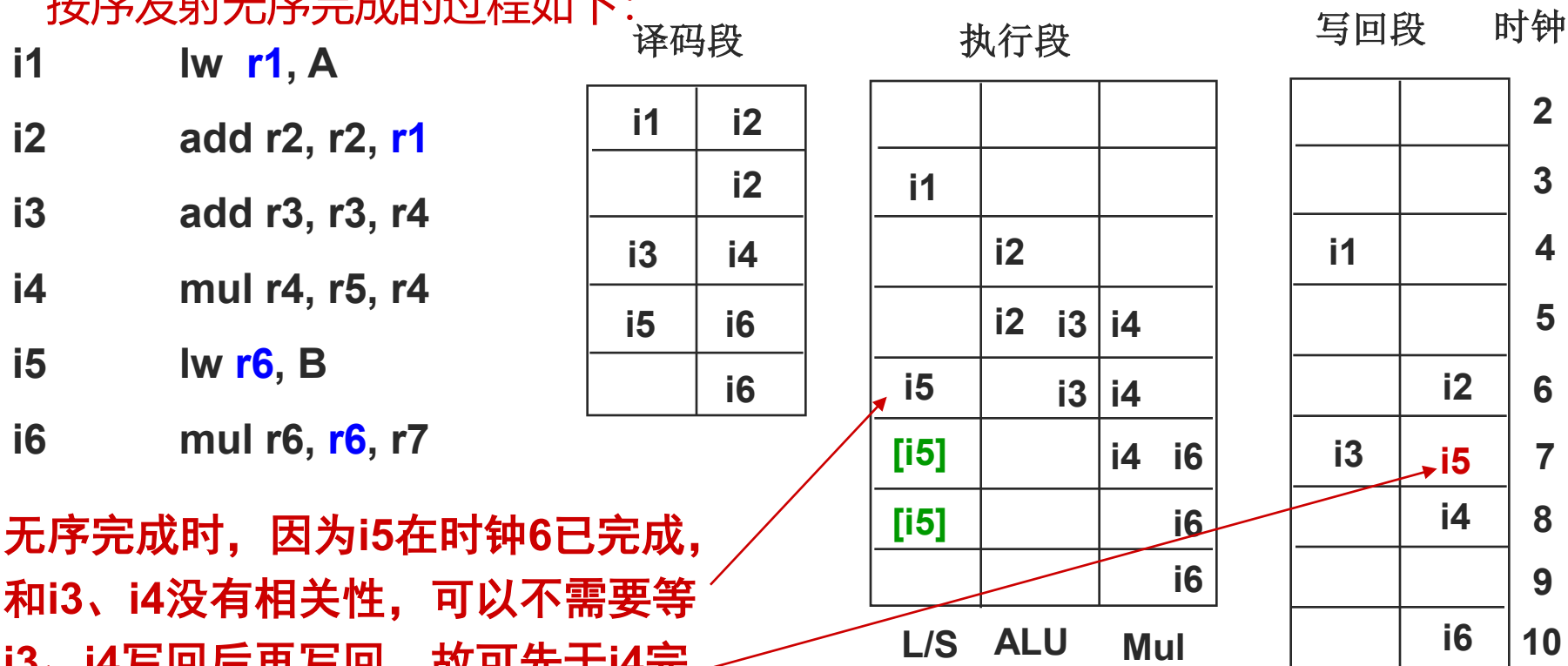
如果还有一条乘法指令，则最多可有三条乘法指令同时在执行



按序发射无序完成

- 举例：2发射超标量，分为取指（F）、译码（D）、执行（E）、写回（W）。F、D、W段在一个时钟周期内完成（可同时有两条指令在这三个阶段）；E段有三个执行部件：Load/Store部件完成数据Cache访问需要1个时钟，整数ALU完成简单ALU操作只需2个时钟，整数乘法器完成乘法运算需要3个时钟。执行部件采用流水化方式。

按序发射无序完成的过程如下：





无序发射无序完成

- 举例：2发射超标量，分为取指（F）、译码（D）、执行（E）、写回（W）。F、D、W段在一个时钟周期内完成（可同时有两条指令在这三个阶段）；E段有三个执行部件：Load/Store部件完成数据Cache访问需要1个时钟，整数ALU完成简单ALU操作只需2个时钟，整数乘法器完成乘法运算需要3个时钟。执行部件采用流水线化方式。

取指和译码按顺序进行，发射前进行相关性检测，无关指令可先行发射和先行完成！

例如：i4在i3前面发射！

无序发射无序完成的过程如下：

i1 lw r1, A

i2 add r2, r2, r1

i3 add r3, r3, r4

i4 mul r4, r5, r4

i5 lw r6, B

i6 mul r6, r6, r7

译码段

i1	i2
i3	i4
i5	i6
i7	i8

执行段

i1		
	i2	i4
i5	i2	i3
	i3	i4
		i6
		i6

L/S ALU Mul

写回段 时钟

		2
		3
i1		4
		5
i2	i5	6
i3	i4	7
		8
	i6	9
		10

无序发射的超标量中，译码后的指令被存放在一个“指令窗口”的缓冲器中，等待发射。当所需功能部件可用、且无冲突或无相关性阻碍指令执行时，就从指令窗口发射，与取指和译码的顺序无关。

只要保证i1和i2、i5和i6之间的发射和完成顺序即可！



动态流水线调度的必要性

- 编译器可依据数据依赖关系调度代码，为什么还要超标量处理器来动态调度？

- 并不是所有阻塞都能事先确定，动态调度可在阻塞时，提前执行无关指令

- ◆ 例如，Cache缺失是不可预见的阻塞

- 动态分支预测需要根据执行的真实情况进行预测

- 采用动态调度使得硬件将处理器细节屏蔽起来

(不同处理器的发射宽度、流水线延时等可能不同，流水线的结构也会影响循环展开的深度。通过动态调度使得处理器细节被屏蔽起来，软件发行商无需针对同一指令集的不同处理器发行相应的编译器，并且，以前的代码也可在新的处理器上运行，无需重新编译)

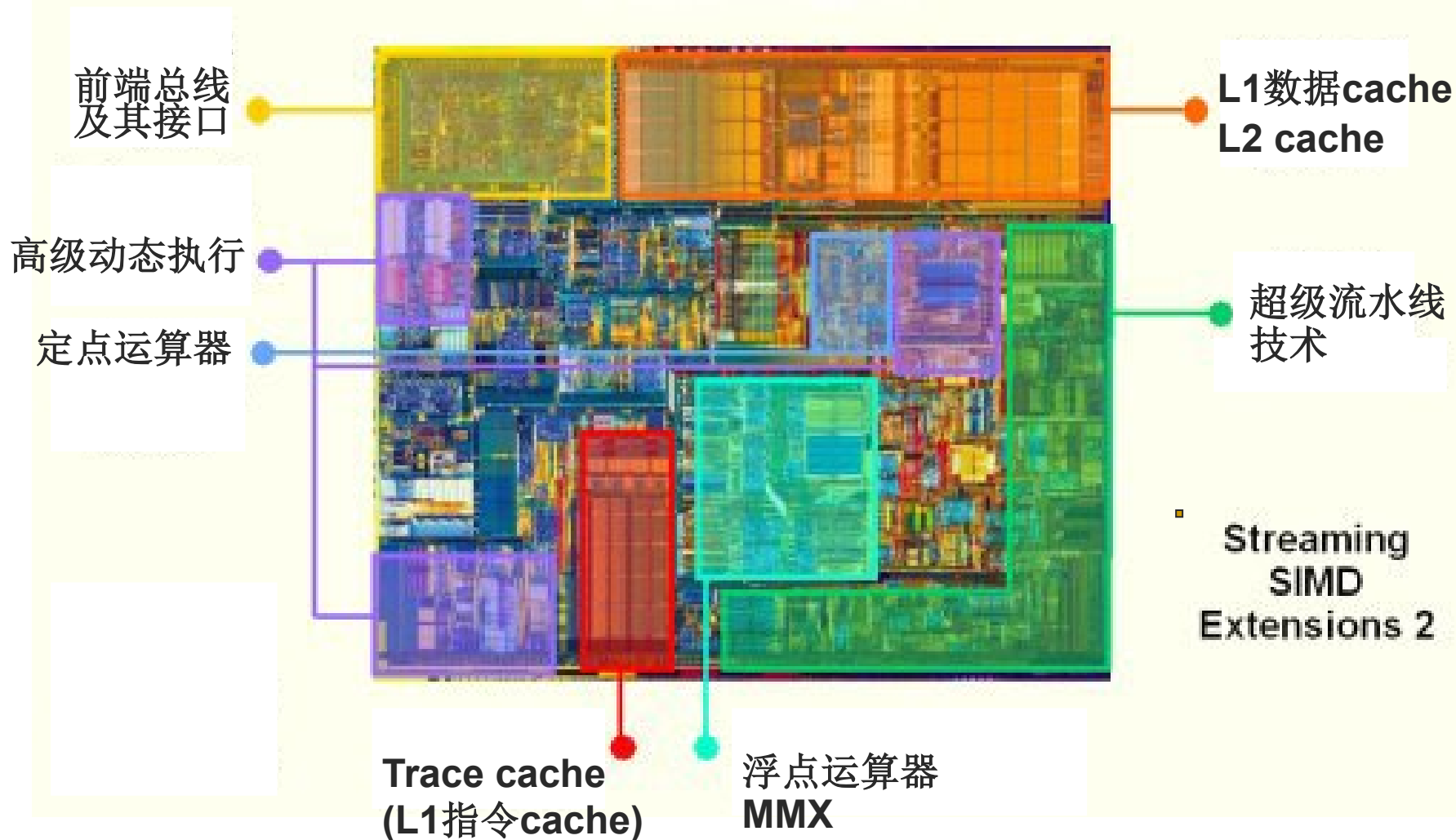
理解程序的性能：

高性能微处理器并不能持续进行多条指令的发射，原因有：

- (1) 指令间的高度依赖关系限制了指令之间的并行执行，特别是隐含依赖关系的存在。例如，使用指针的代码段，存在隐含依赖。
- (2) 分支指令预测错误。
- (3) 内存访问引起的阻塞（Cache缺失、缺页等）使得流水线难以满负荷运转。



Pentium 4 处理器的芯片布局

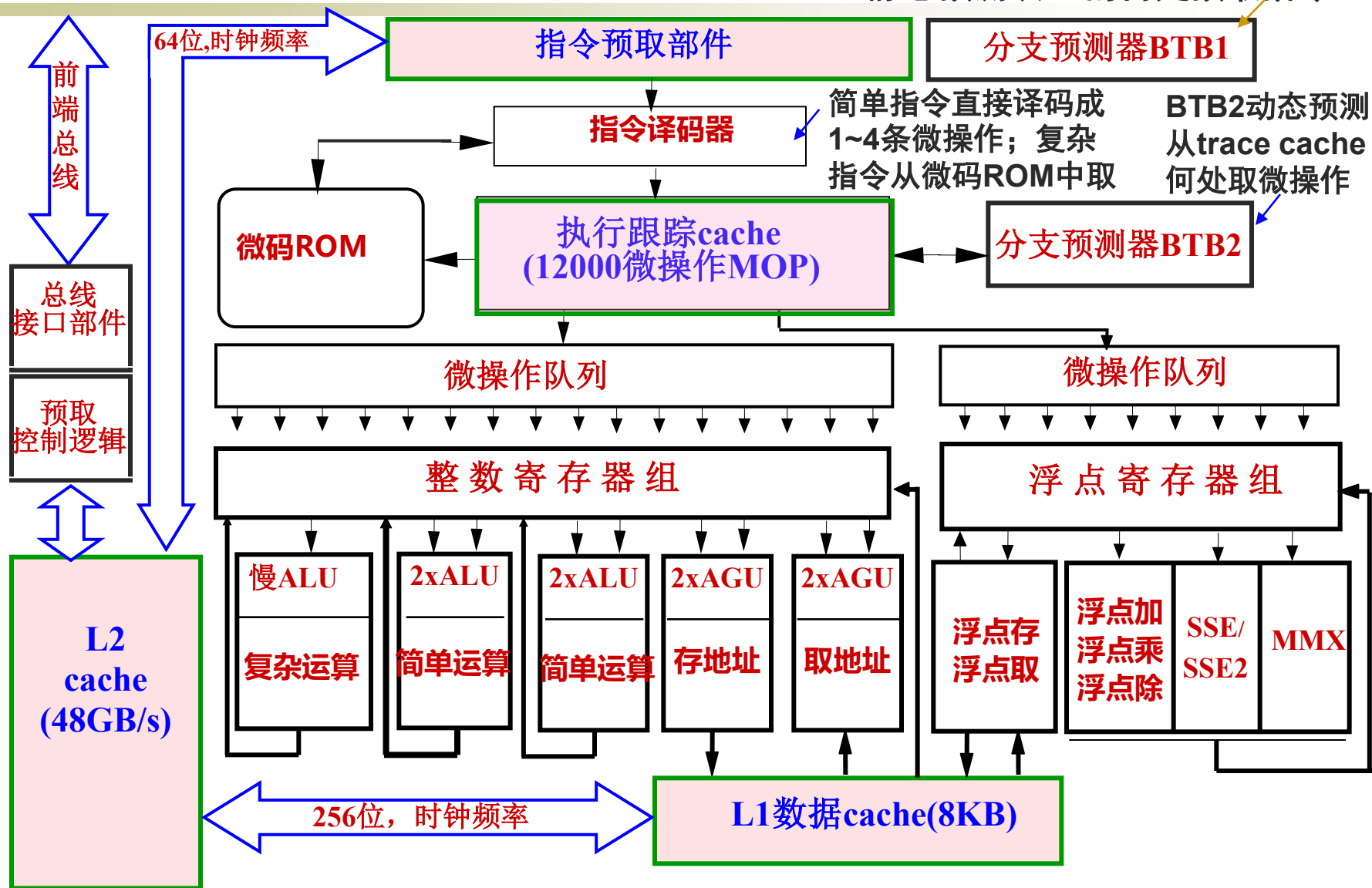




Pentium 4 处理器的逻辑结构

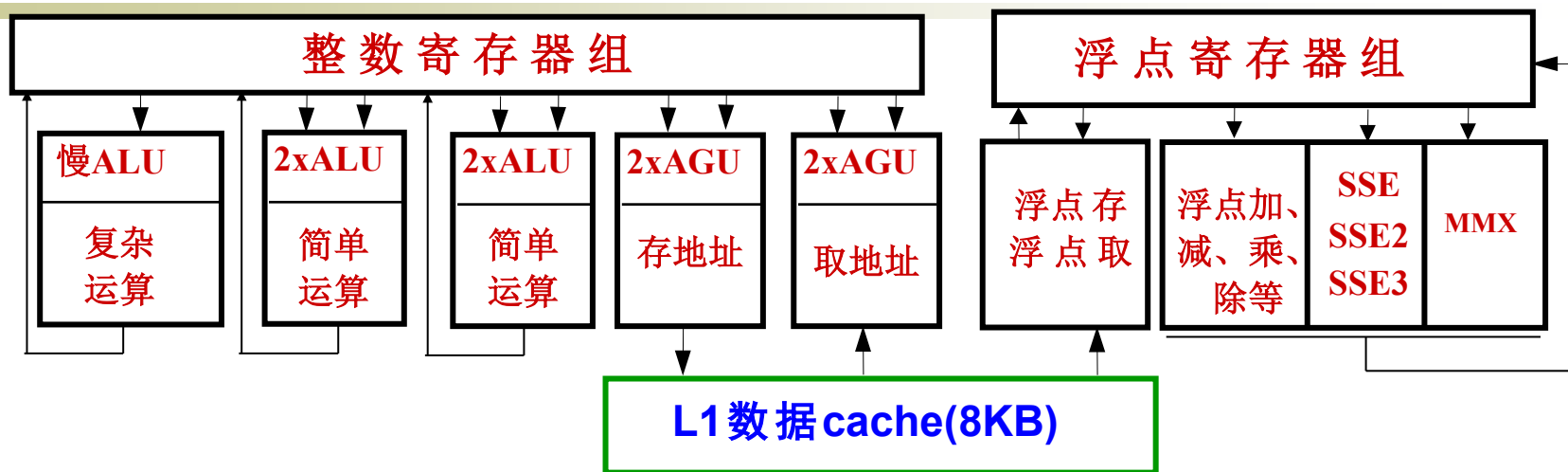
每个MOP相当于一RISC指令，格式没公开

BTB1静态预测从L2的何处预取指令





Pentium 4的超标量结构运算器



- 采用超标量（superscalar）结构，共含9个运算部件，可同时工作，所花时钟不同
 - 2个高速整数ALU(每个时钟周期进行2次操作)，用于完成简单的整数运算(如加/减法)
 - 1个慢速整数ALU(需要多个时钟周期才能完成1次操作)，用于完成整数乘、除法运算
 - 2个地址生成部件（AGU），用于计算操作数的有效地址，所生成的地址分别用于从内存取操作数或向内存保存操作结果
 - 1个运算部件用于完成浮点操作数地址的计算
 - 1个运算部件用于完成浮点加法、乘法和除法运算
 - 1个运算部件用于执行流式的SIMD处理（SSE/SSE2/SSE3指令）
 - 1个运算部件用于完成多媒体信号处理（MMX指令）

在运算部件中执行的是微操作，而不是指令！运算器中的操作采用流水方式！



Pentium 4 的用户可见寄存器组

整数寄存器组

在Pentium4内部，整数和浮点数各有128个物理寄存器

寄存器换名操作：将用户可见的逻辑寄存器换成内部的物理寄存器

寄存器换名时，要确定是真实依赖还是名字依赖（反依赖）

名字依赖时可用不同的物理寄存器替换相同的逻辑寄存器

P4 Pentium 80386 80486 8086 8088

指令计数器
标志寄存器

	31	15	8	7	0
GPR 0	EAX	AX	AH	AL	
GPR 1	ECX	CX	CH	CL	
GPR 2	EDX	DX	DH	DL	
GPR 3	EBX	BX	BH	BL	
GPR 4	ESP	SP			
GPR 5	EBP	BP			
GPR 6	ESI	SI			
GPR 7	EDI	DI			
	EIP	IP			
	EFLAGS	FLAGS			

浮点寄存器组

79	FPR 0	0
	FPR 1	
	FPR 2	
	FPR 3	
	FPR 4	
	FPR 5	
	FPR 6	
	FPR 7	



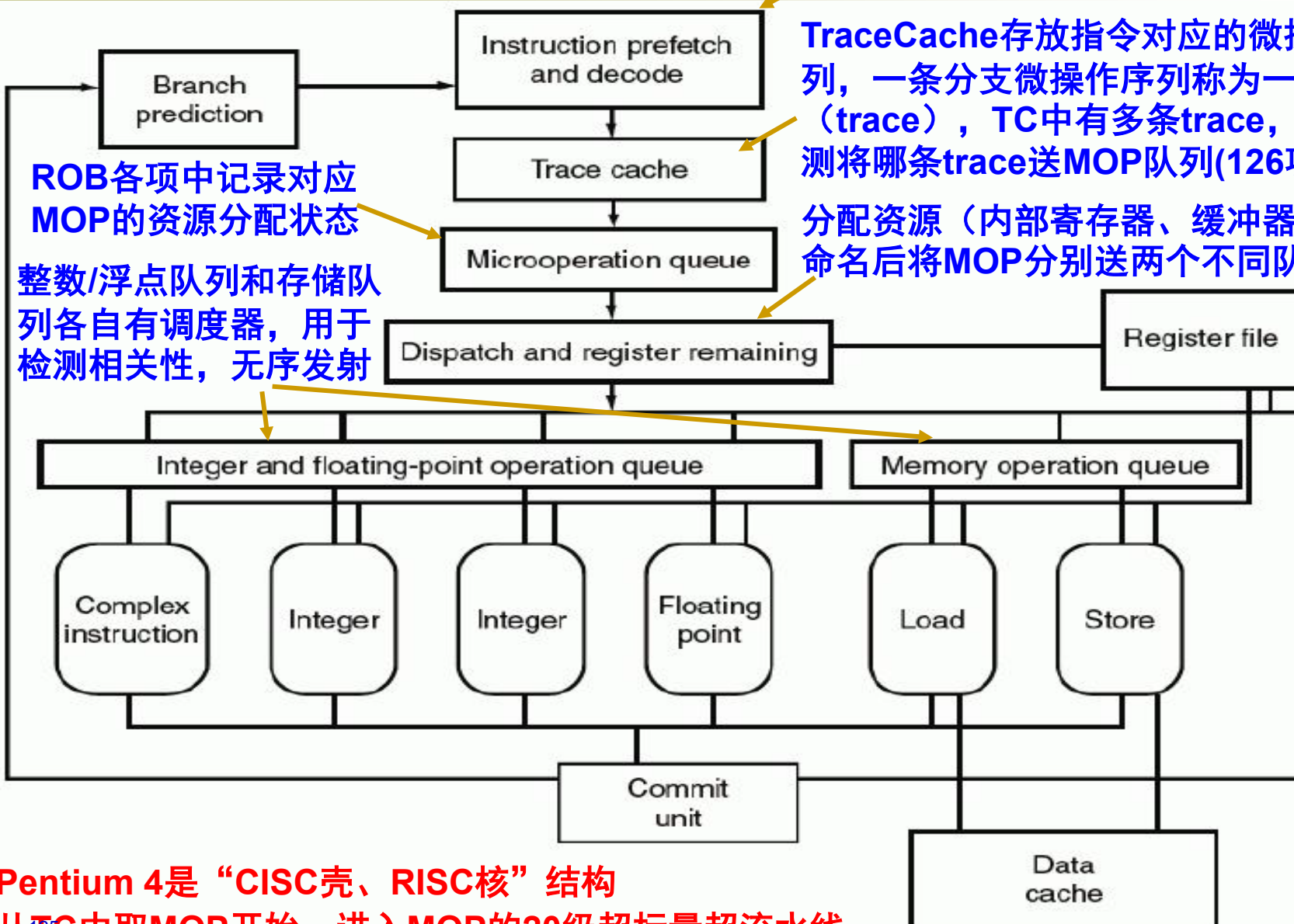
Pentium4 流水线结构部分

每个MOP相当于一RISC指令

译码结果是对应的若干条微操作MOP

TraceCache存放指令对应的微操作序列，一条分支微操作序列称为一个踪迹（trace），TC中有多条trace，动态预测将哪条trace送MOP队列(126项ROB)

分配资源（内部寄存器、缓冲器）并重命名后将MOP分别送两个不同队列



Pentium 4是“CISC壳、RISC核”结构

从TC中取MOP开始，进入MOP的20级超标量超流水线



第六讲小结

- 有以下两种指令级并行(ILP)技术 (即: 高性能流水线形式)
 - 超流水线: 更多的流水线级数
 - 多发射流水线: 同时发射多个指令, 有多条流水线同时进行
 - ◆ 静态多发射 (VLIW处理器+编译器静态调度)
 - ◆ 动态多发射 (超标量处理器+动态流水线调度)
- 静态多发射 (VLIW (超长指令字) 处理器)
 - 由编译器静态推测来完成“指令打包”和“冒险处理”
 - MIPS 2-发射Datapath中有2个执行部件, 将2条指令打包, 并同时译码执行
 - 采用循环展开进行指令调度, 能得到很好的性能
 - IA-64采用VLIW技术, Intel称其为EPIC技术, 3条指令打包
- 动态多发射 (超标量处理器)
 - 指令执行时由硬件动态推测, 多个执行部件, 同时发射多个指令到执行部件
 - 3种动态多发射流水线的执行模式
 - ◆ 按序发射按序完成、按序发射无序完成、无序发射无序完成



本章总结1

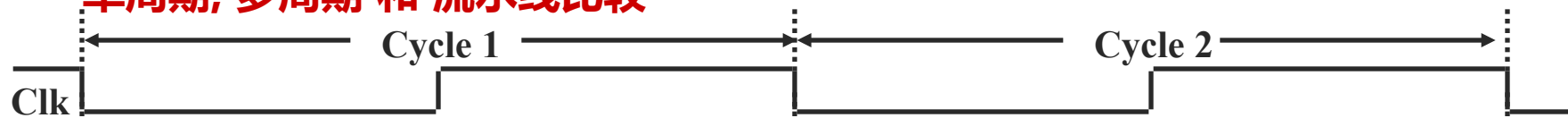
■ 指令流水线的设计

- 将每条指令的执行规整化为若干个同样的流水阶段
- 每个流水阶段的执行时间一样，都等于一个时钟
- 理想情况下，每个时钟有一条指令进入流水线，也有一条指令执行结束
- 每个流水段中的部件都是组合逻辑加寄存器，组合逻辑中产生的结果在时钟到来时被存储到寄存器（如：程序计数器、条件码寄存器、流水线寄存器等）。
- 每两个相邻流水段之间的流水线寄存器，用以记录所有在后面阶段要用到的各种信息，有哪些呢？
 - ◆ 指令的代码、本条指令地址(PC)、扩展后的立即数、参加运算的操作数、指令运算结果、指令异常信息、寄存器读口地址、寄存器写口地址(Rd)、存储器地址等。
- 指令译码得到的控制信号通过流水线寄存器传送到后面各个流水段中

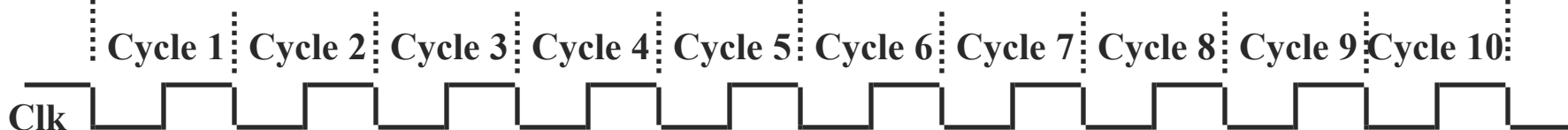


本章总结2

单周期, 多周期 和 流水线比较



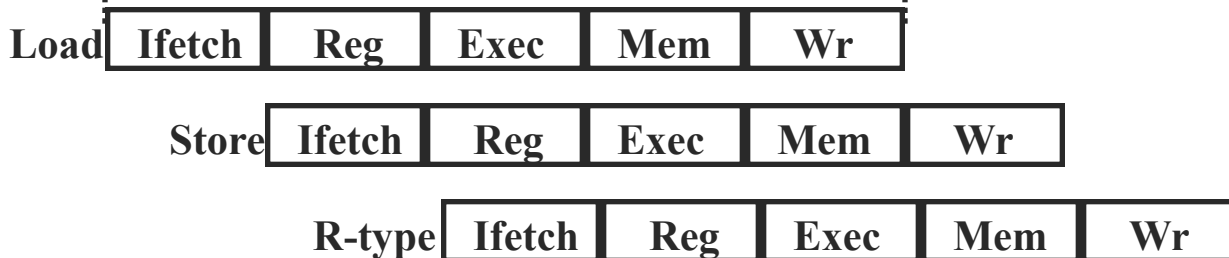
Single Cycle Implementation:



Multiple Cycle Implementation:



Pipeline Implementation:





本章总结3

- 指令流水线的局限性
 - 并不是每条指令都有相同多个流水段
 - 并不是每个流水段都一样长
 - 随着流水线深度的增加，流水线寄存器的额外开销比例也增大
 - 指令在资源冲突、数据相关或控制相关时会发生流水线冒险
- 指令流水线的执行效率
 - 吞吐率：比非流水线方式下大大提高
 - 指令执行时间：相对于非流水线方式，一条指令的执行时间延长了
- 提高流水线指令效率的高级流水线技术
 - 超流水线：级数更多的流水线
 - 多发射流水线：同时发射多条指令的流水线
 - ◆ 静态多发射：VLIW结构、编译器静态推测
 - ◆ 动态多发射：超标量结构、硬件动态推测调度



本章总结4

- 结构冒险（资源冲突）：多条指令同时使用同一个功能部件
 - 规定每个功能部件在一条指令中只能被用一次
 - 规定每个功能部件只能在某个特定的阶段被用
 - 指令存储器(Code Cache)和数据存储器(Data Cache)分开
- 数据冒险（数据相关）：前面指令的结果是后面指令的操作数
 - 软件阻塞：（如：编译器）在后面的数据相关指令前插入nop指令
 - 硬件阻塞：在后面数据相关指令的特定流水段插入“气泡”以“阻塞”指令继续执行，直到取得所需数据为止
 - “转发”（旁路）：把前面指令执行过程中得到的数据直接传送到后面指令相应的流水段中。
 - 对于load-use，采用“阻塞加转发”的方式解决数据冒险



本章总结5

- 控制冒险（控制相关）：返回指令、分支指令等可能改变顺序增量的PC值，由于获取转移目标地址的时间较长，使得在目标地址产生前已经有指令被取到流水线中，如果已经取出执行的指令不是正确的指令，则发生控制冒险。
 - 软件阻塞：（如：编译器）在控制相关指令后面插入nop指令
 - 硬件阻塞：在控制相关指令后面的指令被取出前插入“气泡”，使流水线停顿若干时钟，直到控制相关指令得到正确的PC值为止
 - 采用“分支预测”技术。简单（静态）地预测每次分支结果都一样，或根据分支指令执行历史进行动态预测，动态预测能达到90%以上的成功率
 - 采用延迟分支技术。将前面一条与分支指令无关的指令放到分支指令后面执行，这样，流水线不会发生阻塞现象。这种对指令顺序进行调整的工作在程序编译阶段完成
 - 异常和中断是一种特殊的控制冒险，在流水线中的异常/中断机制的实现具有挑战性